

SQL Advanced, Snowflake, AWS Services & BI

High performance. Delivered.

consulting | technology | outsourcing

Goals

• SQL & RDBMS Mastery

- Understand the **relational database model** and relationships between tables.
- Identify and use **Primary, Foreign, Candidate, Composite, and Unique keys**.
- Design database tables using appropriate **constraints and normalization principles**.
- Perform **DDL operations** using CREATE, ALTER, DROP, and TRUNCATE.
- Perform **DML operations** using INSERT, UPDATE, DELETE, and MERGE/UPSERT.
- Develop **stored procedures** with IN/OUT parameters, conditional logic, loops, and error handling.
- Create and use **scalar, table-valued, and built-in SQL functions**.

Goals

PySpark Training Objectives

- Create and manage **views and materialized views** for abstraction and performance.
- Apply **UNION, UNION ALL, INTERSECT, and EXCEPT** for combining query results.
- Perform data summarization using **SUM, AVG, COUNT, MAX, MIN, GROUP BY, and HAVING**.
- Implement complex data relationships using **INNER, LEFT, RIGHT, FULL OUTER, CROSS, and Self JOINS**.
- Apply advanced **window functions** including **ROW_NUMBER, RANK, DENSE_RANK, NTILE, LAG, LEAD, FIRST_VALUE, and LAST_VALUE**.
- Use **PARTITION BY, ORDER BY, and window frame specifications** such as **ROWS BETWEEN**.
- Build modular queries using **CTEs** and hierarchical queries using **recursive CTEs**.

Goals



PySpark Training Objectives

- Decide when to use **CTEs versus temporary tables**.
- Use analytical SQL techniques for reporting and business intelligence.
- Analyze **query execution plans** and identify performance bottlenecks.
- Optimize queries using appropriate **indexes, predicates, joins, and query hints**.
- Understand and use **clustered, non-clustered, B-tree, bitmap, and full-text indexes**.
- Understand database **ACID transaction properties**.
- Select appropriate transaction isolation levels: **READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, and SERIALIZABLE**.

Goals

PySpark Training Objectives

- **2. AWS QuickSight**

- Connect QuickSight to **S3, RDS, Athena, Redshift, Snowflake, and CSV datasets**.
- Understand **SPICE architecture** and use in-memory datasets for faster analytics.
- Manage **SPICE capacity, full refresh, and incremental refresh**.
- Create datasets and perform **joins, type conversions, column renaming, filters, and aggregations**.
- Build calculated fields using **arithmetic, conditional, string, date, and aggregation functions**.
- Prepare analytical datasets using **custom SQL and data transformations**.
- Create **bar, line, pie, scatter, heat map, pivot table, KPI, combo, and geospatial visualizations**.

• 2. AWS QuickSight

- Design professional **interactive dashboards** with multiple visualizations.
- Implement **filters, parameters, drill-downs, and visual actions**.
- Use parameters to implement **what-if analysis**.
- Configure **Row-Level Security (RLS)** for user- and group-specific data access.
- Optimize QuickSight datasets by reducing data volume and using **aggregated datasets and incremental refreshes**.
- Share dashboards with users and groups and configure **scheduled email reports**.
- Understand **viewer, author, and administrative permissions**.
- Organize dashboards and analyses using appropriate access-control mechanisms.
- Embed QuickSight dashboards into applications using **iframe and SDK-based embedding**.

Goals

Pyspark Training Objectives

- **3. Modern Data Lake Technologies**
 - Understand the purpose and architecture of **Delta Lake** and **Apache Iceberg**.
 - Compare traditional data lakes with **transactional lakehouse formats**.
 - Understand schema evolution, partitioning, metadata management, and ACID capabilities in modern lakehouse platforms.
- **4. AWS Lake Formation**
 - Understand the role of **AWS Lake Formation** in building and governing enterprise data lakes.
 - Configure centralized data access and permissions.
 - Integrate Lake Formation with **S3, Glue Data Catalog, Athena, and other AWS analytics services**.
 - Apply fine-grained data governance and security policies.

Goals

PySpark Training Objectives

- **5. AWS Athena**

- Understand Athena as a **serverless SQL query engine for S3**.
- Create schemas and tables over data stored in S3.
- Integrate Athena with the **AWS Glue Data Catalog**.
- Query **CSV, JSON, Parquet, ORC, and Avro** datasets.
- Implement **partitioning by date, region, or business dimensions** to reduce scanned data.
- Improve query performance using **Parquet/ORC, compression, partition pruning, and column pruning**.
- Understand the cost impact of **data scanned per query**.
- Avoid the **small-files problem** by designing efficient S3 file layouts.

Goals

PySpark Training Objectives

- **7. AWS Lambda**

- Develop and deploy **serverless Lambda functions**.
- Integrate Lambda with **S3, SQS, SNS, EventBridge, and API Gateway**.
- Understand the Lambda **handler and execution lifecycle**.
- Configure **reserved and provisioned concurrency**.
- Reduce **cold-start latency**.
- Configure memory, CPU, timeout, and environment variables.
- Understand the **15-minute maximum execution time**.
- Manage dependencies using **Lambda Layers**.
- Understand **synchronous versus asynchronous invocation**.
- Implement retries and robust error-handling strategies.
- Configure **Dead Letter Queues** and failure destinations.

Goals

PySpark Training Objectives

- **8. AWS SQS & SNS**

- Understand asynchronous messaging using **Amazon SQS**.
- Compare **Standard and FIFO queues**.
- Configure **visibility timeout** and message-processing behavior.
- Handle poison messages using **Dead Letter Queues**.
- Understand ordering, deduplication, scalability, and delivery guarantees.
- Implement pub/sub messaging using **Amazon SNS**.
- Create **topics and subscriptions**.
- Implement the **SNS fanout pattern** using multiple SQS queues.
- Integrate SQS and SNS with **AWS Lambda** for event-driven applications.

Goals

PySpark Training Objectives

- **9. AWS EMR**

- Understand **Amazon EMR architecture** for Hadoop and Spark workloads.
- Design clusters using **Primary/Master, Core, and Task nodes**.
- Select appropriate cluster sizes for data-processing workloads.
- Configure **autoscaling policies**.
- Tune Spark **executor memory, executor cores, and parallelism**.
- Optimize cluster utilization and application performance.
- Use **Spot Instances** appropriately to reduce processing cost.
- Decide when EMR is more appropriate than serverless alternatives.

Goals

PySpark Training Objectives

- **10. AWS Glue**

- Understand AWS Glue as a **serverless ETL platform**.
- Develop **Spark-based Glue ETL jobs and Python Shell jobs**.
- Use the **Glue Data Catalog** for centralized metadata.
- Configure **Glue Crawlers** for automatic schema discovery.
- Create Glue connections to databases and external systems.
- Compare **Glue DynamicFrames and Spark DataFrames**.
- Transform and clean data in Glue jobs.
- Implement incremental ETL using **Glue Job Bookmarks**.
- Build Glue-based pipelines that integrate **S3, Athena, Redshift, RDS, and other AWS services**.

- **11. Snowflake Architecture**

- Understand Snowflake's **cloud-native architecture**.
- Explain the separation of **storage, compute, and cloud services**.
- Understand how Snowflake supports independent compute scaling using **Virtual Warehouses**.
- Select warehouse sizes from **X-Small through larger warehouse configurations**.
- Configure **auto-suspend and auto-resume**.
- Use **multi-cluster warehouses** to handle concurrent workloads.
- Understand **query queueing and workload concurrency**.
- Analyze queries using the **Snowflake Query Profile**.

Goals

PySpark Training Objectives

- **12. Snowflake Database Objects**

- Create and manage **databases, schemas, tables, views, and materialized views.**
- Work with **permanent, temporary, and transient tables.**
- Understand when transient tables are appropriate for **ETL and staging.**
- Create **external tables** for data stored in S3 or other object stores.
- Work with Snowflake **string, date, numeric, semi-structured, and JSON functions.**
- Develop Snowflake **SQL and procedural logic.**

Goals

PySpark Training Objectives

- **13. Snowflake Performance**

- Understand Snowflake **micro-partitions**.
- Explain automatic partitioning and metadata collection.
- Use **micro-partition pruning** to reduce scanned data.
- Identify poorly clustered tables.
- Apply **clustering keys** when appropriate.
- Optimize queries using warehouse sizing and Query Profile analysis.

Goals

PySpark Training Objectives

- **14. Snowflake Data Protection**
 - Use **Time Travel** to query historical versions of data.
 - Recover accidentally deleted data using UNDROP.
 - Understand Time Travel retention periods for different table types and account editions.
 - Understand Snowflake **Fail-safe** and its role in disaster recovery.
 - Differentiate between **Time Travel** and **Fail-safe**.

Goals

PySpark Training Objectives

- **15. Snowflake CDC & Automation**

- Implement **Change Data Capture using Streams**.
- Process INSERT, UPDATE, and DELETE changes incrementally.
- Automate SQL processing using **Snowflake Tasks**.
- Configure **CRON-based schedules**.
- Build dependencies between Tasks to create **task DAGs**.
- Invoke stored procedures and SQL workloads from Tasks.



Goals

PySpark Training Objectives

- **16. Snowflake Data Ingestion**

- Implement continuous ingestion using **Snowpipe**.
- Configure automatic ingestion from **Amazon S3 events**.
- Understand Snowpipe's **continuous micro-batch processing model**.
- Design efficient data-loading patterns for high-volume datasets.

Goals

PySpark Training Objectives

- **17. Snowflake Advanced Capabilities**
 - Create **zero-copy clones** of tables, schemas, and databases.
 - Use cloning to create efficient **development, testing, and QA environments**.
 - Share data securely between Snowflake accounts using **Secure Data Sharing**.
 - Understand how Snowflake enables data sharing **without physically copying data**.

Goals

PySpark Training Objectives

- **18. Snowflake Security**
 - Implement **Role-Based Access Control (RBAC)**.
 - Design appropriate role and privilege hierarchies.
 - Understand encryption **at rest and in transit**.
 - Configure **network policies**.
 - Use **MFA and security controls** to protect Snowflake accounts.

Postgres Client in VSCode



postgresql://postgres:postgres@postgres:5432/bmwdb

Database

- localhost...
- Security
- bmwdb
 - Query
- public 40K
 - Query
 - Types
- Tables (1)
 - vehicle
- Foreign Tables
- Views
- Materialized Views
- Sequences
- Functions
- Procedures
- Triggers
- postgres

Welcome x Edit 127.0.0.1@5432 x

Note: Requires a [Premium License](#) to unlock all features. Database 1/3 , Other 0/3

Server Type

- MySQL
- MariaDB
- PostgreSQL
- SQLite
- SQL Server
- Db2
- Oracle
- Kingbase
- ClickHouse
- JDBC
- SSH
- Docker
- Redis
- ElasticSearch
- MongoDB
- S3
- FTP
- Kafka
- RabbitMQ
- Cassandra
- More

Config Main SSH Tunnel Socks Proxy HTTP Proxy

* Host localhost * Port 5432

* Username potsgres * Password

Database bmwdb Advanced

Features ☒ Type ☒ Trigger ☒ Sequence ☒ Foreign Table ☒ Materialized View

Use Connection String ☒ SSL ☐

* Connection String postgresql://postgres:postgres@localhost:5432/bmwdb Use

Save + Connect X Close



PostgreSQL Strengths

- **Advanced SQL Compliance:** Full support for SQL standard, including window functions, CTEs(Common Table Expressions), and more.
- **Extensibility:** You can define custom data types, operators, functions, and index methods.
- **Strong JSON/JSONB:** Can be used for hybrid NoSQL use-cases.
- **Rich Indexing:** Supports B-tree, GiST, GIN, BRIN, Hash, covering a wide range of use-cases.
- **MVCC:** Superior concurrency without locking readers.
- **Open Source & Free for Commercial Use:** No licensing restrictions.

When to Choose What?

Use Case	Recommended DB	Reason
Web applications needing performance + ease	MySQL	Fast, easy, well-supported in LAMP stacks
Enterprise-grade systems with complex logic	Oracle DB	Strong security, clustering, and tooling
Windows-heavy enterprise systems	SQL Server	Good .NET integration, SSIS/SSRS tools
Research, spatial/geospatial apps	PostgreSQL (PostGIS)	Best GIS support, complex query support
Mobile or embedded applications	SQLite	Lightweight, file-based, no server needed
Cloud-native or analytics-heavy systems	PostgreSQL	Modern features, cloud support, and analytics

Security & Role Management

DB	Role Management	Row-Level Security	Encryption
PostgreSQL	✓ Roles, schemas	✓ Built-in	✓ SSL, TDE via extensions
MySQL	✓ Users, privileges	✗ Limited	✓ SSL only
Oracle	✓ Advanced	✓ VPD, Label Sec	✓ TDE, Data Redaction
SQL Server	✓ Windows/SQL Auth	✓ RLS	✓ TDE, Always Encrypted
SQLite	✗ File-based	✗ None	✗ Application-level only



Key Features of PostgreSQL

- **1. ACID Compliance**
 - Guarantees **Atomicity, Consistency, Isolation, and Durability**.
 - Ensures reliable and safe transactions.
- **2. MVCC (Multi-Version Concurrency Control)**
 - Allows **concurrent reads and writes** without locking.
 - Improves performance in high-concurrency environments.



Key Features of PostgreSQL

- **3. Advanced SQL Support**
- Full support for:
 - **Window Functions**
 - **Common Table Expressions (CTEs)**
 - **Subqueries, Joins, and Set operations**
 - **Stored procedures, triggers, and views**



Key Features of PostgreSQL

- **4. Extensibility**
- Add custom:
 - Data types
 - Functions (in PL/pgSQL, Python, Perl, etc.)
 - Operators
 - Indexing methods
 - Procedural languages



Key Features of PostgreSQL

- **5. Rich Data Types**
- Native support for:
 - Numeric, Integer, Boolean, Text, Date/Time
 - Arrays
 - JSON / JSONB (binary-optimized JSON)
 - hstore (key-value pairs)
 - Geometric types
 - UUID, XML, and more



Key Features of PostgreSQL

- **6. Indexing Options**
- Powerful indexing support:
 - **B-tree** (default)
 - **Hash**
 - **GIN** (for full-text search and JSON)
 - **GiST** (for spatial data)
 - **BRIN** (for large sorted datasets)
 - **Expression & Partial indexes**



Key Features of PostgreSQL

- 7. Full Text Search
 - Built-in full-text search capabilities using GIN/GiST indexes.
 - Ranking, highlighting, stemming, and tokenization.
- 8. JSON & NoSQL Support
 - Native JSON and JSONB data types.
 - Querying, indexing, and storing semi-structured data.
 - Acts as a hybrid SQL + NoSQL database.



Key Features of PostgreSQL

- 9. Data Integrity and Constraints
 - Supports:
 - Primary, foreign, and unique keys
 - Check constraints
 - Exclusion constraints (custom conflict rules)
- 10. Partitioning
 - Declarative partitioning on range/list/hash keys.
 - Efficient data management and performance for large tables.



Key Features of PostgreSQL

- **11. Replication & High Availability**
- Built-in:
 - **Streaming replication**
 - **Logical replication**
 - **Hot standby, Point-in-Time Recovery (PITR)**
- Tools: **pgBackRest, Barman, Patroni, PgBouncer**



Key Features of PostgreSQL

- **12. Security Features**

- Role-based authentication and permissions
- SSL encryption
- **Row-Level Security (RLS)**
- Integration with LDAP, GSSAPI, PAM



Key Features of PostgreSQL

- **13. Stored Procedures & Triggers**
 - Functions written in PL/pgSQL or other languages
 - BEFORE, AFTER, INSTEAD OF triggers
 - Event-driven automation and data integrity enforcement
- **14. Internationalization & Localization**
 - Unicode support
 - Collation and case-sensitivity customization
 - Multilingual data storage



Key Features of PostgreSQL

- **15. Tools and Ecosystem**
 - GUI: pgAdmin, Dbeaver
 - CLI: psql, pg_dump, pg_restore, vacuumdb
 - Extensions: PostGIS, pg_stat_statements, TimescaleDB
- **16. Open Source with Strong Community**
 - PostgreSQL Global Development Group
 - Active mailing lists, regular updates, and transparent roadmap
 - Free for commercial and personal use

Writing Simple Queries

- Creating a Database
 - To create a new database, in this example named mydb, you use the following command:
 - `$ create database mydb;`
- Accessing a Database
 - Running the PostgreSQL interactive terminal program, called psql, which allows you to interactively enter, edit, and execute SQL commands.
 - Using an existing graphical frontend tool like pgAdmin or an office suite with ODBC or JDBC support to create and manipulate a database.



Writing Simple Queries

- Accessing a Database
 - \c mydb;

```
mydb=> SELECT version();
```

```
version
```

```
-----  
-----  
PostgreSQL 13.2 on x86_64-pc-linux-gnu, compiled by gcc (Debian  
4.9.2-10) 4.9.2, 64-bit  
(1 row)
```

```
mydb=> SELECT current_date;  
date
```

```
-----  
2016-01-07  
(1 row)
```

```
mydb=> SELECT 2 + 2;  
?column?
```

```
-----  
4  
(1 row)
```



Writing Simple Queries

The `psql` program has a number of internal commands that are not SQL commands. They begin with the backslash character, “\”. For example, you can get help on the syntax of various PostgreSQL SQL commands by typing:

```
mydb=> \h
```

To get out of `psql`, type:

```
mydb=> \q
```

Data Types

Name	Aliases	Description
<code>bigint</code>	<code>int8</code>	signed eight-byte integer
<code>bigserial</code>	<code>serial8</code>	autoincrementing eight-byte integer
<code>bit [(n)]</code>		fixed-length bit string
<code>bit varying [(n)]</code>	<code>varbit</code> <code>[(n)]</code>	variable-length bit string
<code>boolean</code>	<code>bool</code>	logical Boolean (true/false)
<code>box</code>		rectangular box on a plane
<code>bytea</code>		binary data (“byte array”)
<code>character [(n)]</code>	<code>char [(n)]</code>	fixed-length character string
<code>character varying [(n)]</code>	<code>varchar</code> <code>[(n)]</code>	variable-length character string
<code>cidr</code>		IPv4 or IPv6 network address
<code>circle</code>		circle on a plane
<code>date</code>		calendar date (year, month, day)
<code>double precision</code>	<code>float8</code>	double precision floating-point number (8 bytes)
<code>inet</code>		IPv4 or IPv6 host address
<code>integer</code>	<code>int</code> , <code>int4</code>	signed four-byte integer

Data Types

<code>interval [fields] [(p)]</code>		time span
<code>json</code>		textual JSON data
<code>jsonb</code>		binary JSON data, decomposed
<code>line</code>		infinite line on a plane
<code>lseg</code>		line segment on a plane
<code>macaddr</code>		MAC (Media Access Control) address
<code>macaddr8</code>		MAC (Media Access Control) address (EUI-64 format)
<code>money</code>		currency amount
<code>numeric [(p, s)]</code>	<code>decimal [(p, s)]</code>	exact numeric of selectable precision
<code>path</code>		geometric path on a plane
<code>pg_lsn</code>		PostgreSQL Log Sequence Number
<code>pg_snapshot</code>		user-level transaction ID snapshot
<code>point</code>		geometric point on a plane

Data Types

Name	Aliases	Description
<code>polygon</code>		closed geometric path on a plane
<code>real</code>	<code>float4</code>	single precision floating-point number (4 bytes)
<code>smallint</code>	<code>int2</code>	signed two-byte integer
<code>smallserial</code>	<code>serial2</code>	autoincrementing two-byte integer
<code>serial</code>	<code>serial4</code>	autoincrementing four-byte integer
<code>text</code>		variable-length character string
<code>time [(p)] [without time zone]</code>		time of day (no time zone)
<code>time [(p)] with time zone</code>	<code>timetz</code>	time of day, including time zone
<code>timestamp [(p)] [without time zone]</code>		date and time (no time zone)
<code>timestamp [(p)] with time zone</code>	<code>timestampz</code>	date and time, including time zone
<code>tsquery</code>		text search query
<code>tsvector</code>		text search document
<code>txid_snapshot</code>		user-level transaction ID snapshot (deprecated; see <code>pg_snapshot</code>)
<code>uuid</code>		universally unique identifier
<code>xml</code>		XML data



Numeric Types

Name	Storage Size	Description	Range
<code>smallint</code>	2 bytes	small-range integer	-32768 to +32767
<code>integer</code>	4 bytes	typical choice for integer	-2147483648 to +2147483647
<code>bigint</code>	8 bytes	large-range integer	-9223372036854775808 to +9223372036854775807
<code>decimal</code>	variable	user-specified precision, exact	up to 131072 digits before the decimal point; up to 16383 digits after the decimal point
<code>numeric</code>	variable	user-specified precision, exact	up to 131072 digits before the decimal point; up to 16383 digits after the decimal point
<code>real</code>	4 bytes	variable-precision, inexact	6 decimal digits precision
<code>double precision</code>	8 bytes	variable-precision, inexact	15 decimal digits precision
<code>smallserial</code>	2 bytes	small autoincrementing integer	1 to 32767
<code>serial</code>	4 bytes	autoincrementing integer	1 to 2147483647
<code>bigserial</code>	8 bytes	large autoincrementing integer	1 to 9223372036854775807

Monetary Types

Name	Storage Size	Description	Range
money	8 bytes	currency amount	-92233720368547758.08 to +92233720368547758.07



Binary Input Type

Name	Storage Size	Description
bytea	1 or 4 bytes plus the actual binary string	variable-length binary string

Character Types

Name	Description
<code>character varying (n)</code> , <code>varchar (n)</code>	variable-length with limit
<code>character (n)</code> , <code>char (n)</code>	fixed-length, blank padded
<code>text</code>	variable unlimited length

Boolean Type



Name	Storage Size	Description
boolean	1 byte	state of true or false

Date/Time Types

Name	Storage Size	Description	Low Value	High Value	Resolution
timestamp [(p)] [without time zone]	8 bytes	both date and time (no time zone)	4713 BC	294276 AD	1 microsecond
timestamp [(p)] with time zone	8 bytes	both date and time, with time zone	4713 BC	294276 AD	1 microsecond
date	4 bytes	date (no time of day)	4713 BC	5874897 AD	1 day
time [(p)] [without time zone]	8 bytes	time of day (no date)	00:00:00	24:00:00	1 microsecond
time [(p)] with time zone	12 bytes	time of day (no date), with time zone	00:00:00+1559	24:00:00-1559	1 microsecond
interval [fields] [(p)]	16 bytes	time interval	-178000000 years	178000000 years	1 microsecond

Date/Time Types

Example	Description
1999-01-08	ISO 8601; January 8 in any mode (recommended format)
January 8, 1999	unambiguous in any <code>datestyle</code> input mode
1/8/1999	January 8 in MDY mode; August 1 in DMY mode
1/18/1999	January 18 in MDY mode; rejected in other modes
01/02/03	January 2, 2003 in MDY mode; February 1, 2003 in DMY mode; February 3, 2001 in YMD mode
1999-Jan-08	January 8 in any mode

Time Input

Example	Description
04:05:06.789	ISO 8601
04:05:06	ISO 8601
04:05	ISO 8601
040506	ISO 8601
04:05 AM	same as 04:05; AM does not affect value
04:05 PM	same as 16:05; input hour must be <= 12
04:05:06.789-8	ISO 8601
04:05:06-08:00	ISO 8601
04:05-08:00	ISO 8601
040506-08	ISO 8601
04:05:06 PST	time zone specified by abbreviation
2003-04-12 04:05:06 America/New_York	time zone specified by full name

Oracle Data Types vs Postgre Data Types

Oracle type	Possible PostgreSQL types
CHAR	char, varchar, text
NCHAR	char, varchar, text
VARCHAR	char, varchar, text
VARCHAR2	char, varchar, text, json
NVARCHAR2	char, varchar, text
CLOB	char, varchar, text, json
LONG	char, varchar, text
RAW	uuid, bytea
BLOB	bytea
BFILE	bytea (read-only)
LONG RAW	bytea
NUMBER	numeric, float4, float8, char, varchar, text
NUMBER(n,m) with m<=0	numeric, float4, float8, int2, int4, int8,
	boolean, char, varchar, text
FLOAT	numeric, float4, float8, char, varchar, text

Oracle Data Types vs Postgre Data Types

FLOAT	numeric, float4, float8, char, varchar, text
BINARY_FLOAT	numeric, float4, float8, char, varchar, text
BINARY_DOUBLE	numeric, float4, float8, char, varchar, text
DATE	date, timestamp, timestamptz, char, varchar, text
TIMESTAMP	date, timestamp, timestamptz, char, varchar, text
TIMESTAMP WITH TIME ZONE	date, timestamp, timestamptz, char, varchar, text
TIMESTAMP WITH LOCAL TIME ZONE	
INTERVAL YEAR TO MONTH	interval, char, varchar, text
INTERVAL DAY TO SECOND	interval, char, varchar, text
MDSYS.SDO_GEOMETRY	geometry (see “PostGIS support” below)

Enumerated Type

Declaration of Enumerated Types

Enum types are created using the CREATE TYPE command, for example:

```
CREATE TYPE mood AS ENUM ('sad', 'ok', 'happy');
```

Once created, the enum type can be used in table and function definitions much like any other type:

```
CREATE TYPE mood AS ENUM ('sad', 'ok', 'happy');
CREATE TABLE person (
    name text,
    current_mood mood
);
INSERT INTO person VALUES ('Moe', 'happy');
SELECT * FROM person WHERE current_mood = 'happy';
 name | current_mood
-----+-----
  Moe  |    happy
(1 row)
```


Geometric Types

Name	Storage Size	Description	Representation
point	16 bytes	Point on a plane	(x,y)
line	32 bytes	Infinite line	{A,B,C}
lseg	32 bytes	Finite line segment	((x1,y1),(x2,y2))
box	32 bytes	Rectangular box	((x1,y1),(x2,y2))
path	16+16n bytes	Closed path (similar to polygon)	((x1,y1),...)
path	16+16n bytes	Open path	[(x1,y1),...]
polygon	40+16n bytes	Polygon (similar to closed path)	((x1,y1),...)
circle	24 bytes	Circle	<(x,y),r> (center point and radius)



Geometric Types

```
CREATE EXTENSION postgis;
```

```
CREATE TABLE geometries (name varchar, geom geometry);
```

```
INSERT INTO geometries VALUES
```

```
  ('Point', 'POINT(0 0)'),
```

```
  ('Linestring', 'LINESTRING(0 0, 1 1, 2 1, 2 2)'),
```

```
  ('Polygon', 'POLYGON((0 0, 1 0, 1 1, 0 1, 0 0))'),
```

```
  ('PolygonWithHole', 'POLYGON((0 0, 10 0, 10 10, 0 10, 0 0),(1 1, 1 2, 2 2, 2 1, 1 1))'),
```

```
  ('Collection', 'GEOMETRYCOLLECTION(POINT(2 0),POLYGON((0 0, 1 0, 1 1, 0 1, 0 0)))');
```

```
SELECT name, ST_AsText(geom) FROM geometries;
```

Network Address Types

Name	Storage Size	Description
<code>cidr</code>	7 or 19 bytes	IPv4 and IPv6 networks
<code>inet</code>	7 or 19 bytes	IPv4 and IPv6 hosts and networks
<code>macaddr</code>	6 bytes	MAC addresses
<code>macaddr8</code>	8 bytes	MAC addresses (EUI-64 format)



Network Address Types

```
CREATE TABLE inet_test (  
    address INET  
);  
  
INSERT INTO inet_test (address) VALUES  
('198.24.10.0/24');  
  
INSERT INTO inet_test (address) VALUES  
('198.24.10.0');  
  
INSERT INTO inet_test (address) VALUES ('198.10/8');
```

cidr Type Input

cidr Input	cidr Output	abbrev(cidr)
192.168.100.128/25	192.168.100.128/25	192.168.100.128/25
192.168/24	192.168.0.0/24	192.168.0/24
192.168/25	192.168.0.0/25	192.168.0.0/25
192.168.1	192.168.1.0/24	192.168.1/24
192.168	192.168.0.0/24	192.168.0/24
128.1	128.1.0.0/16	128.1/16
128	128.0.0.0/16	128.0/16
128.1.2	128.1.2.0/24	128.1.2/24
10.1.2	10.1.2.0/24	10.1.2/24
10.1	10.1.0.0/16	10.1/16
10	10.0.0.0/8	10/8
10.1.2.3/32	10.1.2.3/32	10.1.2.3/32
2001:4f8:3:ba::/64	2001:4f8:3:ba::/64	2001:4f8:3:ba/64
2001:4f8:3:ba: 2e0:81ff:fe22:d1f1/128	2001:4f8:3:ba: 2e0:81ff:fe22:d1f1/128	2001:4f8:3:ba: 2e0:81ff:fe22:d1f1/128
::ffff:1.2.3.0/120	::ffff:1.2.3.0/120	::ffff:1.2.3/120
::ffff:1.2.3.0/128	::ffff:1.2.3.0/128	::ffff:1.2.3.0/128

Bit String Types

- Bit strings are strings of 1's and 0's. They can be used to store or visualize bit masks.
- There are two SQL bit types: `bit(n)` and `bit varying(n)`, where `n` is a positive integer.
- `bit` type data must match the length `n` exactly; it is an error to attempt to store shorter or longer bit strings.
- `bit varying` data is of variable length up to the maximum length `n`; longer strings will be rejected.
- Writing `bit` without a length is equivalent to `bit(1)`, while `bit varying` without a length specification means unlimited length.

Bit String Types

```
CREATE TABLE test (a BIT(3), b BIT VARYING(5));  
INSERT INTO test VALUES (B'101', B'00');  
INSERT INTO test VALUES (B'10', B'101');
```

ERROR: bit string length 2 does not match type bit(3)

```
INSERT INTO test VALUES (B'10'::bit(3), B'101');  
SELECT * FROM test;
```

UUID Type

- The data type `uuid` stores Universally Unique Identifiers (UUID) as defined by RFC 4122, ISO/IEC 9834-8:2005, and related standards.
- (This identifier is a 128-bit quantity that is generated by an algorithm chosen to make it very unlikely that the same identifier will be generated by anyone else in the known universe using the same algorithm.
- Therefore, for distributed systems, these identifiers provide a better uniqueness guarantee than sequence generators, which are only unique within a single database.

UUID Type

- PostgreSQL also accepts the following alternative forms for input: use of upper-case digits, the standard format surrounded by braces, omitting some or all hyphens, adding a hyphen after any group of four digits.
- Examples are:
- A0EEBC99-9C0B-4EF8-BB6D-6BB9BD380A11
- {a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11}
- a0eebc999c0b4ef8bb6d6bb9bd380a11
- a0ee-bc99-9c0b-4ef8-bb6d-6bb9-bd38-0a11
- {a0eebc99-9c0b4ef8-bb6d6bb9-bd380a11}

UUID Type

```
CREATE EXTENSION IF NOT EXISTS "uuid-oss";  
SELECT uuid_generate_v1();  
SELECT uuid_generate_v4();  
CREATE TABLE contacts (  
    contact_id uuid DEFAULT uuid_generate_v4 (),  
    first_name VARCHAR NOT NULL,  
    last_name VARCHAR NOT NULL,  
    email VARCHAR NOT NULL,  
    phone VARCHAR,  
    PRIMARY KEY (contact_id)  
);
```



UUID Type

```
INSERT INTO contacts (  
    first_name,  
    last_name,  
    email,  
    phone  
)  
VALUES  
(  
    'John',  
    'Smith',  
    'john.smith@example.com',  
    '408-237-2345'  
)
```



UUID Type

```
(  
    'Jane',  
    'Smith',  
    'jane.smith@example.com',  
    '408-237-2344'  
)  
(  
    'Alex',  
    'Smith',  
    'alex.smith@example.com',  
    '408-237-2343'  
);
```



XML Type

- The xml data type can be used to store XML data.
- Its advantage over storing XML data in a text field is that it checks the input values for well-formedness, and there are support functions to perform type-safe operations on it;
- Use of this data type requires the installation to have been built with `configure --with-libxml`.

XML Type

- The `xml` type can store well-formed “documents”, as defined by the XML standard, as well as “content” fragments, which are defined by reference to the more permissive “document node”¹ of the XQuery and XPath data model.
- Roughly, this means that content fragments can have more than one top-level element or character node.
- The expression `xmlvalue IS DOCUMENT` can be used to evaluate whether a particular `xml` value is a full document or only a content fragment.

XML Type

To produce a value of type `xml` from character data, use the function `xmlparse`:

```
XMLPARSE ( { DOCUMENT | CONTENT } value)
```

Examples:

```
XMLPARSE (DOCUMENT '<?xml version="1.0"?><book><title>Manual</title><chapter>...</chapter></book>')  
XMLPARSE (CONTENT 'abc<foo>bar</foo><bar>foo</bar>')
```

While this is the only way to convert character strings into XML values according to the SQL standard, the PostgreSQL-specific syntaxes:

```
xml '<foo>bar</foo>'  
'<foo>bar</foo>'::xml
```

can also be used.



Xml Type

The `xml` type does not validate input values against a document type declaration (DTD), even when the input value specifies a DTD. There is also currently no built-in support for validating against other XML schema languages such as XML Schema.

The inverse operation, producing a character string value from `xml`, uses the function `xmlserialize`:

```
XMLSERIALIZE ( { DOCUMENT | CONTENT } value AS type )
```

`type` can be `character`, `character varying`, or `text` (or an alias for one of those). Again, according to the SQL standard, this is the only way to convert between type `xml` and character types, but PostgreSQL also allows you to simply cast the value.

When a character string value is cast to or from type `xml` without going through `XMLPARSE` or `XMLSERIALIZE`, respectively, the choice of `DOCUMENT` versus `CONTENT` is determined by the “XML option” session configuration parameter, which can be set using the standard command:

```
SET XML OPTION { DOCUMENT | CONTENT } ;
```

or the more PostgreSQL-like syntax

```
SET xmloption TO { DOCUMENT | CONTENT } ;
```

The default is `CONTENT`, so all forms of XML data are allowed.



JSON Types

- JSON data types are for storing JSON (JavaScript Object Notation) data, as specified in RFC 71592.
- Such data can also be stored as text, but the JSON data types have the advantage of enforcing that each stored value is valid according to the JSON rules.
- There are also assorted JSON-specific functions and operators available for data stored in these data types;
- PostgreSQL offers two types for storing JSON data: json and jsonb.

JSON Types

- The json and jsonb data types accept almost identical sets of values as input.
- The major practical difference is one of efficiency.
- The json data type stores an exact copy of the input text, which processing functions must reparse on each execution; while jsonb data is stored in a decomposed binary format that makes it slightly slower to input due to added conversion overhead, but significantly faster to process, since no reparsing is needed.
- jsonb also supports indexing, which can be a significant advantage.

JSON Types

- Because the json type stores an exact copy of the input text, it will preserve semantically-insignificant white space between tokens, as well as the order of keys within JSON objects.
- Also, if a JSON object within the value contains the same key more than once, all the key/value pairs are kept.
- By contrast, jsonb does not preserve white space, does not preserve the order of object keys, and does not keep duplicate object keys.
- If duplicate keys are specified in the input, only the last value is kept.
- In general, most applications should prefer to store JSON data as jsonb, unless there are quite specialized needs, such as legacy assumptions about ordering of object keys.

JSON Types



JSON primitive type	PostgreSQL type	Notes
string	text	\u0000 is disallowed, as are Unicode escapes representing characters not available in the database encoding
number	numeric	NaN and infinity values are disallowed
boolean	boolean	Only lowercase true and false spellings are accepted
null	(none)	SQL NULL is a different concept

Arrays

```
CREATE TABLE sal_emp (  
    name                text,  
    pay_by_quarter      integer[],  
    schedule            text[]  
);
```

```
INSERT INTO sal_emp  
VALUES ('Bill',  
    '{10000, 10000, 10000, 10000}',  
    '{{"meeting", "lunch"}, {"training", "presentation"}}');
```

```
INSERT INTO sal_emp  
VALUES ('Carol',  
    '{20000, 25000, 25000, 25000}',  
    '{{"breakfast", "consulting"}, {"meeting", "lunch"}}');
```

Sequence

- CREATE [TEMPORARY | TEMP] SEQUENCE [IF NOT EXISTS] name
- [AS data_type]
- [INCREMENT [BY] increment]
- [MINVALUE minvalue | NO MINVALUE] [MAXVALUE maxvalue | NO MAXVALUE]
- [START [WITH] start] [CACHE cache] [[NO] CYCLE]
- [OWNED BY { table_name.column_name | NONE }]

Sequence Parameters

- **TEMPORARY** or **TEMP** — PostgreSQL can create a temporary sequence within a session. Once the session ends, the sequence is automatically dropped.
- **IF NOT EXISTS** — Creates a sequence even if a sequence with an identical name already exists. Replaces the existing sequence.
- **AS** — A new option in PostgreSQL 10. It is for specifying the data type of the sequence. The available options are `smallint`, `integer`, and `bigint` (default). This also determines the maximum and minimum values.
- **INCREMENT BY** — An optional parameter with a default value of 1. Positive values generate sequence values in ascending order. Negative values generate sequence values in descending sequence.

Sequence Parameters

- **START WITH** — The same as Oracle. This is an optional parameter having a default of 1. It uses the MINVALUE for ascending sequences and the MAXVALUE for descending sequences.
- **MAXVALUE | NO MAXVALUE** — Defaults are between 263 for ascending sequences and -1 for descending sequences.
- **MINVALUE | NO MINVALUE** — Defaults are between 1 for ascending sequences and -263 for descending sequences.
- **CYCLE | NO CYCLE** — If the sequence value reaches MAXVALUE or MINVALUE, the CYCLE parameter instructs the sequence to return to the initial value (MINVALUE or MAXVALUE). The default is NO CYCLE.

Sequence Parameters

- **CACHE** — Note that in PostgreSQL, the **NOCACHE** isn't supported. By default, when not specifying the **CACHE** parameter, no sequence values will be pre-cached into memory, which is equivalent to the Oracle **NOCACHE** parameter. The minimum value is 1.
- **OWNED BY | OWNBY NON** — Specifies that the sequence object is to be associated with a specific column in a table, which isn't supported by Oracle. When dropping this type of sequence, an error will be returned because of the sequence/table association.

Arrays

The result of the previous two inserts looks like this:

```
SELECT * FROM sal_emp;
  name |          pay_by_quarter          |          schedule
-----+-----
+-----+-----
  Bill | {10000,10000,10000,10000} | {{meeting,lunch},
{training,presentation}}
  Carol | {20000,25000,25000,25000} | {{breakfast,consulting},
{meeting,lunch}}
(2 rows)
```



Accessing Arrays

```
SELECT name FROM sal_emp WHERE pay_by_quarter[1] <>
pay_by_quarter[2];
```

```
name
-----
Carol
(1 row)
```

```
SELECT pay_by_quarter[3] FROM sal_emp;
```

```
pay_by_quarter
-----
                10000
                25000
(2 rows)
```

```
SELECT schedule[1:2][1:1] FROM sal_emp WHERE name = 'Bill';
```

```
schedule
-----
{{meeting},{training}}
(1 row)
```

Accessing Arrays

```
SELECT schedule[1:2][2] FROM sal_emp WHERE name = 'Bill';
```

It is possible to omit the *lower-bound* and/or *upper-bound* of a slice specifier; the missing bound is replaced by the lower or upper limit of the array's subscripts. For example:

```
SELECT schedule[:2][2:] FROM sal_emp WHERE name = 'Bill';
```

```
      schedule
-----
{{lunch},{presentation}}
(1 row)
```

```
SELECT schedule[:] [1:1] FROM sal_emp WHERE name = 'Bill';
```

```
      schedule
-----
{{meeting},{training}}
(1 row)
```

Accessing Arrays

```
SELECT array_dims(schedule) FROM sal_emp WHERE name = 'Carol';
```

```
array_dims
-----
[1:2] [1:2]
(1 row)
```

```
SELECT array_length(schedule, 1) FROM sal_emp WHERE name = 'Carol';
```

```
array_length
-----
2
(1 row)
```

`cardinality` returns the total number of elements in an array across all dimensions. It is effectively the number of rows a call to `unnest` would yield:

```
SELECT cardinality(schedule) FROM sal_emp WHERE name = 'Carol';
```

```
cardinality
-----
4
(1 row)
```

Modifying Arrays

```
UPDATE sal_emp SET pay_by_quarter = '{25000,25000,27000,27000}'  
WHERE name = 'Carol';
```

or using the ARRAY expression syntax:

```
UPDATE sal_emp SET pay_by_quarter = ARRAY[25000,25000,27000,27000]  
WHERE name = 'Carol';
```

An array can also be updated at a single element:

```
UPDATE sal_emp SET pay_by_quarter[4] = 15000  
WHERE name = 'Bill';
```

or updated in a slice:

```
UPDATE sal_emp SET pay_by_quarter[1:2] = '{27000,27000}'  
WHERE name = 'Carol';
```

Modifying Arrays

```
SELECT ARRAY[1,2] || ARRAY[3,4];
?column?
-----
{1,2,3,4}
(1 row)
```

```
SELECT ARRAY[5,6] || ARRAY[[1,2],[3,4]];
?column?
-----
{{5,6},{1,2},{3,4}}
(1 row)
```

```
SELECT array_dims(ARRAY[1,2] || ARRAY[3,4,5]);
array_dims
-----
[1:5]
(1 row)
```

```
SELECT array_dims(ARRAY[[1,2],[3,4]] || ARRAY[[5,6],[7,8],[9,0]]);
array_dims
-----
[1:5][1:2]
(1 row)
```

Modifying Arrays

```
SELECT array_prepend(1, ARRAY[2,3]);
       array_prepend
-----
 {1,2,3}
(1 row)
```

```
SELECT array_append(ARRAY[1,2], 3);
       array_append
-----
 {1,2,3}
(1 row)
```

```
SELECT array_cat(ARRAY[1,2], ARRAY[3,4]);
       array_cat
-----
 {1,2,3,4}
(1 row)
```

```
SELECT array_cat(ARRAY[[1,2],[3,4]], ARRAY[5,6]);
       array_cat
-----
 {{1,2},{3,4},{5,6}}
(1 row)
```

```
SELECT array_cat(ARRAY[5,6], ARRAY[[1,2],[3,4]]);
       array_cat
-----
 {{5,6},{1,2},{3,4}}
```


Searching in Arrays

To search for a value in an array, each value must be checked. This can be done manually, if you know the size of the array. For example:

```
SELECT * FROM sal_emp WHERE pay_by_quarter[1] = 10000 OR  
                             pay_by_quarter[2] = 10000 OR  
                             pay_by_quarter[3] = 10000 OR  
                             pay_by_quarter[4] = 10000;
```

However, this quickly becomes tedious for large arrays, and is not helpful if the size of the array is unknown. An alternative method is described in Section 9.24. The above query could be replaced by:

```
SELECT * FROM sal_emp WHERE 10000 = ANY (pay_by_quarter);
```

In addition, you can find rows where the array has all values equal to 10000 with:

```
SELECT * FROM sal_emp WHERE 10000 = ALL (pay_by_quarter);
```

Searching in Arrays

```
SELECT
  array_position(ARRAY['sun','mon','tue','wed','thu','fri','sat'],
    'mon');
array_position
```

```
-----
```

```
2
```

```
(1 row)
```

```
SELECT array_positions(ARRAY[1, 4, 3, 1, 3, 4, 2, 1], 1);
array_positions
```

```
-----
```

```
{1,4,8}
```

```
(1 row)
```

Composite Types

- A composite type represents the structure of a row or record; it is essentially just a list of field names and their data types.
- PostgreSQL allows composite types to be used in many of the same ways that simple types can be used.
- For example, a column of a table can be declared to be of a composite type.

Declaration of Composite Types

```
CREATE TYPE complex AS (  
    r          double precision,  
    i          double precision  
);  
  
CREATE TYPE inventory_item AS (  
    name          text,  
    supplier_id   integer,  
    price         numeric  
);
```



Constructing Composite Values

- To write a composite value as a literal constant, enclose the field values within parentheses and separate them by commas.
- We can put double quotes around any field value, and must do so if it contains commas or parentheses.
- Thus, the general format of a composite constant is the following:
- `'( val1 , val2 , ... )'`



Constructing Composite Values

An example is:

```
'("fuzzy dice",42,1.99)'
```

which would be a valid value of the `inventory_item` type defined above. To make a field be NULL, write no characters at all in its position in the list. For example, this constant specifies a NULL third field:

```
'("fuzzy dice",42,)'
```

If you want an empty string rather than NULL, write double quotes:

```
'( "", 42, )'
```

Here the first field is a non-NULL empty string, the third is NULL.



Accessing Composite Types

```
SELECT item.name FROM on_hand WHERE item.price > 9.99;
```

This will not work since the name `item` is taken to be a table name, not a column name of `on_hand`, per SQL syntax rules. You must write it like this:

```
SELECT (item).name FROM on_hand WHERE (item).price > 9.99;
```

or if you need to use the table name as well (for instance in a multitable query), like this:

```
SELECT (on_hand.item).name FROM on_hand WHERE (on_hand.item).price  
> 9.99;
```

Built in Range Types

- `int4range` — Range of integer
- `int8range` — Range of bigint
- `numrange` — Range of numeric
- `tsrange` — Range of timestamp without time zone
- `tstzrange` — Range of timestamp with time zone
- `daterange` — Range of date

Built in Range Types

```
CREATE TABLE reservation (room int, during tsrange);
INSERT INTO reservation VALUES
    (1108, '[2010-01-01 14:30, 2010-01-01 15:30)');

-- Containment
SELECT int4range(10, 20) @> 3;

-- Overlaps
SELECT numrange(11.1, 22.2) && numrange(20.0, 30.0);

-- Extract the upper bound
SELECT upper(int8range(15, 25));

-- Compute the intersection
SELECT int4range(10, 20) * int4range(15, 25);

-- Is the range empty?
SELECT isempty(numrange(1, 5));
```

PostgreSQL Full Text Search Entities

- Document: A set of data on which your full-text search will take place is called Document.
- In PostgreSQL, you can create a document using either a single column or multiple columns or even using multiple tables.
- Lexemes: The PostgreSQL document during processing is parsed into multiple tokens, which consist of words, sentences, etc. of the text present in the document.
- These tokens are modified to form more meaningful entities called lexemes.



PostgreSQL Full Text Search Entities

- Dictionaries: The conversion of Tokens into Lexemes is done using dictionaries.
- PostgreSQL offers both options of either using the built-in dictionaries or creating your own dictionaries.
- These dictionaries are responsible to determine which stop words are not important and whether differently derived words have the same base language.

to_tsvector

- To create a vector for a sentence, you can use the `to_tsvector` function as follows:
- `SELECT to_tsvector('The quick brown fox jumped over the lazy dog.');`
- `to_tsvector -----'brown':3 'dog':9 'fox':4 'jump':5 'lazi':8 'quick':2`



to_tsquery

- This function accepts a list of words that would be checked against the normalized vector created with the `to_tsvector()` function.
- You can use the following Postgres functions to convert text fields to tsquery values such as `to_tsquery`, `plainto_tsquery`, and `phraseto_tsquery`.
- `SELECT to_tsquery('The quick brown fox jumped over the lazy dog')`



Searching

- After creating tsvector from your text data and tsquery from your search terms, you can further perform a full-text search using the @@ operator.
- @@ operator checks if tsquery matches tsvector.
- For instance, if the word to be queried is “fox” for the above-mentioned example, then:
- `SELECT to_tsvector('The quick brown fox jumped over the lazy dog') @@ to_tsquery('fox');`
- Refer courses table in queries.txt



The SQL Language

- Creating a New Table

```
CREATE TABLE weather (  
  city varchar(80),  
  temp_lo int, -- low temperature  
  temp_hi int, -- high temperature  
  prcp real, -- precipitation  
  date date  
);
```



Populating a Table With Rows

- The INSERT statement is used to populate a table with rows:
- `INSERT INTO weather VALUES ('San Francisco', 46, 50, 0.25, '1994-11-27');`
- Note that all data types use rather obvious input formats.
- Constants that are not simple numeric values usually must be surrounded by single quotes ('), as in the example



What is Normalization?

- **Normalization** is the process of organizing data in a database to:
 - Eliminate **redundant data** (repetition).
 - Ensure **data dependencies** make sense.
 - Improve **data integrity**.



Goals of Normalization

- Minimize duplication of data.
- Protect the database against logical or structural anomalies.
- Make database maintenance easier and more efficient.

Normal Forms (NF)

- Normalization is done through a series of **normal forms**, each building on the previous:
-  **1. First Normal Form (1NF)**
- **Atomic values** only (no repeating groups or arrays).
- Each column should contain **unique and indivisible** values.

Normal Forms (NF)

- Normalization is done through a series of **normal forms**, each building on the previous:
-  **1. First Normal Form (1NF)**
- **Atomic values** only (no repeating groups or arrays).
- Each column should contain **unique and indivisible** values.

First Normal Form

Student	Subjects
John	Math, Science

Student	Subject
John	Math
John	Science

2. Second Normal Form (2NF)

- 1NF + every non-key attribute is fully dependent on the entire primary key (no partial dependency).
- Applies only to tables with composite keys.

Violation Example:

OrderID	ProductID	ProductName
---------	-----------	-------------

✓ Fix:

Split into:

- Orders (OrderID, ProductID)
- Products (ProductID, ProductName)

Third Normal Form (3NF)

- 2NF + no transitive dependencies (non-key attribute depends on another non-key attribute).

Violation Example:

| EmployeeID | DepartmentID | DepartmentName |

✓ Fix:

- Employee (EmployeeID, DepartmentID)
- Department (DepartmentID, DepartmentName)

Boyce-Codd Normal Form (BCNF)

- BCNF (Boyce-Codd Normal Form) is a stronger version of the Third Normal Form (3NF).
- **Rule:**
A relation is in **BCNF** if and only if every **determinant** is a **candidate key**.
- A **determinant** is any attribute (or set of attributes) on which some other attribute is **fully functionally dependent**.
- A **candidate key** is a minimal set of attributes that can uniquely identify a tuple.

Example That Is in 3NF but Not in BCNF

Course	Instructor	Room
DBMS	Alice	R1
OS	Bob	R2
DBMS	Alice	R1

Functional Dependencies (FDs):

- Course $\rightarrow$ Instructor  (Each course has one instructor)
- Instructor $\rightarrow$ Course  Not always true
- Instructor $\rightarrow$ Room  (Each instructor has a fixed room)



Why It's in 3NF but Not in BCNF?

- All non-key attributes are dependent on keys, so it satisfies 3NF.
- But Instructor $\rightarrow$ Room violates BCNF because:
 - Instructor is not a candidate key, but it's a determinant.
- Hence, not in BCNF.

Fix: Decompose into BCNF

Split the table into **two relations**:

◆ **Table 1:** Instructor_Room

Instructor	Room
Alice	R1
Bob	R2

FD: Instructor $\rightarrow$ Room ☒ Instructor is the key here.

Fix: Decompose into BCNF

◆ Table 2: Course_Instructor

Course	Instructor
DBMS	Alice
OS	Bob

FD: Course $\rightarrow$ Instructor ☒ Course is the key here.



Higher Normal Forms (4NF, 5NF)

- Handle multi-valued dependencies and join dependencies.
- Rarely used in everyday business applications but important in theoretical or complex systems.

Example: Joining employee and department tables

dept_id	dept_name
1	HR
2	IT
3	Finance

Example: Joining employee and department tables

emp_id	emp_name	dept_id
101	Alice	2
102	Bob	1
103	Charlie	3
104	David	NULL

Create Tables with Constraints

```
-- 1. Create Department table
CREATE TABLE IF NOT EXISTS department (
    dept_id SERIAL PRIMARY KEY,
    dept_name VARCHAR(100) NOT NULL
);

-- 2. Create Employee table with Foreign Key referencing Department
CREATE TABLE IF NOT EXISTS employee (
    emp_id SERIAL PRIMARY KEY,
    emp_name VARCHAR(100) NOT NULL,
    dept_id_fk INTEGER,
    CONSTRAINT fk_department
        FOREIGN KEY (dept_id_fk)
        REFERENCES department(dept_id)
        ON DELETE SET NULL
        ON UPDATE CASCADE
);
```



Create .sql file

- Sudo -i -u postgres
- Psql
- \i /home/ubuntu-user/Documents/Labs/Lab01/script.sql

Describe Table

- \d department

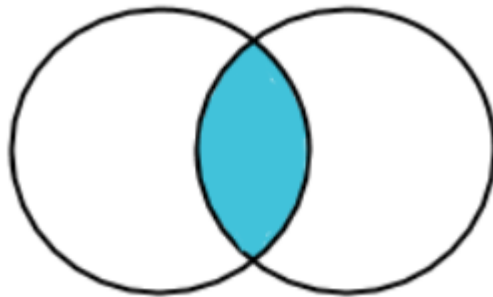
```
Table "public.department"
Column | Type | Collation | Nullable | Default
-----+-----+-----+-----+-----
---
dept_id | integer | | not null | nextval('department_dept_id_seq'::regclas
s)
dept_name | text | | not null |
Indexes:
    "department_pkey" PRIMARY KEY, btree (dept_id)
```



Insert into table

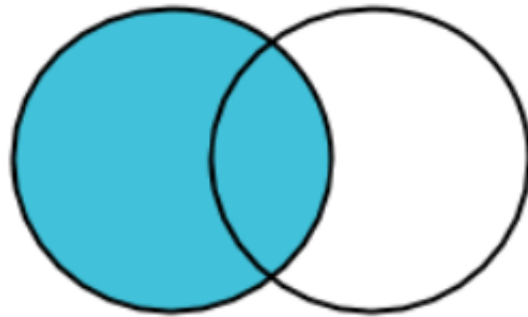
- Insert into department(dept_name) values('HR');
- Insert into
employee(emp_name,dept_id_fk)values('param',1);

- **INNER JOIN:** Match rows in both tables



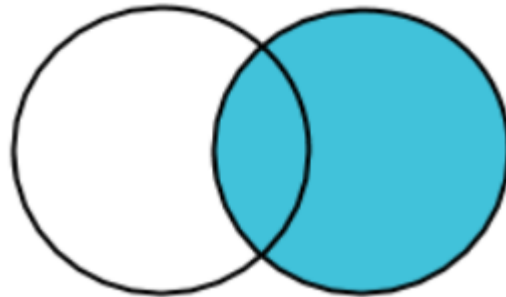
```
SELECT e.emp_id, e.emp_name, d.dept_name  
FROM employee e  
INNER JOIN department d ON e.dept_id_fk = d.dept_id;
```

- **LEFT JOIN: All rows from left + matched rows from right**



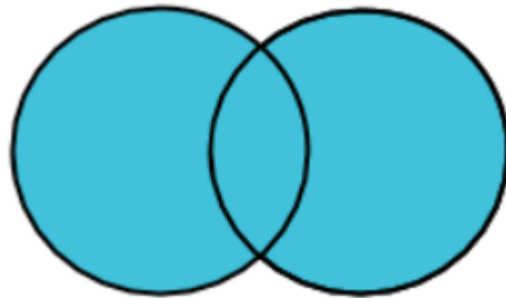
```
SELECT e.emp_id, e.emp_name, d.dept_name  
FROM employee e  
LEFT JOIN department d ON e.dept_id = d.dept_id;
```

- **RIGHT JOIN: All rows from right + matched rows from left**



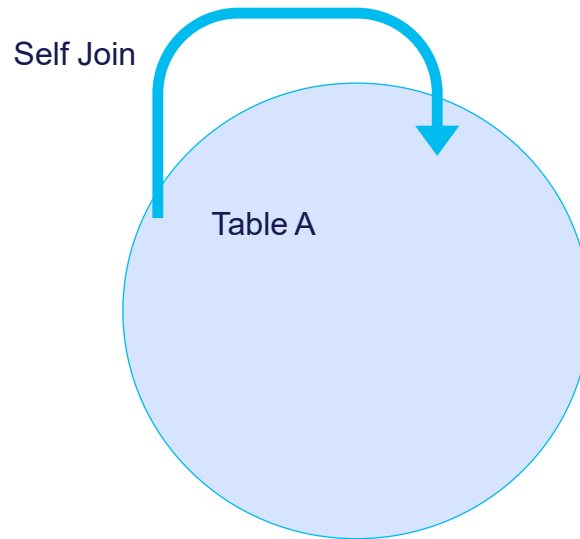
```
SELECT e.emp_id, e.emp_name, d.dept_name  
FROM employee e  
RIGHT JOIN department d ON e.dept_id = d.dept_id;
```

- **FULL OUTER JOIN: All rows from both tables, with NULLs where no match**



```
SELECT e.emp_id, e.emp_name, d.dept_name  
FROM employee e  
FULL OUTER JOIN department d ON e.dept_id = d.dept_id;
```


- **Self Join: Join a table with itself (e.g., employee → manager)**



Joins – Self Join

- **CREATE TABLE employee (**
- **emp_id SERIAL PRIMARY KEY,**
- **emp_name VARCHAR(100),**
- **manager_id INTEGER -- References another employee**
- **);**

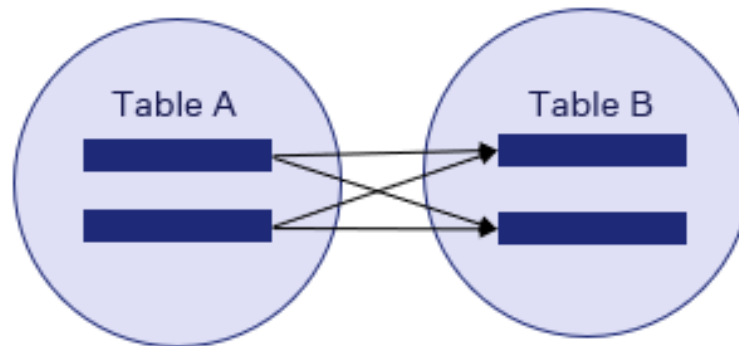
Joins – Self Join

employee_id	employee_name	manager_name
1	Alice	NULL
2	Bob	Alice
3	Charlie	Alice
4	David	Bob

Joins – Self Join

- **SELECT**
- **e.emp_id AS employee_id,**
- **e.emp_name AS employee_name,**
- **m.emp_name AS manager_name**
- **FROM employee e**
- **LEFT JOIN employee m ON e.manager_id = m.emp_id;**

- **Cross Join: Cartesian product of two tables**



```
SELECT e.emp_name, d.dept_name  
FROM employee e  
CROSS JOIN department d;
```

Join with subquery

- **Get employees only from the IT department, using a subquery in the join:**
- `SELECT e.emp_id, e.emp_name, d.dept_name`
- `FROM employee e`
- `JOIN (`
- `SELECT dept_id, dept_name`
- `FROM department`
- `WHERE dept_name = 'IT'`
- `) AS d ON e.dept_id = d.dept_id;`

Joins Summary



Join Type	Matching Rows	Non-Matching Rows
INNER	✓	✗
LEFT	✓ (left)	NULLs from right
RIGHT	✓ (right)	NULLs from left
FULL	✓	NULLs from both

Views

- A VIEW is a pseudo table in PostgreSQL; it is not a solid table but appears as an ordinary table to select.
- A view can also represent joined tables.
- It can contain all rows of a table or selected rows from one or more tables.
- A View simplifies users to perform the following aspects:
 - It structures data naturally and intuitively and makes it easy to find.
 - We can authorize permission to users over a view, which has a complete record that the users are authorized to see.
 - It restricts access to the data such that users can only see limited data instead of complete data.
 - A view provides a dependable layer, even the columns of necessary table modification.
 - It summarizes data from various tables to generate reports.
 - A view helps us to describe the difficulty of a statement as we can write a command of view based on a complex query with the help of a SELECT command.

Views

CREATE [OR REPLACE] VIEW view-name AS

SELECT column(s)

FROM table(s)

[WHERE condition(s)];

CREATE VIEW Price_View AS

SELECT id, price

FROM Price

WHERE price > 200;



Clear the screen

- `\! cls`



EXPLAIN – See the Query Plan

- Shows how PostgreSQL will execute a query, including:
 - Join types
 - Index usage
 - Cost estimates (not actual execution)

```
EXPLAIN SELECT * FROM employee WHERE dept_id = 2;
```

EXPLAIN ANALYZE – Run the Query and Show Actual Execution Details



- It **executes the query** and shows:
 - Actual time taken per step
 - Rows processed
 - Loops
 - Index usage
 - Total execution time

```
EXPLAIN ANALYZE SELECT * FROM employee WHERE dept_id = 2;
```

```
Seq Scan on employee (cost=0.00..25.00 rows=5 width=32) (actual  
time=0.021..0.023 rows=2 loops=1)
```

```
  Filter: (dept_id = 2)
```

```
    Rows Removed by Filter: 3
```

```
Planning Time: 0.100 ms
```

```
Execution Time: 0.050 ms
```

Key Terms Explained:

Term	Meaning
Seq Scan	Sequential scan of the table (may be slow on large tables)
Index Scan	Uses an index – usually faster
cost=X..Y	Estimated startup cost and total cost
rows=N	Estimated number of rows
actual time	Actual time to fetch rows
loops	How many times this step was executed
Rows Removed by Filter	Rows read but discarded by the WHERE clause



Best Practices

- Use EXPLAIN for safe preview.
- Use EXPLAIN ANALYZE to benchmark real execution.
- If the query is slow, look for:
 - Sequential scan instead of index scan
 - High number of loops
 - Large difference between estimated and actual rows

Normal View vs Materialized View

Feature	View	Materialized View
Stores data physically	No	Yes
Always shows latest data	Yes	No
Query speed	Depends on base query	Usually faster
Needs refresh	No	Yes
Can create indexes	Limited	Yes



Materialized View

- CREATE MATERIALIZED VIEW mv_city_sales AS
- SELECT
- city,
- COUNT(*) AS total_sales,
- SUM(price * quantity) AS total_revenue,
- AVG(price) AS avg_price
- FROM vehicle_sales
- GROUP BY city;



Materialized View

- Query it like a table:
- `SELECT * FROM mv_city_sales;`

Refreshing a Materialized View

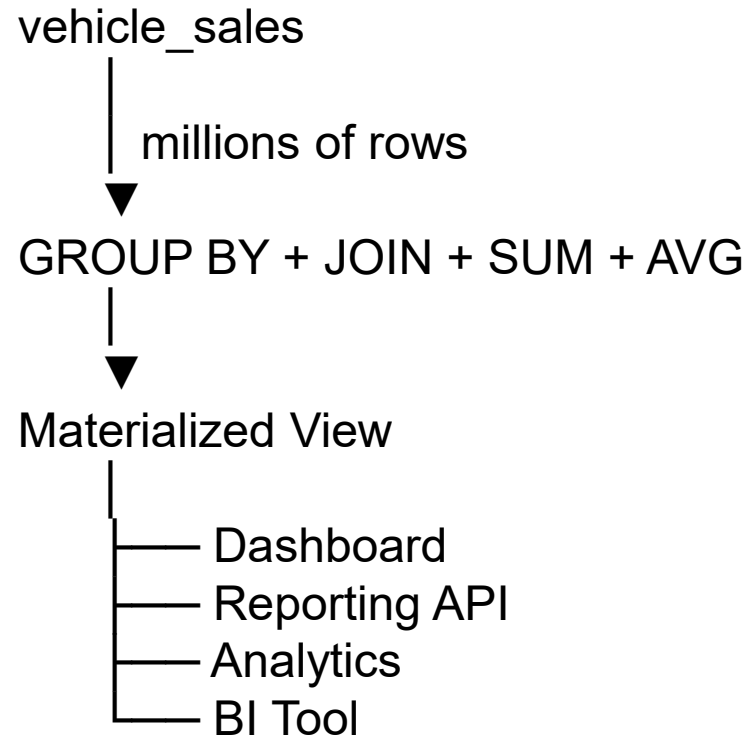
- Changes in the base table are **not automatically reflected**.
- `REFRESH MATERIALIZED VIEW mv_city_sales;`
- Example:
 - `INSERT INTO vehicle_sales`
 - `VALUES (101, 'X5', 'Petrol', 'Chennai', 8500000, 1);`
- -- materialized view still contains old data
 - `REFRESH MATERIALIZED VIEW mv_city_sales;`



Concurrent refresh

- For larger applications:
 - `REFRESH MATERIALIZED VIEW CONCURRENTLY mv_city_sales;`
 - This allows reads to continue while PostgreSQL refreshes the materialized view.

When should you use one?



Explain and Analyze

```
public | profile      | table | enterprisedb
public | starrating        | table | admin
(5 rows)

accountdb=# explain analyze select * from producttransaction;
                                QUERY PLAN
-----
Seq Scan on producttransaction (cost=0.00..1.02 rows=2 width=143) (actual time=10.385..10.388 rows=2 loops=1)
Planning Time: 7.873 ms
Execution Time: 29.357 ms
(3 rows)

accountdb=# explain analyze select * from customer;
                                QUERY PLAN
-----
Seq Scan on customer (cost=0.00..1.01 rows=1 width=40) (actual time=0.022..0.024 rows=1 loops=1)
Planning Time: 0.103 ms
Execution Time: 0.044 ms
(3 rows)

accountdb=# explain analyze select * from globalcountryinfo;
                                QUERY PLAN
-----
Seq Scan on globalcountryinfo (cost=0.00..1.01 rows=1 width=34) (actual time=27.068..27.072 rows=1 loops=1)
Planning Time: 372.275 ms
Execution Time: 27.098 ms
(3 rows)

accountdb=#
```

Materialized View

```
(3 rows)

accountdb=# explain analyze select * from customer;
               QUERY PLAN
-----
Seq Scan on customer (cost=0.00..1.01 rows=1 width=40) (actual time=0.022..0.024 rows=1 loops=1)
Planning Time: 0.103 ms
Execution Time: 0.044 ms
(3 rows)

accountdb=# explain analyze select * from globalcountryinfo;
               QUERY PLAN
-----
Seq Scan on globalcountryinfo (cost=0.00..1.01 rows=1 width=34) (actual time=27.068..27.072 rows=1 loops=1)
Planning Time: 372.275 ms
Execution Time: 27.098 ms
(3 rows)

accountdb=# create Materialized View mv_globalcountryinfo as select * from globalcountryinfo;
SELECT 1
accountdb=# explain analyze select * from mv_globalcountryinfo;
               QUERY PLAN
-----
Seq Scan on mv_globalcountryinfo (cost=0.00..23.10 rows=1310 width=34) (actual time=0.015..0.015 rows=1 loops=1)
Planning Time: 0.294 ms
Execution Time: 0.027 ms
(3 rows)

accountdb=#
```

Refreshing a MATERIALIZED VIEW



EDB-PSQL - 13

```
accountdb=# refresh materialized view mv_globalcountryinfo;  
REFRESH MATERIALIZED VIEW  
accountdb=#
```

— □ ×

Postgres Index

- An Index is the structure or object by which we can retrieve specific rows or data faster.
- Indexes can be created using one or multiple columns or by using the partial data depending on your query requirement conditions.
- Index will create a pointer to the actual rows in the specified table.

Postgres Index



Table

	Column A	Column B
RowId 1	Some value 1	Some another value 1
RowId 2	Some value 2	Some another value 2
RowId 3	Some value 3	Some another value 3
RowId 4	Some value 1	Some another value 4
RowId 5	Some value 5	Some another value 5
RowId 6	Some value 6	Some another value 6
RowId 7	Some value 1	Some another value 7
RowId 8	Some value 8	Some another value 8

Index

Some value 1	"RowId 1" "RowId 4" "RowId 7"
Some value 2	RowId 2
Some value 3	RowId 3
Some value 5	RowId 5
Some value 6	RowId 6
Some value 8	RowId 8

Index by column "Column A"



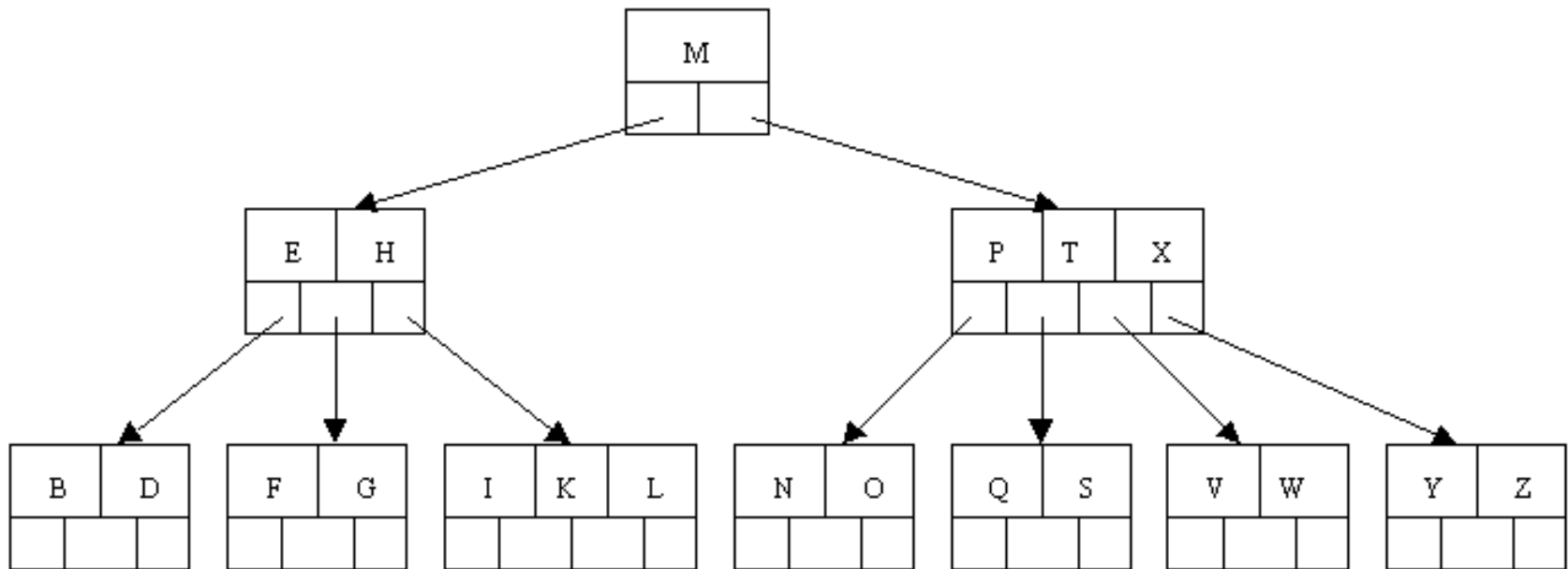
Postgres Index

- Types Of Indexes
 - PostgreSQL server provides following types of indexes, which each uses a different algorithm:
 - B-tree
 - Hash
 - GiST
 - SP-GiST
 - GIN
 - BRIN

B-tree index

- The most common and widely used index type is the B-tree index.
- This is the default index type for the `CREATE INDEX` command, unless you explicitly mention the type during index creation.
- If the indexed column is used to perform the comparison by using comparison operators such as `<`, `<=`, `=`, `>=`, and `>`, then the Postgres optimizer uses the index created by the B-tree option for the specified column.

B-tree index



B-tree index

- Advantages:
 - Retain sorting data
 - Support the search for the unary and binary predicates
 - Allow the entire sequence of data to estimate cardinality (number of entries) for the entire index (and therefore the table), range, and with arbitrary precision without scanning
- Disadvantages:
 - For their construction is require to perform a full sorting pairs (row, RowId) (slow operation)
 - Take up a lot of disk space. Index on unique “Integers” weights twice more as the column (because additionaly RowId need stored)
 - Recording unbalances tree constantly, and begins to store data sparsely, and the access time is increased by increasing the amount of disk information. What is why, B-Tree indexes require monitoring and periodic rebuilding



List indexes

- SELECT
 - tablename,
 - indexname,
 - indexdef
- FROM
 - pg_indexes
- WHERE
 - schemaname = 'public'
- ORDER BY
 - tablename,
 - indexname;

List indexes



EDB-PSQL - 13

```
Output: transaction_id, transaction_date_time, amount, mode, status
Filter: (producttransaction.amount > 0)
Planning Time: 0.137 ms
Execution Time: 0.033 ms
(5 rows)
```

```
accountdb=# SELECT
accountdb=#     tablename,
accountdb=#     indexname,
accountdb=#     indexdef
accountdb=# FROM
accountdb=#     pg_indexes
accountdb=# WHERE
accountdb=#     schemaname = 'public'
accountdb=# ORDER BY
accountdb=#     tablename,
accountdb=#     indexname;
```

tablename	indexname	indexdef
customer	customer_pkey	CREATE UNIQUE INDEX customer_pkey ON public.customer USING btree (customer_id)
globalcountryinfo	globalcountryinfo_pkey	CREATE UNIQUE INDEX globalcountryinfo_pkey ON public.globalcountryinfo USING btree (countrycode)
producttransaction	amount_index	CREATE INDEX amount_index ON public.producttransaction USING btree (amount)
producttransaction	producttransaction_pkey	CREATE UNIQUE INDEX producttransaction_pkey ON public.producttransaction USING btree (transaction_id)
profile	profile_pkey	CREATE UNIQUE INDEX profile_pkey ON public.profile USING btree (profile_id)

(5 rows)

```
accountdb=#
```



Type here to search



B-Tree Index

EDB-PSQL - 13

```

mode          | character varying(50) |
status        | boolean               |

```

Indexes:

```
"producttransaction_pkey" PRIMARY KEY, btree (transaction_id)
```

```
accountdb=# create index amount_index on producttransaction(amount);
```

```
CREATE INDEX
```

```
accountdb=# explain (analyze,verbose) select * from producttransaction;
QUERY PLAN
```

```
-----
Seq Scan on public.producttransaction (cost=0.00..1.02 rows=2 width=143) (actual time=0.016..0.017 rows=2 loops=1)
```

```
Output: transaction_id, transaction_date_time, amount, mode, status
```

```
Planning Time: 0.626 ms
```

```
Execution Time: 0.032 ms
```

```
(4 rows)
```

```
accountdb=# \d producttransaction
```

		Table "public.producttransaction"			Default
Column	Type	Collation	Nullable		
transaction_id	bigint		not null	nextval('producttransaction_transaction_id_seq'::regclass)	
transaction_date_time	timestamp with time zone				
amount	bigint				
mode	character varying(50)				
status	boolean				

Indexes:

```
"producttransaction_pkey" PRIMARY KEY, btree (transaction_id)
```

```
"amount_index" btree (amount)
```

```
accountdb=#
```


B-Tree Index

```
index_demo=# explain (analyze,verbose) select * from public."Track" where "Name"='Princess of the
```

QUERY PLAN

```
-----  
Bitmap Heap Scan on public."Track" (cost=4.29..8.30 rows=1 width=71) (actual time=0.923..0.924 ro
```

```
Output: "TrackId", "Name", "AlbumId", "Bytes", "UnitPrice"
```

```
Recheck Cond: (("Track"."Name")::text = 'Princess of the Dawn'::text)
```

```
Heap Blocks: exact=1
```

```
-> Bitmap Index Scan on "Track_Name_idx" (cost=0.00..4.29 rows=1 width=0) (actual time=0.578..
```

```
Index Cond: (("Track"."Name")::text = 'Princess of the Dawn'::text)
```

```
Planning time: 0.128 ms
```

```
Execution time: 0.953 ms
```

```
(8 rows)
```

B-Tree Index



EDB-PSQL - 13

```
accountdb=# refresh materialized view mv_globalcountryinfo;
REFRESH MATERIALIZED VIEW
accountdb=# explain (analyze,verbose) select * from globalcountryinfo;
               QUERY PLAN
-----
Seq Scan on public.globalcountryinfo  (cost=0.00..1.01 rows=1 width=34) (actual time=0.016..0.017 rows=1 loops=1)
  Output: countrycode, country_state
  Planning Time: 0.074 ms
  Execution Time: 0.037 ms
(4 rows)

accountdb=# explain (analyze,verbose) select * from mv_globalcountryinfo;
               QUERY PLAN
-----
Seq Scan on public.mv_globalcountryinfo  (cost=0.00..23.10 rows=1310 width=34) (actual time=0.017..0.017 rows=1 loops=1)
  Output: countrycode, country_state
  Planning Time: 0.412 ms
  Execution Time: 0.030 ms
(4 rows)

accountdb=# explain (analyze,verbose) select * from producttransaction;
               QUERY PLAN
-----
Seq Scan on public.producttransaction  (cost=0.00..1.02 rows=2 width=143) (actual time=0.019..0.021 rows=2 loops=1)
  Output: transaction_id, transaction_date_time, amount, mode, status
  Planning Time: 0.079 ms
  Execution Time: 0.033 ms
(4 rows)

accountdb=#
```

B-Tree Index

Here is a query which can use the created B-tree index and then again try to select the data with the same **WHERE** clause:

```
index_demo_=# create index "Track_Name_idx" on public."Track" ("Name");

CREATE INDEX

index_demo_=#

index_demo_=# explain (analyze,verbose) select * from public."Track" where "Name"='Princess of the Dawn';

                                QUERY PLAN

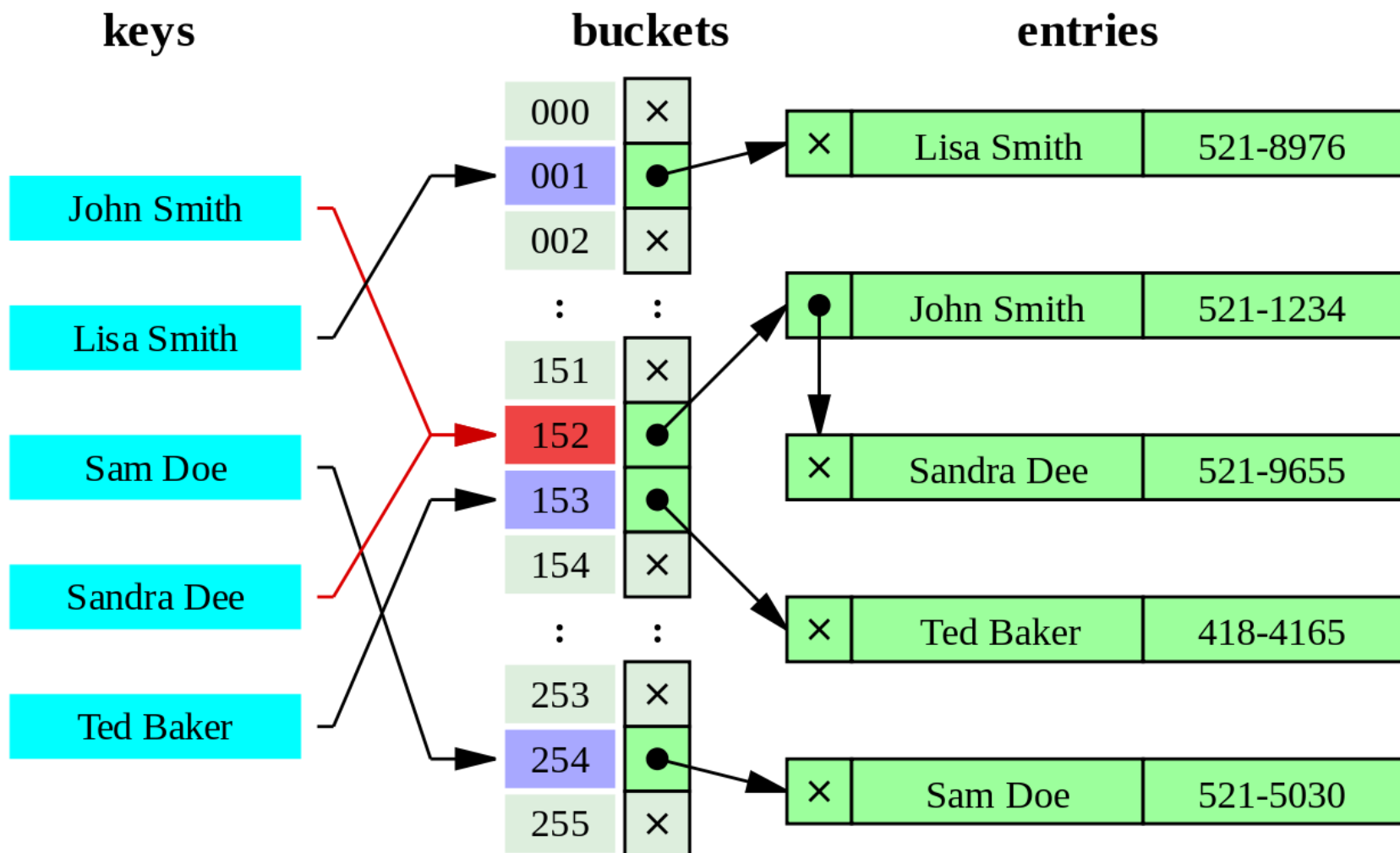
-----
Index Scan using "Track_Name_idx" on public."Track"  (cost=0.28..8.30 rows=1 width=71) (actual time=0.571 ms)
    Output: "TrackId", "Name", "AlbumId", "MediaTypeId", "GenreId", "Composer", "Milliseconds", "Bytes"
    Index Cond: (("Track"."Name")::text = 'Princess of the Dawn'::text)

Planning time: 0.151 ms

Execution time: 0.571 ms

(5 rows)
```

Hash index



Hash index

- Advantages:
 - Very fast search $O(1)$
 - Stability - the index does not need to be rebuild
- Disadvantages:
- Hash is very sensitive to collisions. In the case of “bad” data distribution, most of the entries will be concentrated in a few bouquets, and in fact the search will occur through collision resolution.

Hash index

- The Hash index can be used only if the equality condition = is being used in the query.

```
index_demo_=# create index "Track_Name_idx" on public."Track" using HASH ("Name");
```

```
CREATE INDEX
```

```
index_demo_=# \d+ public."Track"
```

```

                                Table "public.Track"

```

Column	Type	Collation	Nullable	Default	Storage	Stats target	Description
TrackId	integer		not null		plain		
Name	character varying(200)		not null		extended		
AlbumId	integer				plain		
MediaTypeId	integer		not null		plain		
GenreId	integer				plain		
Composer	character varying(220)				extended		
Milliseconds	integer		not null		plain		
Bytes	integer				plain		
UnitPrice	numeric(10,2)		not null		main		

```
Indexes:
```

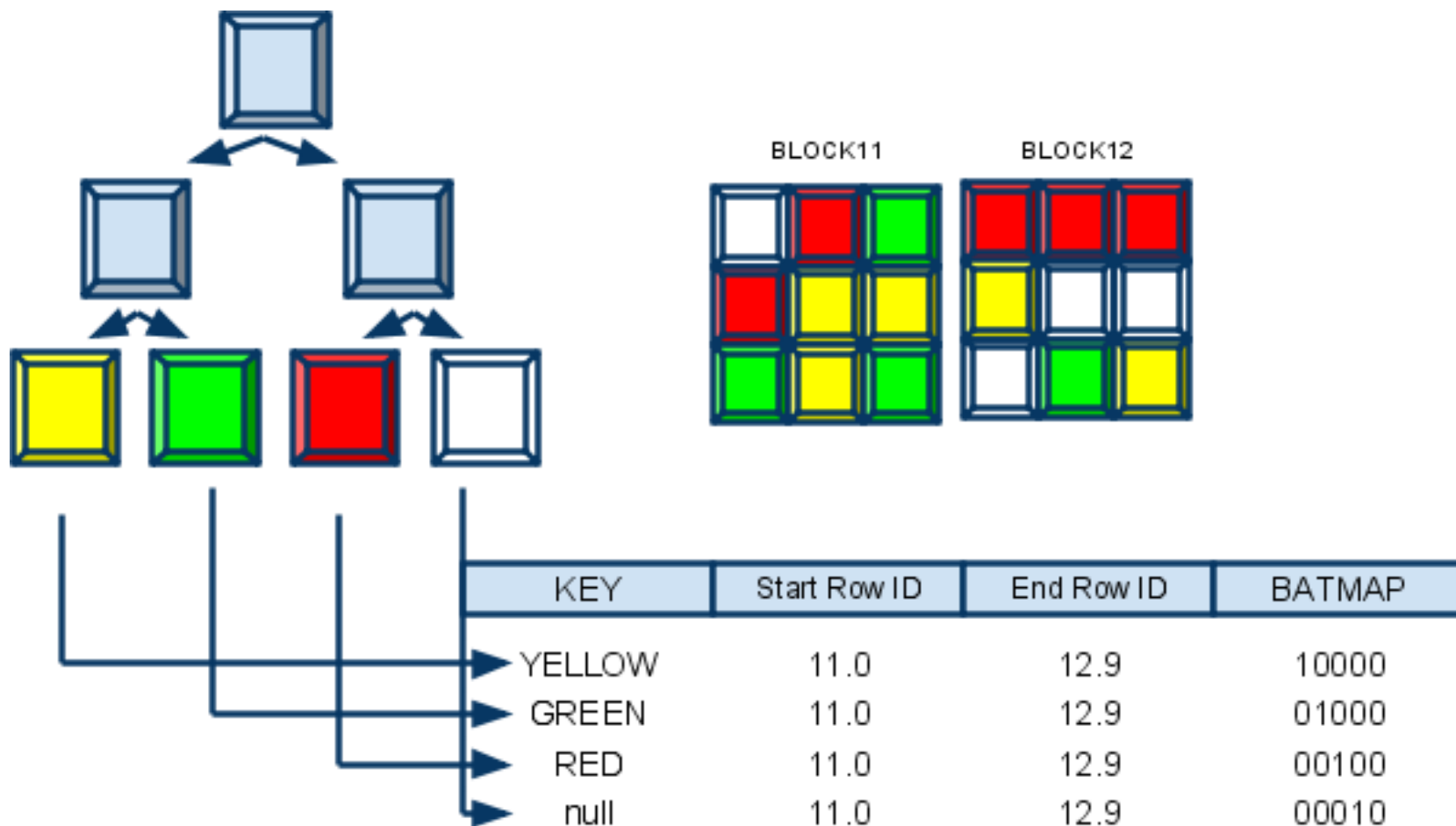
```
"PK_Track" PRIMARY KEY, btree ("TrackId")
```

```
"Track_Name_idx" hash ("Name")
```

Bitmap index

- Bitmap index create a separate bitmap (a sequence of 0 and 1) for each possible value of the column, where each bit corresponds to a string with an indexed value. Bitmap indexes are optimal for data where bit unique values (example, gender field).

Bitmap index



Bitmap index

- Advantages:
 - Compact representation (small amount of disk space)
 - Fast reading and searching for the predicate “is”
 - Effective algorithms for packing masks (even more compact representation, than indexed data)
- Disadvantages:
 - You can not change the method of encoding values in the process of updating the data. From this it follows that if the distribution data has changed, it is required the index to be completely rebuild



GiST index (Generalized Search Tree index)

- GiST stands for Generalized Search Tree Index.
- The main point of GiST is to be able to index queries that simply are not indexable using B-tree.
- So GiST is useful when you have queries that are not btree-indexable.
- The column can be of tsvector or tsquery type.
- There is a well-known, and pretty fast, geographic extension to PostgreSQL — PostGIS, which uses GiST for its purposes. GiST indexes are what is used if you want to speed up full text searches.

GiST index (Generalized Search Tree index)

Create a table with some geographical data points such as circle or point and index it using GiST:

```
index_demo_=# create table geo_point(circle_details circle);

CREATE TABLE

index_demo_=# create index ON geo_point using gist(circle_details);

CREATE INDEX

index_demo_=# \d+ geo_point

               Table "public.geo_point"

  Column | Type          | Collation | Nullable | Default | Storage | Stats target | Description
-----+-----+-----+-----+-----+-----+-----+-----
 circle_details | circle        |           |          |         | plain   |              |
Indexes:
"geo_point_circle_details_idx" gist (circle_details)

index_demo_=#
```

GiST index (Generalized Search Tree index)

Create a table with some geographical data points such as circle or point and index it using GiST:

```
index_demo_=# create table geo_point(circle_details circle);

CREATE TABLE

index_demo_=# create index ON geo_point using gist(circle_details);

CREATE INDEX

index_demo_=# \d+ geo_point

               Table "public.geo_point"

  Column | Type   | Collation | Nullable | Default | Storage | Stats target | Description
-----+-----+-----+-----+-----+-----+-----+-----
 circle_details | circle |          |          |         | plain   |              |

Indexes:
"geo_point_circle_details_idx" gist (circle_details)

index_demo_=#
```

SP-GiST (Space-Partitioned GiST)



- SP-GiST stands for “space-partitioned GiST.”
- This index type supports non-balanced data structures such as quadtrees, k-d trees, and radix trees.



GIN (Generalized Inverted Index)

- GIN stands for “Generalized Inverted Index.” This are the “Inverted Indexes” that get used for multiple component values such as arrays.
- The standard distribution of PostgreSQL includes a GIN operator class for arrays, which supports indexed queries using these operators: `<@`, `@>`, `=`, and `&&`.



BRIN (Block Range index)

- BRIN stands for “Block Range index.” This featured index type was introduced with Postgres v 9.5.
- “Because a BRIN index is very small, scanning the index adds little overhead compared to a sequential scan, but may avoid scanning large parts of the table that are known not to contain matching tuples.”
- “The size of the block range is determined at index creation time by the `pages_per_range` storage parameter.
- The number of index entries will be equal to the size of the relation in pages divided by the selected value for `pages_per_range`.
- Therefore, the smaller the number, the larger the index becomes (because of the need to store more index entries), but at the same time the summary data stored can be more precise and more data blocks can be skipped during an index scan.”

BRIN (Block Range index)

- In a BRIN index, PostgreSQL reads your selected column's maximum and minimum values for each 8k-size page of stored data, and then stores that information (page number and minimum and maximum values of column) into the BRIN index.

BRIN (Block Range index)

- B-tree indexes have entries for each row in your table, duplicating the data in the indexed columns.
- This permits super-fast index-only scans, but can be wasteful of disk space.
- A B-tree index also stores the data in sorted order, which permits very fast lookups of single rows.
- It also stores the full location of each row in the table, which can require a lot of work when running queries that return many rows.
- However, a BRIN index contains only the minimum and maximum values contained in a group of database pages.
- Therefore, the reason for using the BRIN index is that this is a very straightforward way to optimize for speed, and the BRIN index only takes up very little space compared to a B-tree index.
- BRIN is designed for handling very large tables in which certain columns have some natural correlation with their physical location within the table.



BRIN (Block Range index)

- A good example of where you could use a BRIN index is for data you would consider immutable.
- For the example that follows, the BRIN index is 72kb in size, while the B-tree index is 676mb in size.
- Yet the performance is pretty much identical for selects.

-



BRIN (Block Range index)

```
index_demo_=# create index "Track_TrackId_idx" on public."Track" using brin("TrackId");
```

```
CREATE INDEX
```

```
index_demo_=# \d+ public."Track"
```

Table "public.Track"

Column	Type	Collation	Nullable	Default	Storage	Stats target	Description
TrackId	integer		not null		plain		
Name	character varying(200)		not null		extended		
AlbumId	integer				plain		
MediaTypeId	integer		not null		plain		
GenreId	integer				plain		
Composer	character varying(220)				extended		
Milliseconds	integer		not null		plain		
Bytes	integer				plain		
UnitPrice	numeric(10,2)		not null		main		

```
Indexes:
```

```
"PK_Track" PRIMARY KEY, btree ("TrackId")
```

```
"Track_TrackId_idx" brin ("TrackId")
```

```
index_demo_=# \di+
```

Partial Index

- The partial index is useful in case you have commonly used WHERE conditions which use constant values as follows:

```
SELECT *  
FROM table_name  
WHERE column_name = constant_value;
```

Partial Index

- You can optimize this query by creating an index for the active column as follows:
- `CREATE INDEX idx_customer_active`
- `ON customer(active);`
- This index fulfills its purpose, however, it includes many rows that are never searched, namely all the active customers.
- To define an index that includes only inactive customers, you use the following statement:
- **`CREATE INDEX idx_customer_inactive`**
- **`ON customer(active)`**
- **`WHERE active = 0;`**

Built-In Functions- String

Function	Description	Example
LENGTH(string)	Returns the number of characters	LENGTH('hello') → 5
LOWER(string)	Converts to lowercase	LOWER('HELLO') → 'hello'
UPPER(string)	Converts to uppercase	UPPER('hello') → 'HELLO'
INITCAP(string)	Capitalizes first letter of each word	INITCAP('hello world') → 'Hello World'
TRIM([LEADING TRAILING BOTH] 'x' FROM str)		
BTRIM(str, chars)	Trims specified characters from both ends	BTRIM('xyHelloxy', 'xy') → 'Hello'
CONCAT(str1, str2, ...)	Concatenates strings	CONCAT('Hello', ' ', 'World') → 'Hello World'
`		`

Built-In Functions- String

Function	Description	Example
SUBSTRING(str FROM x FOR y)	Extracts substring	SUBSTRING('abcdef' FROM 2 FOR 3) → 'bcd'
LEFT(str, n)	Leftmost n characters	LEFT('abcdef', 3) → 'abc'
RIGHT(str, n)	Rightmost n characters	RIGHT('abcdef', 3) → 'def'
REPLACE(str, from, to)	Replace substring	REPLACE('hello', 'l', 'x') → 'hexxo'
POSITION(substr IN str)	Position of substring	POSITION('e' IN 'hello') → 2
STRPOS(str, substr)	Alias for POSITION	STRPOS('hello', 'e') → 2
OVERLAY(str PLACING str2 FROM pos FOR len)	Replaces part of string	OVERLAY('abcdef' PLACING 'xyz' FROM 2 FOR 3) → 'axyzef'

Built-In Functions- String

Function	Description	Example
REVERSE(str)	Reverses a string	REVERSE('abc') → 'cba'
LPAD(str, len, padstr)	Pads left side	LPAD('123', 5, '0') → '00123'
RPAD(str, len, padstr)	Pads right side	RPAD('123', 5, '0') → '12300'
REGEXP_REPLACE(str, pattern, replacement)	Regex-based replacement	REGEXP_REPLACE('abc123', '[0-9]', '', 'g') → 'abc'
REGEXP_MATCHES(str, pattern)	Returns regex match array	REGEXP_MATCHES('abc123', '[a-z]+') → {abc}

```
SELECT LENGTH('PostgreSQL'), LOWER('HELLO'), SUBSTRING('abcdef'
FROM 2 FOR 3), REPLACE('banana', 'a', 'x'), LPAD('42', 5, '0');
```


Built in Date Functions

Function	Description	Example
CURRENT_DATE	Returns today's date	SELECT CURRENT_DATE;
CURRENT_TIME	Returns current time	SELECT CURRENT_TIME;
CURRENT_TIMESTAMP or NOW()	Returns current date and time	SELECT NOW();
AGE(timestamp)	Interval between now and given timestamp	AGE('2000-01-01')
AGE(timestamp1, timestamp2)	Interval between two timestamps	AGE('2025-01-01', '2000-01-01')
DATE_PART(field, source)	Extract part of a date (year, month, etc.)	DATE_PART('year', NOW())
EXTRACT(field FROM source)	Same as DATE_PART	EXTRACT(DAY FROM NOW())
DATE_TRUNC(field, source)	Truncates date/time to a unit	DATE_TRUNC('month', NOW()) → 2025-07-01 00:00:00
TO_CHAR(timestamp, format)	Format a timestamp	TO_CHAR(NOW(), 'YYYY-MM-DD')
TO_DATE(text, format)	Convert string to date	TO_DATE('2025-07-20', 'YYYY-MM-DD')

Built in Date Functions

Function	Description	Example
TO_TIMESTAMP(text, format)	Convert string to timestamp	TO_TIMESTAMP('2025-07-20 15:00', 'YYYY-MM-DD HH24:MI')
CLOCK_TIMESTAMP()	Returns actual current time (not cached)	SELECT CLOCK_TIMESTAMP();
STATEMENT_TIMESTAMP()	Timestamp when current statement started	SELECT STATEMENT_TIMESTAMP();
TRANSACTION_TIMESTAMP()	Timestamp when transaction started	SELECT TRANSACTION_TIMESTAMP();
JUSTIFY_DAYS(interval)	Converts excess days into months	JUSTIFY_DAYS(INTERVAL '90 days')
JUSTIFY_INTERVAL(interval)	Normalizes months and days	JUSTIFY_INTERVAL(INTERVAL '1 year 25 months')

```
-- Add 7 days to today
SELECT CURRENT_DATE + INTERVAL '7 days';

-- Subtract 1 month
SELECT CURRENT_DATE - INTERVAL '1 month';
```



Date Functions

- SELECT
- `EXTRACT(YEAR FROM CURRENT_DATE) AS year,`
- `EXTRACT(MONTH FROM CURRENT_DATE) AS month,`
- `EXTRACT(DAY FROM CURRENT_DATE) AS day;`

Date Functions

- `SELECT '2025-01-01'::date < '2025-12-31'::date;`
- `--` returns true
- `SELECT TO_CHAR(NOW(), 'YYYY-MM-DD HH24:MI:SS');`
- `--` Output: '2025-07-20 18:34:12'

Built-in Functions - JSON/JSONB Functions in PostgreSQL



Feature	JSON	JSONB
Storage	As text	Binary format
Speed	Slower to query	Faster and indexed
Ordering	Preserved	Not preserved
Duplicate Keys	Allowed	Deduplicated (last wins)

Built-in Functions - JSON/JSONB Functions in PostgreSQL



```
CREATE TABLE employees (  
  id SERIAL PRIMARY KEY,  
  name TEXT,  
  data JSONB -- store nested info  
);
```

```
INSERT INTO employees (name, data) VALUES  
(  
  'Alice', '{"age": 30, "skills": ["SQL", "Python"]}'  
,  
  'Bob', '{"age": 25, "skills": ["Java", "React"]}'  
);
```

Built-in Functions - JSON/JSONB Functions in PostgreSQL



Operator / Function	Description	Example
->	Get JSON object field by key (returns JSON)	data -> 'age'
->>	Get JSON object field as text	data ->> 'age'
#>	Get nested JSON object	data #> '{skills,0}'
#>>	Get nested JSON as text	data #>> '{skills,0}'
@>	Does JSONB contain the specified key/value?	data @> '{"age": 30}'
<@	Is JSONB contained in another?	'{ "age": 30 }' <@ data
?	Does key exist?	data ? 'age'
`?`	`	Any of the keys exist?
?&	All keys exist?	data ?& array['age', 'skills']
jsonb_array_elements()	Expands JSON array to rows	see below
jsonb_each_text()	Extracts key-value pairs as text	see below

Built-in Functions - JSON/JSONB Functions in PostgreSQL



```
SELECT name, (data ->> 'age')::int AS age FROM employees;
```

```
SELECT * FROM employees WHERE data @> '{"age": 30}';
```

```
SELECT name, jsonb_array_elements_text(data->'skills') AS skill  
FROM employees;
```

```
SELECT name, key, value  
FROM employees, jsonb_each_text(data);
```

```
CREATE INDEX idx_employee_data ON employees USING GIN  
(data);
```




Moving Data the Copy command

- import and export data using CSV files in PostgreSQL
 - 1. Creating a .csv file
 - 2. Creating the table structure
 - 3. Importing from a psql prompt
 - 4. Importing from a shell prompt
 - 5. Exporting data to a .csv file



Moving Data the Copy command

- Creating a .csv file
- The first step is to create a .csv file. You can use any existing file, or you can use the data below that comprises a basic .csv file:
- Capital_City,State,Abbreviation,Zip_Code
- Montgomery,Alabama,AL,36104
- Juneau,Alaska,AK,99801
- Phoenix,Arizona,AZ,85001
- Little Rock,Arkansas,AR,72201



Moving Data the Copy command

- `CREATE TABLE usa`
- `(Capital_city varchar,`
- `State varchar,`
- `Abbreviation varchar(2),`
- `zip_code numeric(5)`
- `);`

Importing from a psql prompt

- `\COPY <table name> FROM 'location + file_name' DELIMITER ',' CSV HEADER;`
- To understand the execution steps, let's look at each item in this command:
- `\COPY`: This is the command to copy the record to / from the .csv file.
- `<table name>`: Provides the table name where you want to import the data.
- `FROM`: Specifies that we are going to import from a file (we will also be using `TO` in order to export it to a file at a later stage).



Importing from a psql prompt

- 'location + file_name': An absolute path to the file (make sure you have read access to the file).
- DELIMITER ',': Specifies the delimiter, which in our case is a comma: ','.
- CSV Specifies the file type from which we are going to import.
- HEADER Signifies that we have a header row in our .csv file and while importing we should ignore the first row (similarly, while exporting we can use this to specify whether we want to include or exclude the header file).



Importing from a psql prompt

EDB-PSQL - 13

```
accountdb=# \copy usa from 'd:/basic.csv' delimiter ',' csv header;  
COPY 4  
accountdb=#
```

— □ ×

Exporting data to a .csv file



EDB-PSQL - 13

```
accountdb=# \copy usa from 'd:/basic.csv' delimiter ',' csv header;  
COPY 4  
accountdb=# \COPY usa TO 'd:/output.csv' DELIMITER ',' CSV HEADER;  
COPY 4  
accountdb=#
```

Understanding And Reading the PostgreSQL System Catalog (Meta Data)



- The PostgreSQL System Catalog is a schema with tables and views that contain metadata about all the other objects inside the database and more.
- With it, we can discover when various operations happen, how tables or indexes are accessed, and even whether or not the database system is reading information from memory or needing to fetch data from disk.

Understanding And Reading the PostgreSQL System Catalog (Meta Data)



- PostgreSQL stores the metadata information about the database and cluster in the schema 'pg_catalog'.
- This information is partially used by PostgreSQL itself to keep track of things itself, but it also is presented so external people / processes can understand the inside of the databases too.

Understanding And Reading the PostgreSQL System Catalog (Meta Data)



- Database Information
- General database info is stored in `pg_database` and statistics are stored in `pg_stat_database`.
- `SELECT oid,* FROM pg_database WHERE datname = 'accountdb';`
- `SELECT * FROM pg_stat_database WHERE datname = 'accountdb';`

Understanding And Reading the PostgreSQL System Catalog (Meta Data)



- Database Activity and Locks
- There are two views that show current user activity, `pg_stat_activity` and `pg_locks`.
- When queried, these show information about current connections to the databases, and what kind of locks they have on what relations.
- `SELECT * FROM pg_stat_activity;`
- `SELECT * FROM pg_locks;`

Understanding And Reading the PostgreSQL System Catalog (Meta Data)



- `SELECT * FROM pg_stat_user_tables WHERE schemaname = 'public' AND relname = 'history';`

Understanding And Reading the PostgreSQL System Catalog (Meta Data)



- Table Vacuum Activity
- Table maintenance is done through either VACUUM or AUTOVACUUM, and statistics are gathered through ANALYZE or AUTOANALYZE.
- The next four columns contain the dates for when each of these operations were last run: 'last_vacuum', 'last_autovacuum', 'last_analyze', 'last_autoanalyze'.
- We also have four more convenient columns that simply count how many times the previous actions occur.
- Using these, we can see which tables get the most activity: 'vacuum_count', 'autovacuum_count', 'analyze_count', and 'autoanalyze_count'.

Understanding And Reading the PostgreSQL System Catalog (Meta Data)



- `SELECT * FROM pg_statio_user_tables WHERE schemaname = 'public' AND relname = history;`
- Index Meta data
- `SELECT * FROM pg_stat_user_indexes WHERE indexrelname = 'history_pkey';`



How to use table partitioning to scale PostgreSQL

- 1. Benefits of partitioning
- 2. When to use partitioning
- 3. How to use partitioning
 - a. List partition
 - b. Range partition
 - c. Hash partition
 - d. Multilevel partition
- 4. Limitations

Partitioning

- With huge data being stored in databases, performance and scaling are two main factors that are affected.
- As table size increases with data load, more data scanning, swapping pages to memory, and other table operation costs also increase.
- Partitioning may be a good solution, as It can help divide a large table into smaller tables and thus reduce table scans and memory swap problems, which ultimately increases performance.
- Partitioning helps to scale PostgreSQL by splitting large logical tables into smaller physical tables that can be stored on different storage media based on uses.
- Users can take better advantage of scaling by using declarative partitioning along with foreign tables using `postgres_fdw`.

Benefits of Partitioning

- PostgreSQL declarative partitioning is highly flexible and provides good control to users.
- Users can create any level of partitioning based on need and can modify, use constraints, triggers, and indexes on each partition separately as well as on all partitions together.
- Query performance can be increased significantly compared to selecting from a single large table.
- Partition-wise-join and partition-wise-aggregate features increase complex query computation performance as well.
- Bulk loads and data deletion can be much faster, as based on user requirements these operations can be performed on individual partitions.
- Each partition can contain data based on its frequency of use and so can be stored on media that may be cheaper or slower for low-use data.

When to use partitioning

- Most benefits of partitioning can be enjoyed when a single table is not able to provide them.
- So we can say that if a lot of data is going to be written on a single table at some point, users need partitioning.
- Apart from data, there may be other factors users should consider, like update frequency of the data, use of data over a time period, how small a range data can be divided, etc.
- With good planning and taking all factors into consideration, table partitioning can give a great performance boost and scale your PostgreSQL to larger datasets.



How to use partitioning

- As of PostgreSQL12 release List, Range, Hash and combinations of these partition methods at different levels are supported.

LIST PARTITION

- A list partition is created with predefined values to hold in a partitioned table.
- A default partition (optional) holds all those values that are not part of any specified partition.
- `CREATE TABLE customers (id INTEGER, status TEXT, arr NUMERIC) PARTITION BY LIST(status);`
- `CREATE TABLE cust_active PARTITION OF customers FOR VALUES IN ('ACTIVE');`
- `CREATE TABLE cust_archived PARTITION OF customers FOR VALUES IN ('EXPIRED');`
- `CREATE TABLE cust_others PARTITION OF customers DEFAULT;`



RANGE PARTITION

- A range partition is created to hold values within a range provided on the partition key.
- Both minimum and maximum values of the range need to be specified, where minimum value is inclusive and maximum value is exclusive.
- `CREATE TABLE customers (id INTEGER, status TEXT, arr NUMERIC) PARTITION BY RANGE(arr);`
- `CREATE TABLE cust_arr_small PARTITION OF customers FOR VALUES FROM (MINVALUE) TO (25);`

RANGE PARTITION

- CREATE TABLE cust_arr_medium PARTITION OF customers FOR VALUES FROM (25) TO (75);
- CREATE TABLE cust_arr_large PARTITION OF customers FOR VALUES FROM (75) TO (MAXVALUE);

HASH PARTITION

- A hash partition is created by using modulus and remainder for each partition, where rows are inserted by generating a hash value using these modulus and remainders.
- `CREATE TABLE customers (id INTEGER, status TEXT, arr NUMERIC) PARTITION BY HASH(id);`
- `CREATE TABLE cust_part1 PARTITION OF customers FOR VALUES WITH (modulus 3, remainder 0);`

HASH PARTITION

- CREATE TABLE cust_part2 PARTITION OF customers FOR VALUES WITH (modulus 3, remainder 1);
- CREATE TABLE cust_part3 PARTITION OF customers FOR VALUES WITH (modulus 3, remainder 2);



MULTILEVEL PARTITION

- PostgreSQL multilevel partitions can be created up to N levels.
- Partition methods LIST-LIST, LIST-RANGE, LIST-HASH, RANGE-RANGE, RANGE-LIST, RANGE-HASH, HASH-HASH, HASH-LIST, and HASH-RANGE can be created in PostgreSQL declarative partitioning.

MULTILEVEL PARTITION

- `CREATE TABLE customers (id INTEGER, status TEXT, arr NUMERIC) PARTITION BY LIST(status);`
- `CREATE TABLE cust_active PARTITION OF customers FOR VALUES IN ('ACTIVE','RECURRING','REACTIVATED') PARTITION BY RANGE(arr);`
- `CREATE TABLE cust_arr_small PARTITION OF cust_active FOR VALUES FROM (MINVALUE) TO (101) PARTITION BY HASH(id);`

MULTILEVEL PARTITION

- CREATE TABLE cust_arr_small PARTITION OF cust_active FOR VALUES FROM (MINVALUE) TO (101) PARTITION BY HASH(id);
- CREATE TABLE cust_part11 PARTITION OF cust_arr_small FOR VALUES WITH (modulus 2, remainder 0);
- CREATE TABLE cust_part12 PARTITION OF cust_arr_small FOR VALUES WITH (modulus 2, remainder 1);
- CREATE TABLE cust_other PARTITION OF customers DEFAULT PARTITION BY RANGE(arr);

MULTILEVEL PARTITION

- CREATE TABLE cust_arr_large PARTITION OF cust_other FOR VALUES FROM (101) TO (MAXVALUE) PARTITION BY HASH(id);
- CREATE TABLE cust_part21 PARTITION OF cust_arr_large FOR VALUES WITH (modulus 2, remainder 0);
- postgres=# CREATE TABLE cust_part22 PARTITION OF cust_arr_large FOR VALUES WITH (modulus 2, remainder 1);

- PL/pgSQL procedural language adds many procedural elements e.g., control structures, loops, and complex computations to extend standard SQL.
- It allows you to develop complex functions and stored procedures in PostgreSQL that may not be possible using plain SQL.
- Since PostgreSQL 9.0, PL/pgSQL is installed by default.



- PL/pgSQL procedural language is similar to the Oracle PL/SQL. The following are reasons to learn PL/pgSQL:
 - PL/pgSQL is easy to learn and simple to use.
 - PL/pgSQL comes with PostgreSQL by default. The user-defined functions and stored procedures developed in PL/pgSQL can be used like any built-in functions and stored procedures.
 - PL/pgSQL inherits all user-defined types, functions, and operators.
 - PL/pgSQL has many features that allow you to develop complex functions and stored procedures.
 - PL/pgSQL can be defined to be trusted by the PostgreSQL database server.

- PL/pgSQL is a procedural programming language for the PostgreSQL database system.
- PL/pgSQL allows you to extend the functionality of the PostgreSQL database server by creating server objects with complex logic.
- PL/pgSQL was designed to :
 - Create user-defined functions, stored procedures, and triggers.
 - Extend standard SQL by adding control structures such as if, case, and loop statements.
 - Inherit all user-defined functions, operators, and types.

Advantages of using PL/pgSQL

- SQL is a query language that allows you to query data from the database easily. However, PostgreSQL only can execute SQL statements individually.
- It means that you have multiple statements, you need to execute them one by one like this:
 - First, send a query to the PostgreSQL database server.
 - Next, wait for it to process.
 - Then, process the result set.
 - After that, do some calculations.
 - Finally, send another query to the PostgreSQL database server and repeat this process.

Advantages of using PL/pgSQL

- This process incurs the interprocess communication and network overheads.
- To resolve this issue, PostgreSQL uses PL/pgSQL.
 - PL/pgSQL wraps multiple statements in an object and store it on the PostgreSQL database server.
 - So instead of sending multiple statements to the server one by one, you can send one statement to execute the object stored in the server. This allows you to:
 - Reduce the number of round trips between the application and the PostgreSQL database server.
 - Avoid transferring the immediate results between the application and the server.



PostgreSQL PL/pgSQL disadvantages

- Besides the advantages of using PL/pgSQL, there are some caveats:
 - Slower in software development because PL/pgSQL requires specialized skills that many developers do not possess.
 - Difficult to manage versions and hard to debug.
 - May not be portable to other database management systems.



Dollar-Quoted String Constants

- In PostgreSQL, you use single quotes for a string constant like this:
 - `select 'String constant';`
- When a string constant contains a single quote ('), you need to escape it by doubling up the single quote. For example:
 - `select 'I''m also a string constant';`

Dollar-Quoted String Constants

- PostgreSQL version 8.0 introduced the dollar quoting feature to make string constants more readable.
- The following shows the syntax of the dollar-quoted string constants:
 - `$tag$<string_constant>$tag$`
- In this syntax, the tag is optional. It may contain zero or many characters.
 - Between the `$tag$`, you can place any string with single quotes (`'`) and backslashes (`\`).
 - For example:
 - `select $$I'm a string constant that contains a backslash \$$;`



Dollar-Quoted String Constants

- In this example, we did not specify the tag between the two dollar signs(\$).
- The following example uses the dollar-quoted string constant syntax with a tag:
 - `SELECT $message$I'm a string constant that contains a backslash \message$;`
 - In this example, we used the string message as a tag between the two dollar signs (\$)

Using dollar-quoted string constant in anonymous blocks



do

'declare

 film_count integer;

begin

 select count(*) into film_count

 from film;

 raise notice "The number of films: %", film_count;

end;;

Using dollar-quoted string constant in anonymous blocks



To avoid escaping every single quotes and backslashes, you can use the dollar-quoted string as follows

do

\$\$

declare

 film_count integer;

begin

 select count(*) into film_count

 from film;

 raise notice 'The number of films: %', film_count;

end;

\$\$

Using dollar-quoted string constant in anonymous blocks



Dashboard Properties SQL Statistics Dependencies Dependents accountdb/enterprisedb@EDB Postgres Advanced Server 13 *

accountdb/enterprisedb@EDB Postgres Advanced Server 13

Query Editor Query History

```
1 do
2 $$
3 declare
4     film_count integer;
5 begin
6     select count(*) into film_count
7     from films;
8     raise notice 'The number of films: %', film_count;
9 end;
10 $$
```

Data Output Explain Messages Notifications

NOTICE: The number of films: 3
DO

Query returned successfully in 1 secs 14 msec.



Using dollar-quoted string constants in functions

```
create function find_film_by_id(  
    id int  
) returns film  
language sql  
as  
$$  
    select * from film  
    where film_id = id;  
$$;
```

Using dollar-quoted string constants in functions



EDB-PSQL - 13

```
accountdb=# create function find_film_by_id(  
accountdb(#   id int  
accountdb(# ) returns films  
accountdb=# language sql  
accountdb=# as  
accountdb=# $$  
accountdb$$  select * from films  
accountdb$$  where film_id = id;  
accountdb$$ $$;  
CREATE FUNCTION
```

accountdb/enterisedb@EDB Postgres Advanced Server 13

Query Editor Query History

```
1 select find_film_by_id(1) from dual;
```

Data Output Explain Messages Notifications

	find_film_by_id films	
1	(1,Joker)	

PL/pgSQL Block Structure

- PL/pgSQL is a block-structured language, therefore, a PL/pgSQL function or stored procedure is organized into blocks.

```
[ <<label>> ]
```

```
[ declare  
    declarations ]
```

```
begin  
    statements;
```

```
    ...
```

```
end [ label ];
```

PL/pgSQL Block Structure

- PL/pgSQL is a block-structured language, therefore, a PL/pgSQL function or stored procedure is organized into blocks.

```
[ <<label>> ]
```

```
[ declare  
    declarations ]
```

```
begin  
    statements;
```

```
    ...
```

```
end [ label ];
```



PL/pgSQL Block Structure

- Each block has two sections: declaration and body.
- The declaration section is optional while the body section is required.
- A block is ended with a semicolon (;) after the END keyword.
- A block may have an optional label located at the beginning and at the end.
- We use the block label when you want to specify it in the EXIT statement of the block body or when you want to qualify the names of variables declared in the block.
- The declaration section is where you declare all variables used within the body section.
- Each statement in the declaration section is terminated with a semicolon (;).
- The body section is where you place the code. Each statement in the body section is also terminated with a semicolon (;).



PL/pgSQL Block Structure

```
do $$
<<first_block>>
declare
    film_count integer := 0;
begin
    -- get the number of films
    select count(*)
    into film_count
    from films;
    -- display a message
    raise notice 'The number of films is %', film_count;
end first_block $$;
```

PL/pgSQL Block Structure



Dashboard Properties SQL Statistics Dependencies Dependents accountdb/enterprisedb@EDB Postgres Advanced Server 13 *

accountdb/enterprisedb@EDB Postgres Advanced Server 13

Query Editor Query History

```
1 do $$
2 <<first_block>>
3 declare
4   film_count integer := 0;
5 begin
6   -- get the number of films
7   select count(*)
8   into film_count
9   from films;
10  -- display a message
11  raise notice 'The number of films is %', film_count;
12 end first_block $$;
```

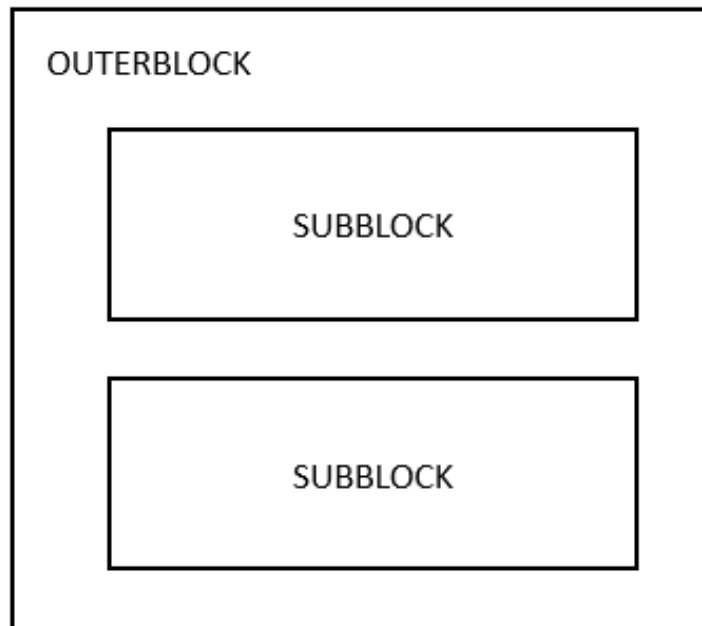
Data Output Explain Messages Notifications

NOTICE: The number of films is 3
DO

Query returned successfully in 217 msec.

PL/pgSQL Subblocks

- PL/pgSQL allows you to place a block inside the body of another block.
- The block nested inside another block is called a subblock. The block that contains the subblock is referred to as an outer block.





Introduction to variables in PL/pgSQL

- A variable is a meaningful name of a memory location.
- A variable holds a value that can be changed through the block.
- A variable is always associated with a particular data type.
- `variable_name data_type [:= expression];`



Introduction to variables in PL/pgSQL

- First, specify the name of the variable.
- It is a good practice to assign a meaningful name to a variable.
- For example, instead of naming a variable `i` you should use `index` or `counter`.
- Second, associate a specific data type with the variable.
- The data type can be any valid data type such as integer, numeric, varchar, and char.
- Third, optionally assign a default value to a variable. If you don't do so, the initial value of the variable is NULL.



Introduction to variables in PL/pgSQL

- First, specify the name of the variable.
- It is a good practice to assign a meaningful name to a variable.
- For example, instead of naming a variable `i` you should use `index` or `counter`.
- Second, associate a specific data type with the variable.
- The data type can be any valid data type such as integer, numeric, varchar, and char.
- Third, optionally assign a default value to a variable. If you don't do so, the initial value of the variable is NULL.
- Note that you can use either `:=` or `=` assignment operator to initialize and assign a value to a variable.



Introduction to variables in PL/pgSQL

```
do $$
declare
    counter    integer := 1;
    first_name varchar(50) := 'John';
    last_name  varchar(50) := 'Doe';
    payment    numeric(11,2) := 20.5;
begin
    raise notice '% % % has been paid % USD',
        counter,
        first_name,
        last_name,
        payment;
end $$;
```

Variable initialization timing

```
do $$  
declare  
    created_at time := now();  
begin  
    raise notice '%', created_at;  
    perform pg_sleep(10);  
    raise notice '%', created_at;  
end $$;
```

Variable initialization timing

accountdb/enterprisedb@EDB Postgres Advanced Server 13 ▾

Query Editor Query History

```
1  do $$
2  declare
3      created_at time := now();
4  begin
5      raise notice '%', created_at;
6      perform pg_sleep(10);
7      raise notice '%', created_at;
8  end $$;
9
```

Data Output Explain Messages Notifications

NOTICE: 20:46:10.686542
NOTICE: 20:46:10.686542
DO

Query returned successfully in 10 secs 636 msec.

Copying data types

- The %type provides the data type of a table column or another variable.
- Typically, you use the %type to declare a variable that holds a value from the database or another variable.
 - `variable_name table_name.column_name%type;`
- And the following shows how to declare a variable with the data type of another variable:
 - `variable_name variable%type;`

Copying data types

```
do $$
declare
    film_title films.title%type;
    featured_title film_title%type;
begin
    -- get title of the film id 100
    select title
    from films
    into film_title
    where film_id = 1;

    -- show the film title
    raise notice 'Film title id 100: %s', film_title;
end; $$
```




Variables in block and subblock

- When you declare a variable in a subblock which has the same name as another variable in the outer block, the variable in the outer block is hidden in the subblock.

```
do $$
<<outer_block>>
declare
  counter integer := 0;
begin
  counter := counter + 1;
  raise notice 'The current value of the
counter is %', counter;

  declare
    counter integer := 0;
  begin
    counter := counter + 10;
    raise notice 'Counter in the subblock is
%', counter;
    raise notice 'Counter in the outer block
is %', outer_block.counter;
  end;

  raise notice 'Counter in the outer block is
%', counter;

end outer_block $$;
```



PL/pgSQL row types example

```
do $$
declare
    selected_film films%rowtype;
begin
    -- select film with id 1
    select *
    from films
    into selected_film
    where film_id = 1;

    -- show the number of actor
    raise notice 'The film name is % %',
        selected_film.film_id,
        selected_film.title;
end; $$
```



PL/pgSQL Record Types

```
do
$$
declare
    rec record;
begin
    -- select the film
    select film_id, title
    into rec
    from films
    where film_id = 1;

    raise notice '% %', rec.film_id, rec.title;

end;
$$
```



Using record variables in the for loop statement

```
do
$$
declare
    rec record;
begin
    for rec in select title
                    from films
                    where film_id > 0
                    order by title
    loop
        raise notice '%', rec.title;
    end loop;
end;
$$
```

PL/pgSQL Constants

- do \$\$
- declare
 - vat constant numeric := 0.1;
 - net_price numeric := 20.5;
- begin
- raise notice 'The selling price is %', net_price * (1 +
 vat);
- end \$\$;

PL/pgSQL Errors and Messages

- Reporting messages
- To raise a message, you use the raise statement as follows:
 - raise level format;
- Following the raise statement is the level option that specifies the error severity.
- PostgreSQL provides the following levels:
 - debug
 - log
 - notice
 - info
 - warning
 - exception

Reporting Messages

do \$\$

begin

raise info 'information message %', now() ;

raise log 'log message %', now();

raise debug 'debug message %', now();

raise warning 'warning message %', now();

raise notice 'notice message %', now();

end \$\$;



Raising errors

- To raise an error, you use the exception level after the raise statement.
- Note that raise statement uses the exception level by default.

Raising errors

- Besides raising an error, you can add more information by using the following additional clause:
- using option = expression
- The option can be:
 - message: set error message
 - hint: provide the hint message so that the root cause of the error is easier to be discovered.
 - detail: give detailed information about the error.
 - errcode: identify the error code, which can be either by condition name or directly five-character SQLSTATE code. Please refer to the table of error codes and condition names.



Raising Errors

```
do $$  
declare  
    email varchar(255) := 'info@postgresqltutorial.com';  
begin  
    -- check email for duplicate  
    -- ...  
    -- report duplicate email  
    raise exception 'duplicate email: %', email  
        using hint = 'check the email again';  
end $$;
```



Assert Statement

```
do $$  
declare  
    film_count integer;  
begin  
    select count(*)  
    into film_count  
    from films;  
  
    assert film_count > 1000, '1000 Film not found, check the  
film table';  
end$$;
```



PL/pgSQL IF Statement

- The if statement determines which statements to execute based on the result of a boolean expression.
- PL/pgSQL provides you with three forms of the if statements.
 - if then
 - if then else
 - if then elsif

PL/pgSQL IF Statement

- PL/pgSQL if-then statement
- The following illustrates the simplest form of the if statement:
 - if condition then
 - statements;
 - end if;



PL/pgSQL IF Statement

```
do $$  
declare  
    selected_film films%rowtype;  
    input_film_id films.film_id%type := 0;  
begin  
    select * from films  
    into selected_film  
    where film_id = input_film_id;  
    if not found then  
        raise notice 'The film % could not be found',  
            input_film_id;  
    end if;  
end $$
```



PL/pgSQL if-then-else statement

if condition then

 statements;

else

 alternative-statements;

END if;



PL/pgSQL if-then-else statement

if condition then

 statements;

else

 alternative-statements;

END if;



PL/pgSQL if-then-else statement

```
do $$
declare
    selected_film films%rowtype;
    input_film_id films.film_id%type := 100;
begin
    select * from films
    into selected_film
    where film_id = input_film_id;
    if not found then
        raise notice 'The film % could not be found',
            input_film_id;
    else
        raise notice 'The film title is %', selected_film.title;
    end if;
```



PL/pgSQL if-then-elsif Statement

- if condition_1 then
- statement_1;
- elsif condition_2 then
- statement_2
- ...
- elsif condition_n then
- statement_n;
- else
- else-statement;
- end if;



PL/pgSQL if-then-elsif Statement

```
do $$  
declare  
    v_film films%rowtype;  
    len_description varchar(100);  
begin  
  
    select * from films  
    into v_film  
    where film_id = 100;  
  
    if not found then  
        raise notice 'Film not found';  
    else
```

PL/pgSQL if-then-elsif Statement

```
if v_film.length > 0 and v_film.length <= 50 then
    len_description := 'Short';
elsif v_film.length > 50 and v_film.length < 120 then
    len_description := 'Medium';
elsif v_film.length > 120 then
    len_description := 'Long';
else
    len_description := 'N/A';
end if;

raise notice 'The % film is %.',
    v_film.title,
    len_description;

end if;
end $$
```



PL/pgSQL CASE Statement

- The case statement selects a when section to execute from a list of when sections based on a condition.
- The case statement has two forms:
 - Simple case statement
 - Searched case statement

Simple case statement

case search-expression

 when expression_1 [, expression_2, ...] then

 when-statements

 [...]

 [else

 else-statements]

END case;

Searched case statement

- case
- when boolean-expression-1 then
- statements
- [when boolean-expression-2 then
- statements
- ...]
- [else
- statements]
- end case;

While Loop

- do \$\$
- declare
 - counter integer := 0;
- begin
 - while counter < 5 loop
 - raise notice 'Counter %', counter;
 - counter := counter + 1;
 - end loop;
- end\$\$;



For Loop

- [<<label>>]
- for loop_counter in [reverse] from.. to [by step] loop
- statements
- end loop [label];

For Loop

- do
- \$\$
- declare
- f record;
- begin
- for f in select title
- from films
- order by title
- limit 10
- loop
- raise notice '%', f.title;
- end loop;
- end;
- \$\$

Introduction to PostgreSQL CREATE PROCEDURE statement



- create [or replace] procedure
 procedure_name(parameter_list)
- language plpgsql
- as \$\$
- declare
- -- variable declaration
- begin
- -- stored procedure body
- end; \$\$

Introduction to PostgreSQL CREATE PROCEDURE statement



- First, specify the name of the stored procedure after the create procedure keywords.
- Second, define parameters for the stored procedure. A stored procedure can accept zero or more parameters.
- Third, specify plpgsql as the procedural language for the stored procedure. Note that you can use other procedural languages for the stored procedure such as SQL, C, etc.
- Finally, use the dollar-quoted string constant syntax to define the body of the stored procedure.



Create Function

- create [or replace] function function_name(param_list)
- returns return_type
- language plpgsql
- as
- \$\$
- declare
- -- variable declaration
- begin
- -- logic
- end;
- \$\$

PL/pgSQL Function Parameter Modes: IN, OUT, INOUT



IN	OUT	INOUT
The default	Explicitly specified	Explicitly specified
Pass a value to function	Return a value from a function	Pass a value to a function and return an updated value.
in parameters act like constants	out parameters act like uninitialized variables	inout parameters act like an initialized variables
Cannot be assigned a value	Must assign a value	Should be assigned a value



Introduction to PL/pgSQL Function Overloading

- PostgreSQL allows multiple functions to share the same name as long as they have different arguments.
- If two or more functions share the same name, the function names are overloaded.
- When you can call an overloading function, PostgreSQL select the best candidate function to execute based on the the function argument list.

How to Develop a PL/pgSQL Function That Returns a Table



-
- create or replace function function_name (
 - parameter_list
 -)
 - returns table (column_list)
 - language plpgsql
 - as \$\$
 - declare
 - -- variable declaration
 - begin
 - -- body
 - end; \$\$

Triggers

- A PostgreSQL trigger is a function invoked automatically whenever an event such as insert, update, or delete occurs.

Triggers

- A PostgreSQL trigger is a function invoked automatically whenever an event associated with a table occurs.
- An event could be any of the following: INSERT, UPDATE, DELETE or TRUNCATE.
- A trigger is a special user-defined function associated with a table.
- To create a new trigger, you define a trigger function first, and then bind this trigger function to a table.
- The difference between a trigger and a user-defined function is that a trigger is automatically invoked when a triggering event occurs.
- PostgreSQL provides two main types of triggers: row and statement-level triggers. The differences between the two kinds are how many times the trigger is invoked and at what time.

Triggers

- For example, if you issue an UPDATE statement that affects 20 rows, the row-level trigger will be invoked 20 times, while the statement level trigger will be invoked 1 time.
- You can specify whether the trigger is invoked before or after an event.
- If the trigger is invoked before an event, it can skip the operation for the current row or even change the row being updated or inserted.
- In case the trigger is invoked after the event, all changes are available to the trigger.

Triggers



- Triggers are useful in case the database is accessed by various applications, and you want to keep the cross-functionality within the database that runs automatically whenever the data of the table is modified.
- For example, if you want to keep the history of data without requiring the application to have logic to check for every event such as INSERT or UPDATE.

Triggers

- You can also use triggers to maintain complex data integrity rules which you cannot implement elsewhere except at the database level.
- For example, when a new row is added into the customer table, other rows must be also created in tables of banks and credits.

Triggers

- The main drawback of using a trigger is that you must know the trigger exists and understand its logic to figure out the effects when data changes.
- Even though PostgreSQL implements SQL standard, triggers in PostgreSQL has some specific features:
 - PostgreSQL fires trigger for the TRUNCATE event.
 - PostgreSQL allows you to define the statement-level trigger on views.
 - PostgreSQL requires you to define a user-defined function as the action of the trigger, while the SQL standard allows you to use any SQL commands.



PostgreSQL CREATE TRIGGER

- To create a new trigger in PostgreSQL, you follow these steps:
 - First, create a trigger function using CREATE FUNCTION statement.
 - Second, bind the trigger function to a table by using CREATE TRIGGER statement.



PostgreSQL CREATE TRIGGER

```
CREATE FUNCTION trigger_function()  
    RETURNS TRIGGER  
    LANGUAGE PLPGSQL  
AS $$  
BEGIN  
    -- trigger logic  
END;  
$$
```



Inheritance

```
CREATE TABLE CUSTOMERS (  
    name text,  
    age int,  
    address text  
);
```

```
CREATE TABLE OFFICE_CUSTOMERS(  
    office_address text  
) INHERITS (customers)
```



Inheritance

```
INSERT INTO CUSTOMERS VALUES('ravi','32','32',  
head street');
```

```
INSERT INTO CUSTOMERS VALUES('michael','35','56',  
gotham street');
```

```
INSERT INTO OFFICE_CUSTOMERS  
VALUES('bane','28','56, circadia street','92 homebush');
```



Non Atomic Columns

```
CREATE TABLE customer (  
    name          text,  
    Address       text,  
    payment_schedule integer[],  
);
```



Non Atomic Columns

```
INSERT INTO CUSTOMER_SCHEDULE VALUES( 'jack',  
'Athens, Colarado',  
'{1,2,3,4}'  
)
```

```
INSERT INTO CUSTOMER_SCHEDULE VALUES(  
'jackson',  
'Tennessey, greece',  
'{1,7,3,4}'  
)
```



Non Atomic Columns

```
SELECT * FROM CUSTOMER_SCHEDULE WHERE  
CUSTOMER_SCHEDULE.payment_schedule[1] <>  
CUSTOMER_SCHEDULE.payment_schedule[2];
```



Postgres Cluster

- `.\initdb.exe -D e:/PostgreSQL/14/data`
- Change the server port in data folder config file
- `.\pg_ctl.exe start -D e:\PostgreSQL\14\data`
- Server started
- `psql -h localhost -d postgres -p 5436 -W`



Postgres Replication

- #5432
 - CREATE TABLE LogicalReplicationTest (id int PRIMARY KEY);
 - INSERT INTO LogicalReplicationTest VALUES (generate_series(1,10000));
 - SELECT count(*) FROM LogicalReplicationTest;
 - CREATE PUBLICATION testpub FOR TABLE LogicalReplicationTest;
- #5436
 - CREATE TABLE LogicalReplicationTest (id int PRIMARY KEY);
 - CREATE SUBSCRIPTION testsub CONNECTION 'host=localhost port=5432 user=postgres password=vignesh dbname=testdb' PUBLICATION testpub;

Basic Transaction Syntax:

```
BEGIN;  
-- SQL operations  
COMMIT;  -- Save changes  
-- or  
ROLLBACK; -- Undo changes
```

```
BEGIN;  
  
UPDATE accounts SET balance = balance - 100 WHERE id = 1;  
UPDATE accounts SET balance = balance + 100 WHERE id = 2;  
  
COMMIT;
```

Basic Transaction Syntax:

```
BEGIN;
```

```
UPDATE products SET stock = stock - 5 WHERE id = 100;
```

```
-- Oops: this fails due to constraint
```

```
INSERT INTO logs VALUES (NULL, 'bad insert');
```

```
ROLLBACK; -- Reverts the stock update too
```

Transaction Control Summary

Command	Purpose
BEGIN	Start transaction
COMMIT	Apply changes
ROLLBACK	Undo changes
SAVEPOINT name	Set save point inside tx
ROLLBACK TO SAVEPOINT name	Partial rollback

1. Window Functions

- Window functions perform calculations across a set of related rows **without collapsing the rows** as GROUP BY does.

- Consider:

```
CREATE TABLE vehicle_sales (  
    sale_id    INT,  
    model      VARCHAR(50),  
    city       VARCHAR(50),  
    sale_date  DATE,  
    price      NUMERIC(12,2),  
    quantity   INT  
);
```

1. Window Functions

- The general syntax is:
- `function_name(...)`
- `OVER (`
- `PARTITION BY ...`
- `ORDER BY ...`
- `ROWS BETWEEN ...`
- `)`
- Think of it as:
- `PARTITION BY` → divide rows into groups
- `ORDER BY` → arrange rows inside each group
- `FRAME` → choose rows around current row
- `FUNCTION` → perform calculation

2. ROW_NUMBER()

- ROW_NUMBER() assigns a unique sequential number.

```
SELECT
```

```
    city,
```

```
    model,
```

```
    price,
```

```
    ROW_NUMBER() OVER (
```

```
        PARTITION BY city
```

```
        ORDER BY price DESC
```

```
    ) AS row_num
```

```
FROM vehicle_sales;
```

2. ROW_NUMBER()

- **Top 3 models per city**

```
WITH ranked_sales AS (  
    SELECT  
        city,  
        model,  
        price,  
        ROW_NUMBER() OVER (  
            PARTITION BY city  
            ORDER BY price DESC  
        ) AS rn  
    FROM vehicle_sales  
)
```

2. ROW_NUMBER()

- SELECT *
- FROM ranked_sales
- WHERE rn <= 3;
- This combination of **CTE + window function** is extremely common.



3. RANK vs DENSE_RANK vs ROW_NUMBER

- Suppose prices are:
- Model Price
- -----
- i7 100
- X7 90
- X5 90
- iX 80

3. RANK vs DENSE_RANK vs ROW_NUMBER

- Query:
SELECT
 model,
 price,
 ROW_NUMBER() OVER (
 ORDER BY price DESC
) AS row_number,
 RANK() OVER (
 ORDER BY price DESC
) AS rank,
 DENSE_RANK() OVER (
 ORDER BY price DESC
) AS dense_rank
FROM vehicle_sales;



3. RANK vs DENSE_RANK vs ROW_NUMBER

Model	Price	ROW_NUMBER	RANK	DENSE_RANK
i7	100	1	1	1
X7	90	2	2	2
X5	90	3	2	2
iX	80	4	4	3

4. NTILE()

- NTILE(n) divides ordered data into approximately equal groups.

```
SELECT  
    model,  
    price,  
    NTILE(4) OVER (  
        ORDER BY price DESC  
    ) AS quartile FROM vehicle_sales;
```

- NTILE(4) creates four groups:
 - 1 → Top 25%
 - 2 → Next 25%
 - 3 → Next 25%
 - 4 → Bottom 25%

4. NTILE()

- Example:
- SELECT
- customer_id,
- total_purchase,
- NTILE(10) OVER (
- ORDER BY total_purchase DESC
-) AS customer_decile
- FROM customer_sales;
- NTILE(10) creates ten approximately equal buckets.

5. LAG()

- LAG() accesses a previous row.

- Suppose:

- Date Revenue

- -----

- 2026-01-01 100

- 2026-01-02 120

- 2026-01-03 150

5. LAG()

- SELECT
- sale_date,
- revenue,
- LAG(revenue) OVER (
- ORDER BY sale_date
-) AS previous_revenue
- FROM daily_sales;

5. LAG()



Date	Revenue	Previous

Jan 1	100	NULL
Jan 2	120	100
Jan 3	150	120

6. LEAD()

- LEAD() does the opposite of LAG().
- SELECT
 - sale_date, revenue,
 - LEAD(revenue) OVER (
 - ORDER BY sale_date
 -) AS next_revenue
 - FROM daily_sales;
- Conceptually:
- Current LAG LEAD
- 100 NULL 120
- 120 100 150
- 150 120 NULL

7. FIRST_VALUE()

- Returns the first value within the window.
- `SELECT`
 - `city,`
 - `model,`
 - `price,`
 - `FIRST_VALUE(model) OVER (`
 - `PARTITION BY city`
 - `ORDER BY price DESC`
 - `) AS most_expensive_model`
- `FROM vehicle_sales;`
- Every row in each city can display the highest-priced model.

8. LAST_VALUE()

- LAST_VALUE() has an important trap.
- This query may not produce what beginners expect:
- LAST_VALUE(price) OVER (
 - PARTITION BY city
 - ORDER BY price
 -)

9. PARTITION BY

- `SELECT city, model, price, SUM(price) OVER (PARTITION BY city ) AS city_total FROM vehicle_sales;`

10. ORDER BY Inside Window

- The ORDER BY inside OVER() is different from the final query's ORDER BY.
- SELECT
 - city,
 - model,
 - price,
 - ROW_NUMBER() OVER (
 - PARTITION BY city
 - ORDER BY price DESC
 -) AS rank_number
- FROM vehicle_sales
- ORDER BY city;

CTEs — Common Table Expressions

- A **CTE (Common Table Expression)** is a temporary named result set defined using the WITH clause. It exists only for the duration of the SQL statement.
- **Basic syntax**

```
WITH cte_name AS (  
    SELECT column1, column2  
    FROM table_name  
    WHERE condition  
)  
SELECT *  
FROM cte_name;
```

CTEs — Common Table Expressions

- WITH electric_sales AS (
 - SELECT
 - model,
 - city,
 - price,
 - quantity
 - FROM vehicle_sales
 - WHERE fuel_type = 'Electric'
 -)
- SELECT *
- FROM electric_sales;

CTEs — Common Table Expressions

- WITH city_revenue AS (
 - SELECT
 - city,
 - SUM(price * quantity) AS total_revenue
 - FROM vehicle_sales
 - GROUP BY city
 -)
- SELECT *
- FROM city_revenue
- WHERE total_revenue > 10000000;

Multiple CTEs — Common Table Expressions

- WITH electric_sales AS (
 - SELECT *
 - FROM vehicle_sales
 - WHERE fuel_type = 'Electric'
 -),
- city_summary AS (
 - SELECT
 - city,
 - SUM(price * quantity) AS revenue
 - FROM electric_sales
 - GROUP BY city
 -)
 - SELECT *
 - FROM city_summary
 - ORDER BY revenue DESC;

CTE with Window Function

- WITH ranked_sales AS (
 - SELECT
 - city,
 - model,
 - price,
 - ROW_NUMBER() OVER (
 - PARTITION BY city
 - ORDER BY price DESC
 -) AS rn
- FROM vehicle_sales
-)

CTE with Window Function

- SELECT
- city,
- model,
- price
- FROM ranked_sales
- WHERE rn = 1;

CTE vs SubQuery

Feature	CTE	Subquery
Syntax	WITH name AS (...)	(SELECT ...)
Readability	Excellent for complex queries	Good for small queries
Can reference earlier result	Yes	Less convenient
Recursive queries	Yes	Normally no
Exists permanently	No	No
Useful with window functions	Excellent	Possible but less readable

Postgres Security

-  1. AuthenticationControls who can connect to the database.
- Configured in `pg_hba.conf` (Host-Based Authentication)
- Common methods:
 - trust: No password (not recommended for production)
 - md5 / scram-sha-256: Password-based (md5 deprecated gradually)
 - peer: OS user must match database user (local Unix)
 - cert: SSL certificate authentication
 - ldap, gss, kerberos, pam, radius: External systems

Postgres Security

- Select name,setting from pg_settings;

name	setting
allow_in_place_tablespaces	off
allow_system_table_mods	off
application_name	psql
archive_cleanup_command	
archive_command	(disabled)
archive_mode	off
archive_timeout	0
array_nulls	on
authentication_timeout	60
autovacuum	on
autovacuum_analyze_scale_factor	0.1
autovacuum_analyze_threshold	50
autovacuum_freeze_max_age	200000000
autovacuum_max_workers	3
autovacuum_multixact_freeze_max_age	400000000
autovacuum_naptime	60
autovacuum_vacuum_cost_delay	2
autovacuum_vacuum_cost_limit	-1
autovacuum_vacuum_scale_factor	0.2
autovacuum_vacuum_threshold	50
autovacuum_work_mem	-1
backend_flush_after	0
backslash_quote	safe_encoding
bgwriter_delay	200
bgwriter_flush_after	64
bgwriter_lru_maxpages	100
bgwriter_lru_multiplier	2
block_size	8192
bonjour	off
bonjour_name	
bytea_output	hex
check_function_bodies	on
checkpoint_completion_target	0.5
checkpoint_flush_after	32
checkpoint_timeout	300

Postgres Security

-  **2. Authorization (Roles & Privileges)**
- Controls what a user can do.
-  **Roles**
 - Superusers (CREATE ROLE myuser SUPERUSER)
 - Normal users (LOGIN privilege)
 - Can be granted or revoked:

```
CREATE ROLE readonly;  
GRANT CONNECT ON DATABASE mydb TO readonly;  
GRANT USAGE ON SCHEMA public TO readonly;  
GRANT SELECT ON ALL TABLES IN SCHEMA public TO readonly;
```

Postgres Security



```
CREATE ROLE appuser LOGIN PASSWORD 'secret'  
NOSUPERUSER NOCREATEDB NOCREATEROLE  
NOINHERIT NOREPLICATION;
```

Object	Privileges
Table	SELECT, INSERT, UPDATE, DELETE, TRUNCATE, REFERENCES, TRIGGER
Database	CONNECT, TEMP
Schema	USAGE, CREATE
Function	EXECUTE

Postgres Security Object Level Permission

```
GRANT SELECT ON employees TO readonly;  
REVOKE DELETE ON employees FROM appuser;
```

Object	Privileges
Table	SELECT, INSERT, UPDATE, DELETE, TRUNCATE, REFERENCES, TRIGGER
Database	CONNECT, TEMP
Schema	USAGE, CREATE
Function	EXECUTE

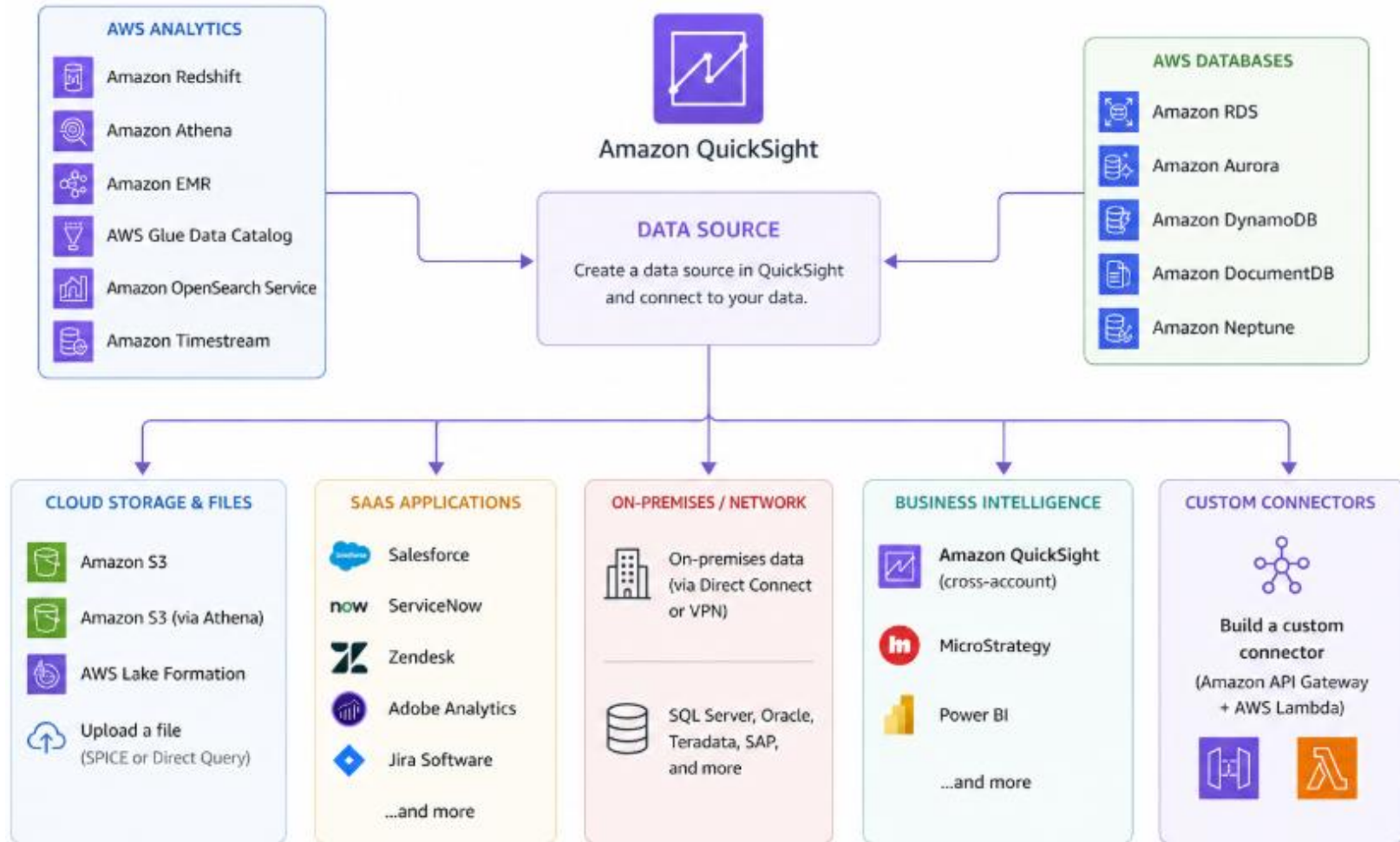


Postgres Security Row-Level Security (RLS)

```
ALTER TABLE employees ENABLE ROW LEVEL SECURITY;  
CREATE POLICY hr_policy  
  ON employees  
  USING (department = current_setting('app.current_department'));  
ALTER TABLE employees ENABLE POLICY hr_policy;
```

Use SET app.current_department = 'HR'; at session level.

AWS QuickSight – Data Source Connections



QuickSight supports SPICE (in-memory) and Direct Query connections (live) depending on the data source. Choose the right connection type based on performance, data freshness, and use case.



AWS QuickSight – Data Source Connections

- **Amazon S3** – Connect to CSV/JSON files in S3, usually through an S3 manifest file or supported dataset ingestion workflow.
- **Amazon RDS** – Connect directly to relational databases such as **PostgreSQL, MySQL, MariaDB, SQL Server, and Oracle**, depending on the RDS engine.
- **Amazon Athena** – Query data stored in S3 using Athena tables and use the query results for QuickSight dashboards.
- **Amazon Redshift** – Connect to Redshift clusters/serverless warehouses for large-scale analytical datasets.
- **Snowflake** – Connect using the Snowflake account/server, database, warehouse, username, and credentials.
- **CSV Upload** – Upload a local .csv file directly into QuickSight for small or training datasets.

Amazon S3



Amazon S3

>

Buckets

>

bmwsensordata

Info

Objects

Metadata

Properties

Permissions

Metrics

Management

File systems - new

Amazon S3

<

▼ Buckets

General purpose buckets

Directory buckets

Table buckets

Vector buckets

▼ Files

File systems

▼ Access management and security

Access Points

Access Points for FSx

Access Grants

IAM Access Analyzer

▼ Storage management and

bmwsensordata

Info

<

Objects

Metadata

Properties

Permissions

Metrics

Management

File systems - new

>

Objects (0)

↺

Copy S3 URI

↺

Copy URL

⬇

Download

↗

Open

🗑

Delete

⌵

Actions

Create folder

⬆

Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

🔍 Find objects by prefix

🔌 Show versions

<

1

>

⚙

☐

Name

▲

Type

▼

Last modified

▼

Size

▼

Storage class

▼

No objects

You don't have any objects in this bucket.

329

Amazon S3



```
(pysparkenv) E:\Datascience2026\pysparkws>aws configure
AWS Access Key ID [*****OX62]:
AWS Secret Access Key [*****CP4s]:
Default region name [us-east-1]:
Default output format [None]:

(pysparkenv) E:\Datascience2026\pysparkws>
(pysparkenv) E:\Datascience2026\pysparkws>aws sts get-caller-identity
{
  "UserId": "AIDAZCDME2XPXV5B6IWCR",
  "Account": "622997067231",
  "Arn": "arn:aws:iam::622997067231:user/eswaribala"
}
```

Amazon S3



```
(pysparkenv) E:\Datascience2026\pysparkws>aws configure
AWS Access Key ID [*****OX62]:
AWS Secret Access Key [*****CP4s]:
Default region name [us-east-1]:
Default output format [None]:

(pysparkenv) E:\Datascience2026\pysparkws>
(pysparkenv) E:\Datascience2026\pysparkws>aws sts get-caller-identity
{
  "UserId": "AIDAZCDME2XPXV5B6IWCR",
  "Account": "622997067231",
  "Arn": "arn:aws:iam::622997067231:user/eswaribala"
}
```

Amazon S3



```
src > utils > aws_s3.py > ...
```

```
1  import boto3
2
3  bucket_name = "bmwsensordata"
4  local_file = "src/data/bmw_connected_cars.csv"
5  s3_file_name = "sensor_data/bmw_connected_cars.csv"
6
7  s3 = boto3.client(
```

Problems

Terminal 1

Terminal

```
(pysparkenv) E:\Datascience2026\pysparkws>python src/utils/aws_s3.py
```

```
File uploaded successfully
```

```
s3://bmwsensordata/sensor_data/bmw_connected_cars.csv
```

```
(pysparkenv) E:\Datascience2026\pysparkws>
```




Amazon S3 with Pypark and Hadoop

- set HADOOP_HOME=D:\software\hadoop-3.5.0\hadoop-3.5.0
- set PATH=%HADOOP_HOME%\bin;%PATH%
- Verify Hadoop:
 - echo %HADOOP_HOME%
- where winutils
- Then verify AWS authentication:
- aws sts get-caller-identity
- Check your S3 location:
- aws s3 ls s3://bmwsensordata/sensor_data/



Amazon S3 with Pypark and Hadoop

- To expose AWS CLI credentials as environment variables for PySpark/S3A:
- `aws configure export-credentials --format windows-cmd`
- Or set them manually from the AWS CLI profile:
- `for /f "delims=" %i in ('aws configure get aws_access_key_id') do @set "AWS_ACCESS_KEY_ID=%i"`
- `for /f "delims=" %i in ('aws configure get aws_secret_access_key') do @set "AWS_SECRET_ACCESS_KEY=%i"`
- Then set the AWS region:
 - `set AWS_DEFAULT_REGION=us-east-1`
 - `set AWS_REGION=us-east-1`
- Verify everything:
 - `echo %AWS_ACCESS_KEY_ID%`
 - `echo %AWS_DEFAULT_REGION%`

Amazon S3 with Pypark and Hadoop

```

pyspark_s3.py U X
src > utils > pyspark_s3.py > ...
1 from pyspark.sql import SparkSession
2
3 spark = (
4     SparkSession.builder
5         .appName("BMW S3")
6         .master("local[*]")
7         .config(
8             "spark.jars.packages",
9             "org.apache.hadoop:hadoop-aws:3.5.0"
10        )
11        .config(
12            "spark.hadoop.fs.s3a.impl",
13            "org.apache.hadoop.fs.s3a.S3AFileSystem"

```

Problems Terminal 1

> Terminal

V003	i4	Electric	Asia	45	130	2024-07-01
V004	3 Series	Diesel	North America	0	80	2024-07-01
V005	iX	Electric	Europe	15	60	2024-07-01
V006	X3	Petrol	Asia	0	105	2024-07-01
V007	i4	Electric	North America	90	140	2024-07-01
V008	5 Series	Diesel	Europe	0	70	2024-07-01
V009	iX	Electric	Asia	8	50	2024-07-01
V010	X1	Petrol	North America	0	85	2024-07-01

Amazon S3



[bmwsensordata](#) > [sensor_data/](#)

sensor_data/

Copy S3 URI

Objects

Properties

Objects (1)



Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create folder

Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

Show versions

< 1 >



☐	Name	Type	Last modified	Size	Storage class
☐	bmw_connected_cars.csv	csv	August 29, 2026, 13:41:46 (UTC+05:30)	711.0 B	Standard

AWS RDS



bmw-cluster-db



☐ Code snippets

Use when connecting through SDK, APIs, or third-party tools including agents.

☐ CloudShell

Use for a quick access to AWS CLI that launches directly from the AWS Management Console.

☒ Endpoints

Use when connecting through any IDE interface.

Database name

postgres

Master username

postgres

Internet access gateway

Enabled

Port

5432

IAM authentication token

[Get token](#)

Endpoints (2)

Actions ▼

[Create custom endpoint](#)

Filter resources

< 1 >

	Endpoint name ▲	Status ▼	Type ▼	Port ▼
	bmw-cluster-db.cluster-cy9am4cyomlc.us-east-1.rds.amazonaws.com	Available	Writer	5432
	bmw-cluster-db.cluster-ro-cy9am4cyomlc.us-east-1.rds.amazonaws.com	Available	Reader	5432

AWS RDS - Postgres



SQLTools

Connections

- rdsgn postgres@bmw-cluster-...
- postgres database
 - Schemas
 - public
 - Event triggers
 - Nothing here
- Bookmarks

SQLTools Settings x rdsconn.session.sql

Detach file from rdsconn | ▶Run on active connection | ≡Select block | ☞Active: localhost | bmwdb

1

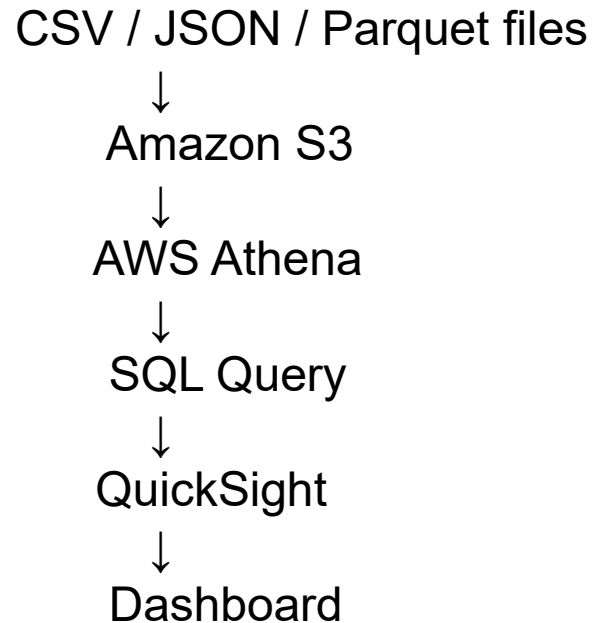
select * from public.customers; Untitled-1

▶Run | +Tab | JSON | ☞Active: localhost | bmwdb | ▶Run on active connection | ≡Select block

1 select * from public.customers;

AWS Athena

- **AWS Athena** is a **serverless SQL query service** used to analyze data stored mainly in **Amazon S3** without setting up a database server.

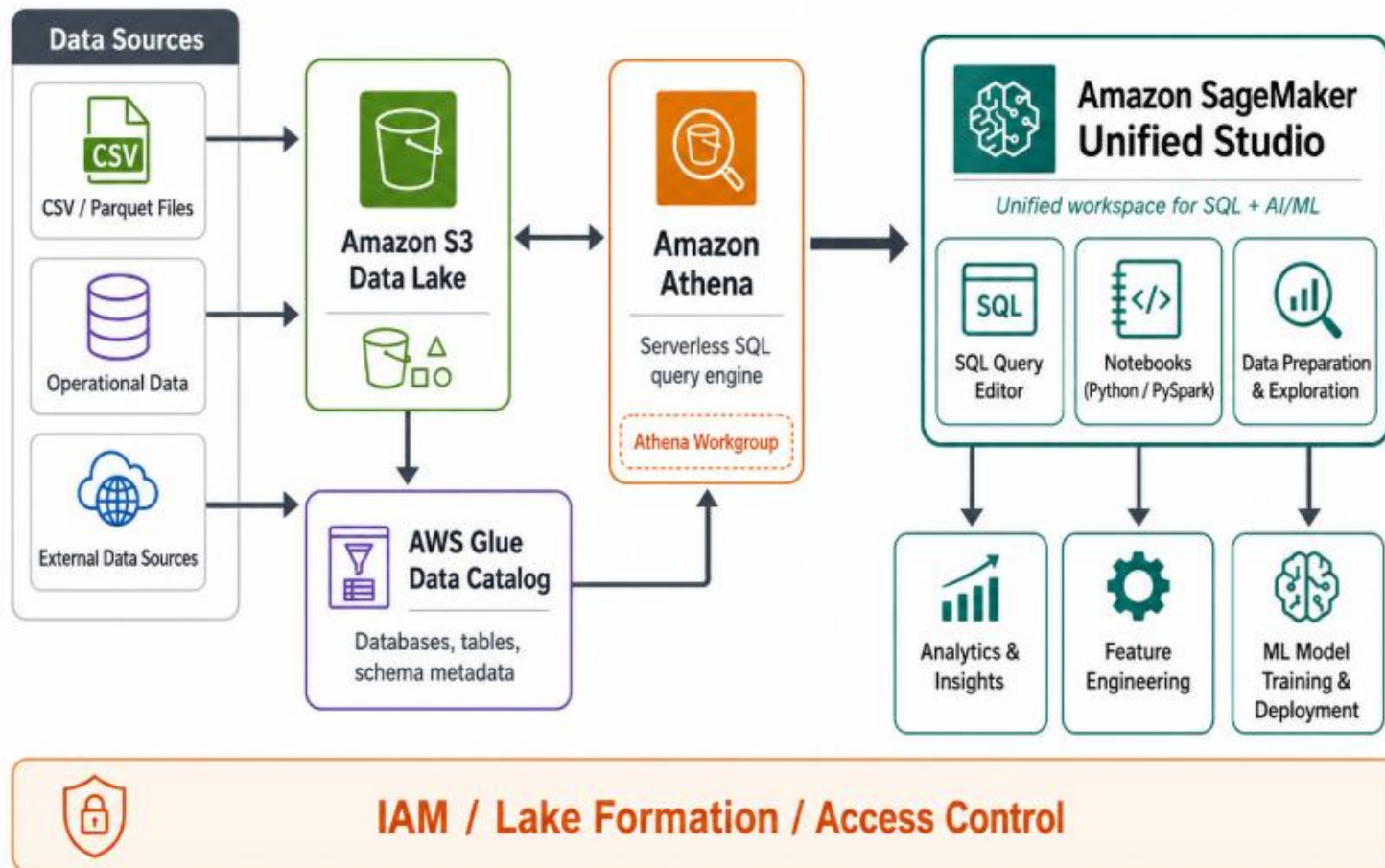


Amazon SageMaker Unified Studio



Amazon SageMaker Unified Studio + Amazon Athena Architecture

Query data in S3 with Athena and analyze/build ML workflows in SageMaker Unified Studio



AWS Athena



Dat... ⓘ ↺ ✖ + Add

▼ Catalogs

▼ AwsDataCatalog

> bmw_database

▼ default

No data

▼ s3tablescatalog

No data

▼ Connections

No data

▼ Buckets

> amazon-sagemaker...

▼ bmwsensordata

▼ sensor_data/

bmw_connect...

Untitled 1 ✖ +

▶ Run all

AwsDataCatalog • Database ▼

Athena (SQL) ▼

🔗 📄 🕒 ✨

▶ | 🔍 Limit 100 #1

1 CREATE DATABASE bmw_database;

Row 1, Col 30, Chr 29

Result 1

📄 📊 | ✖

✔ Query executed successfully

```
CREATE EXTERNAL TABLE bmw_connected_cars (  
    vehicle_id STRING,  
    model STRING,  
    fuel_type STRING,  
    region STRING,  
    battery_level INT,  
    speed INT,  
    last_update STRING  
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
LOCATION 's3://bmwsensordata/sensor_data/'  
TBLPROPERTIES (  
    'skip.header.line.count'='1'  
);
```

AWS Athena



Dat... ⓘ ↺ ✱ + Add

▼ Catalogs

▼ AwsDataCatalog

- > bmw_database
- ▼ default
 - > tables (1)
 - > views (0)

▼ s3tablescatalog
No data

▼ Connections
No data

▼ Buckets

- > amazon-sagemaker...
- ▼ bmwsensordata
 - ▼ sensor_data/
 - bmw_connect...

Untitled 1 x +

▶ Run all

AwsDataCatalog • bmw_database ▼

Athena (SQL) ▼

🔗 📄 🕒 ✨ ⋮

▶ | Limit 100 #1

```
1 CREATE EXTERNAL TABLE bmw_connected_cars (  
2     vehicle_id STRING,  
3     model STRING,  
4     fuel_type STRING,  
5     region STRING,  
6     battery_level INT,  
7     speed INT,  
8     last update STRING
```

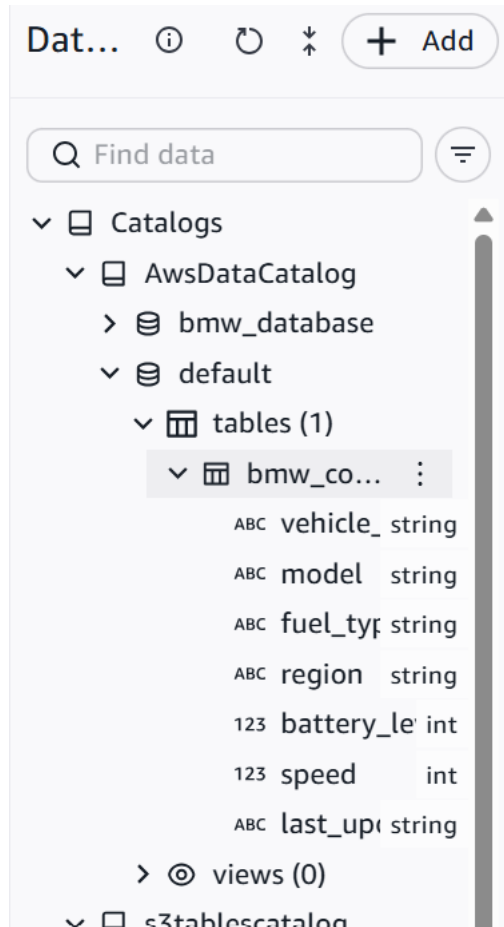
Row 5, Col 19, Chr 342

Result 1

📄 |📊 | ✕

✔ Query executed successfully

AWS Athena



AWS Athena



Dat... ⓘ ↺ ✱ + Add

Find data

- ✓ Catalogs
 - ✓ AwsDataCatalog
 - > bmw_database
 - ✓ default
 - ✓ tables (1)
 - ✓ bmw_co...
 - ABC vehicle_ string
 - ABC model string
 - ABC fuel_ty string
 - ABC region string
 - 123 battery_le int
 - 123 speed int
 - ABC last_up string
 - > views (0)
 - ✓ s3tablescatalog

No data

Untitled 1 × +

Run all AwsDataCatalog • default ▾

Athena (SQL) ▾

Limit 100 #1

```
1 SELECT * FROM "AwsDataCatalog"."default"."bmw_connected_cars";
```

Row 1, Col 63, Chr 62

Result 1

vehicle_id	model	fuel_type	re
V001	iX	Electric	
V002	X5	Petrol	
V003	i4	Electric	

AWS Athena



Feature	AWS Athena	PostgreSQL / RDS
What it is	Serverless SQL query service	Relational database server
Where data lives	Usually S3	Inside database storage
Server required	No	Yes
Installation/management	None	Database instance must run
Best for	Analytics on files/data lake	Application CRUD and transactions
INSERT/UPDATE/DELETE	Limited/not its main purpose	Fully supported
Transactions	Not designed for OLTP transactions	Full ACID transactions
Query frequency	Occasional/analytical queries	Continuous application queries
Scaling	Automatic/serverless	Instance/server sizing required
Pricing	Mainly data scanned per query	Pay for running DB instance/storage
Typical formats	CSV, JSON, Parquet, ORC	Tables/rows stored by PostgreSQL
Primary use	Data lake analytics	Operational application database

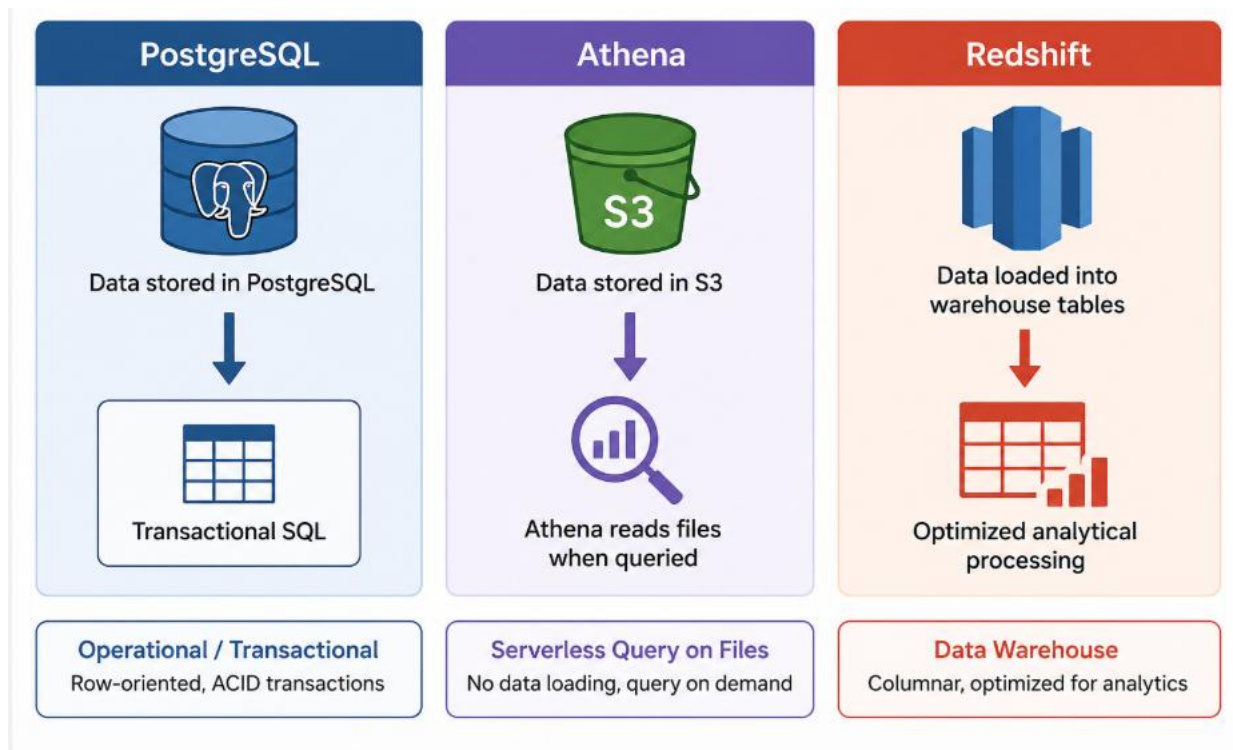


AWS Redshift

- Amazon **Redshift** is AWS's managed **cloud data warehouse**.
- It is designed for large-scale analytics using SQL across millions or billions of rows.

Service	Main purpose
PostgreSQL / RDS	Application database, CRUD, transactions
Athena	Serverless SQL directly on S3 files
Redshift	High-performance analytical data warehouse
S3	Object storage / data lake

Key Architectural Difference



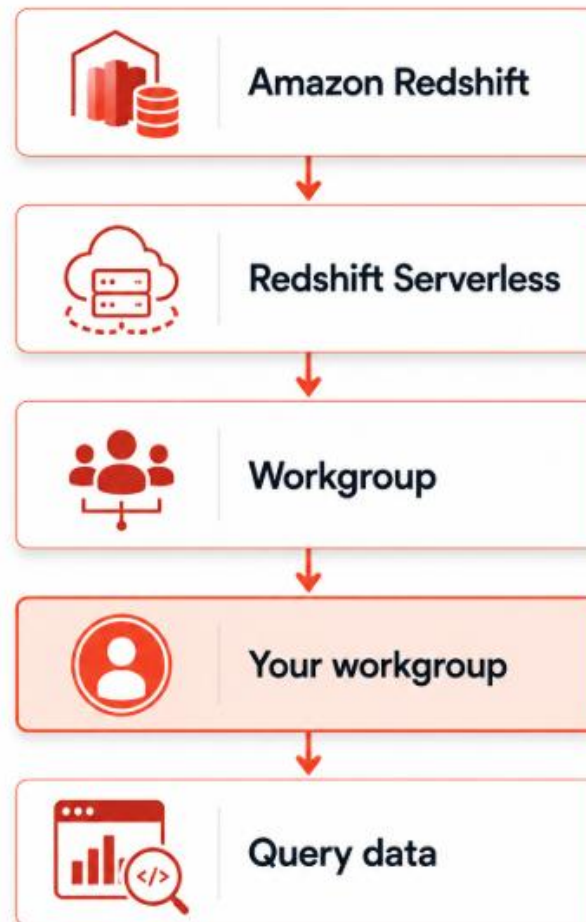
Athena versus Redshift

Feature	Athena	Redshift
Type	Serverless query engine	Data warehouse
Main data location	S3	Redshift warehouse
Setup	Very little	Warehouse/workgroup setup
Best for	Ad-hoc S3 queries	Repeated BI/analytics workloads
Performance	Depends heavily on files scanned	Optimized warehouse performance
Data loading	Usually unnecessary	Typically load/transform data
QuickSight	Yes	Yes
Large repeated joins	Possible, but not ideal for every workload	Strong use case

Query Data Flow



Amazon Redshift Serverless – Query Data Flow





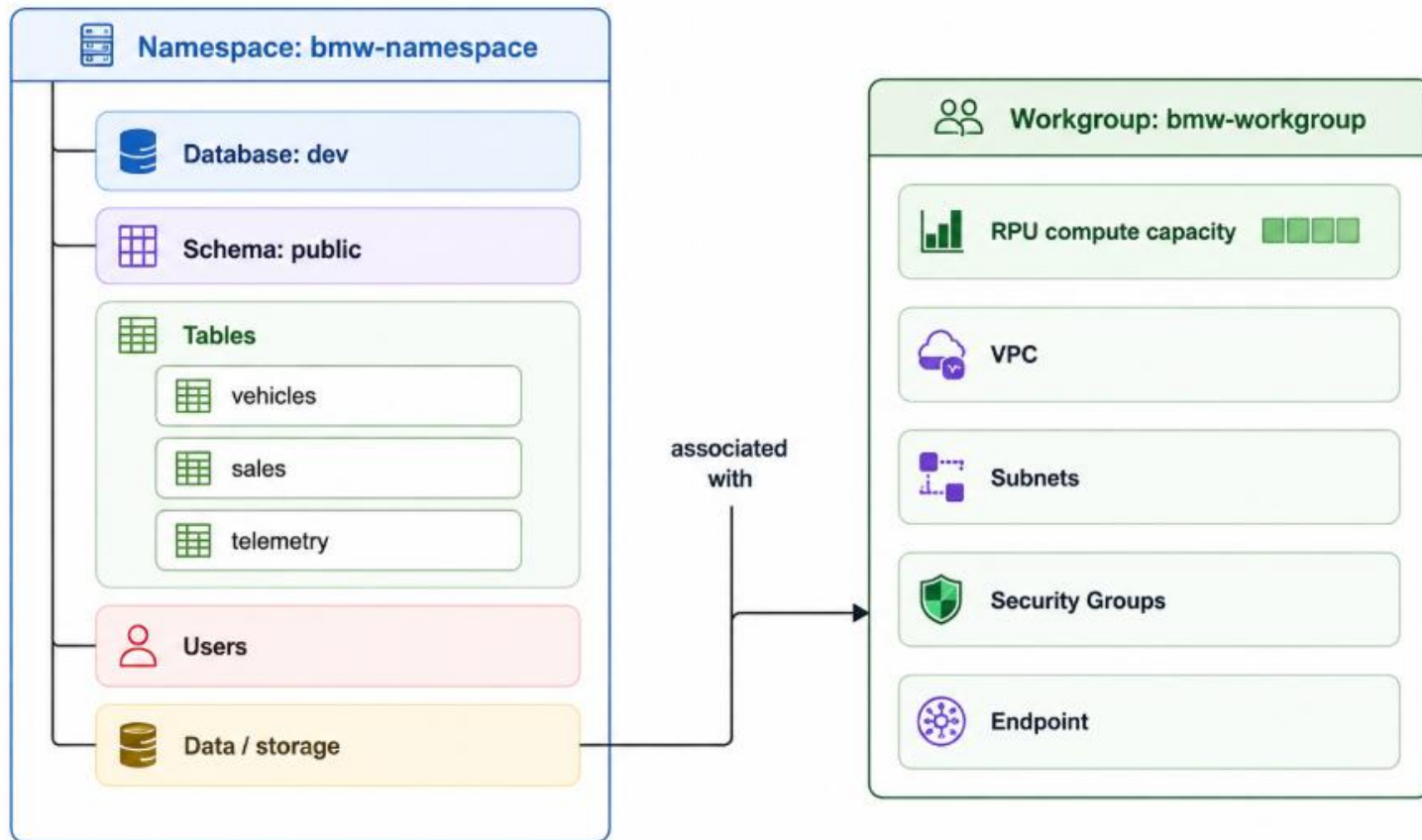
Query Data Flow

Namespace	Workgroup
Logical container for your Redshift data	Compute environment that runs SQL
Contains databases, schemas, tables	Contains Redshift Processing Units (RPU's)
Contains database users	Controls compute capacity/performance
Encryption / KMS settings	VPC, subnets, security groups
Datashares, recovery points	Provides endpoint/JDBC connection
Think Storage + Database objects	Think Compute + Network

Query Data Flow



Amazon Redshift Serverless





Query Data Flow – Use Cases

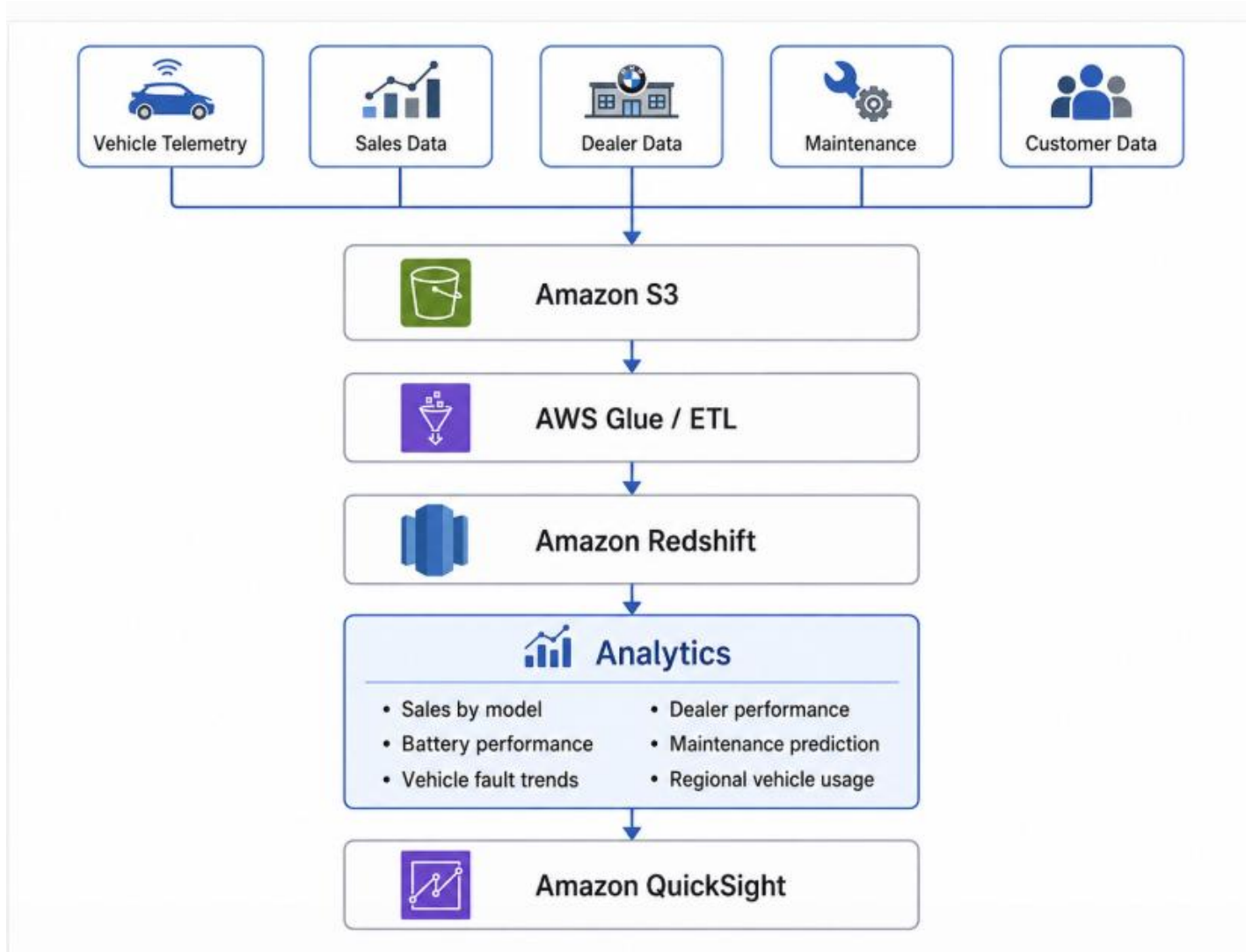
- **Sales analytics** — revenue by region, product, dealer, month, customer segment.
- **Customer 360** — combine CRM, website, purchase, support, and campaign data into one analytical view.
- **IoT / vehicle analytics** — analyze telemetry such as speed, battery level, faults, location, charging, and usage patterns.
- **Manufacturing analytics** — production volume, defects, downtime, OEE, plant performance, supplier quality.
- **Financial reporting** — revenue, cost, profitability, budget vs actual, forecasting datasets.
- **Log and event analytics** — application logs, clickstream, API events, device events, security events.



Query Data Flow – Use Cases

- **BI dashboards** — Redshift as the warehouse behind Amazon QuickSight, Tableau, Power BI, or Looker.
- **Machine-learning datasets** — prepare historical analytical data for SageMaker or other ML platforms.
- **Data lake analytics** — query warehouse tables together with data stored in S3 using Redshift Spectrum.
- **Near-real-time analytics** — ingest streaming events from services such as Kinesis or MSK and make them available for operational dashboards.

Query Data Flow – Use Cases



Query Data Flow

- **Confirm the CSV is in S3**
- For example:
- `s3://bmwsensordata/sensor_data/bmw_connected_cars.csv`
- Redshift will read this file from S3 and load it into the warehouse table.

- The flow is:

S3 file



COPY



Redshift table

Query Data Flow

- **Give Redshift permission to read S3**
- This must be done **before COPY will work.**
- Go to:

AWS Console



Amazon Redshift



Redshift Serverless



Namespace configuration



Select namespace used by bmwvvg



Security and encryption



Permissions / IAM roles

[Amazon Redshift Serverless](#) > [Namespace configuration](#) > [bmwns](#) > Manage IAM roles

Manage IAM roles

Permissions

i Associate an IAM role so that your serverless endpoint can LOAD and UNLOAD data. You can create an IAM role as the default for this configuration that has the [AmazonRedshiftAllCommandsFullAccess](#)  policy attached. This policy includes permissions to run SQL commands to COPY, UNLOAD, and query data with Amazon Redshift Serverless. This policy also grants permissions to run SELECT statements for related services, such as Amazon S3, Amazon CloudWatch logs, Amazon SageMaker, and AWS Glue. You won't be able to run these SQL commands without an IAM role attached to your namespace.

Associated IAM roles (0)

Set default ▼

Actions ▼

Associate IAM roles

Create IAM role

Create, associate, or remove an IAM role. You can associate up to 50 IAM roles. You can also choose an IAM role and set it as the default.

Query Data Flow

configuration that has the [AmazonRedshiftAllCommandsFullAccess](#) policy attached. This policy includes permissions to run SQL commands to COPY, UNLOAD, and query data with Amazon Redshift Serverless. This policy also grants permissions to run SELECT statements for related services, such as Amazon S3, Amazon CloudWatch logs, Amazon SageMaker, and AWS Glue. You won't be able to run these SQL commands without an IAM role attached to your namespace.

Select Access Type

Select access type for S3 bucket.

- ☐ Read Only
- ☒ Full Access

Specify an S3 bucket for the IAM role to access

To create a new bucket, [visit S3](#)

- ☐ No additional S3 bucket
Create the IAM role without specifying S3 buckets.
- ☒ Any S3 bucket
Allow users that have access to your Redshift Serverless data to also access any S3 bucket and its contents in your AWS account.
- ☐ Specific S3 buckets
Specify one or more S3 buckets that the IAM role being created has permission to access.

 An IAM role is currently assigned as the default role. If you create an IAM role as default, this new IAM role replaces as the default role.

Cancel

Create IAM role as default

Query Data Flow



Not registered - OTT

Workgroup | Data backup | Database | **Security and encryption** | Datashares | Integrations | Resource policy | Tags

Permissions [Info](#) Manage IAM roles

IAM roles	Status	Amazon Resource Name (ARN)
AmazonRedshift-CommandsAccessRole-20260829T183959	⌚ adding	arn:aws:iam::622997067231:role/service-role/AmazonRedshift-CommandsAccessRol

Query Data Flow – Edit IAM policy

Workgroup | Data Backup | Database | **Security and encryption** | Datashares | Integrations | Resource policy | Tags

Permissions [Info](#) Manage IAM roles

IAM roles	Status	Amazon Resource Name (ARN)
AmazonRedshift-CommandsAccessRole-20260830T074933	✓ in-sync	arn:aws:iam::622997067231:role/service-role/AmazonRedshift-CommandsAccessRo

AmazonRedshift-CommandsAccessRole-20260830T074933 [Info](#) Delete

Summary Edit

Creation date August 30, 2026, 07:49 (UTC+05:30)	ARN  arn:aws:iam::622997067231:role/service-role/AmazonRedshift-CommandsAccessRole-20260830T074933
Last activity ✓ 36 minutes ago	Maximum session duration 1 hour

[Permissions](#) | [Trust relationships](#) | [Tags](#) | [Last Accessed](#) | [Revoke sessions](#)



Query Data Flow – Edit IAM policy

Last activity

✓ 36 minutes ago

Maximum session duration

1 hour

Permissions

Trust relationships

Tags

Last Accessed

Revoke sessions

Permissions policies (2) [Info](#)



[Simulate](#)

[Remove](#)

[Add permissions](#)

You can attach up to 20 managed policies.

Filter by Type

All types

< 1 >



☐	Policy name ↗	Type	Attached entities
☐	AmazonRedshift-CommandsA...	Customer managed	1
☐	AmazonRedshiftAllComm...	AWS managed	2

Query Data Flow – Edit IAM policy

☐	Policy name ↗	Type ▲	Attached entities ▼
☐	AmazonRedshift-CommandsA...	Customer managed	1

AmazonRedshift-CommandsAccessPolicy-20260830T074933

[Copy JSON](#) [Edit ↗](#)

```
20     ]
21   },
22   {
23     "Sid": "S3ObjectLevelAccess",
24     "Effect": "Allow",
25     "Action": [
26       "s3:GetObject",
27       "s3:PutObject",
28       "s3:DeleteObject",
29       "s3:AbortMultipartUpload",
30       "s3:ListMultipartUploadParts"
31     ],
32     "Resource": [
33       "arn:aws:s3:::redshift/*",
34       "arn:aws:s3:::bmwsensordata/*"
35     ]
36   },
37   {
38     "Sid": "S3ListAllBuckets",
39     "Effect": "Allow",
```



Query Data Flow – Edit IAM policy

- Refer json file link
- [E:\AIML\athena.json](#)

Query Data Flow



✓ You have successfully created bmwwg and attached it to bmwns

[Amazon Redshift Serverless](#) > [Workgroup configuration](#) > bmwwg



This workgroup is associated with a namespace

To manage database objects and users, navigate to the namespace that this workgroup is associated with.

bmwns



bmwwg Info

Actions ▼

Query data ▲

Query in query editor v2 ↗

Query in Amazon SageMaker Unified Studio ↗

General information

Workgroup

bmwwg

Namespace

[bmwns](#)

Workgroup ARN

arn:aws:redshift-serverless:us-east-

Date created

August 29, 2026, 16:47 (UTC+05:30)

Price-performance target

50 (Balanced)

Custom domain name

-

Endpoint

bmwwg.622997067231.us-east-1.redshift-serverless.amazonaws.com:5439/dev

JDBC URL

jdbc:redshift://bmwwg.622997067231.us-east-1.redshift-serverless.amazonaws.com:5439/dev

Query Data Flow



Redshift query editor v2

Create

Load data

Filter resources

>

Serverless: bmwwg

>

Serverless: default-workgroup

×

Connect to bmwwg

☐

IAM Identity Center
Connect to Amazon Redshift with your single sign-on credentials from your identity provider (IdP). Your cluster or workgroup must be enabled for IAM Identity Center.

☒

Other ways to connect [Learn more](#)

☒

Federated user
The query editor v2 generates a temporary password to connect to the database.

☐

Database user name and password
Provide a database user and password for the database that you are connecting to. The query editor v2 stores your credentials in AWS Secrets Manager on your behalf.

☐

AWS Secrets Manager
Choose a secret with credentials that are associated with the namespace or that you created in AWS Secrets Manager. Only secrets tagged with a key starting with 'Redshift' are listed.

Database

dev

The database name must be 1-64 characters. Valid characters are lowercase alphanumeric characters.

Cancel

Create connection

Query Data Flow



Redshift query editor v2

⊕ Create ▾ ⬆ Load data ⏪

🔍 Filter resources ↻

- ✓ 🌱 Serverless: bmwwg ⓘ
 - ✓ 🗄 native databases (2)
 - ✓ 📁 dev
 - ✓ 📁 public
 - ✓ 📊 Tables 0
 - > 👁 Views 0
 - > ⚡ Functi... 0
 - > ➡ Stored...0
 - > 📁 sample_data_dev
 - > 🗄 external databases (1)
 - > 🌱 Serverless: default-workgroup ⓘ

+ ☰ Untitled 1 x

▶ Run ^ 🔵 Limit 100 ⚪ Explain 🔵 Isolated session ⓘ

📅 Schedule 💾 ↺ ⋮

```
1 SELECT
2   current_database(),
3   current_user;
```

📊 Result 1 (1)

☐	current_database	current_user
☐	dev	IAM:RootIdentity

Query Data Flow



Redshift query editor v2

⊕ Create ▾ ⬆ Load data ⏪

🔍 Filter resources ↻

- ▼ Serverless: bmwwg ⓘ
 - ▼ native databases (2)
 - ▼ dev
 - ▼ public
 - ▼ Tables 0
 - > Views 0
 - > f λ Functi... 0
 - > Stored...0
 - > sample_data_dev
 - > external databases (1)
 - > Serverless: default-workgroup ⓘ

+ ≡ Untitled 1 ×

▶ Run ⬆ ☒ Limit 100 ☐ Explain ☒ Isolated

📅 Schedule 💾 ↺ ⋮

```
1 CREATE SCHEMA IF NOT EXISTS bmw;
2
3
4 SELECT schema_name
5 FROM information_schema.schemata
6 WHERE schema_name = 'bmw';
7
```

📊 Result 1 (1)

☐	schema_name
☐	bmw

Query Data Flow



Redshift query editor v2

⊕ Create ▾ ⬆ Load data ⏪

🔍 Filter resources ↻

- ▼ Serverless: bmwwg ⓘ
 - ▼ native databases (2)
 - ▼ dev
 - ▼ public
 - ▼ Tables 0
 - > Views 0
 - > f λ Functi... 0
 - > Stored...0
 - > sample_data_dev
 - > external databases (1)
 - > Serverless: default-workgroup ⓘ

+ ≡ Untitled 1 ×

▶ Run ⬆ ☒ Limit 100 ☐ Explain ☒ Isolated

📅 Schedule 💾 ↺ ⋮

```
1 CREATE SCHEMA IF NOT EXISTS bmw;
2
3
4 SELECT schema_name
5 FROM information_schema.schemata
6 WHERE schema_name = 'bmw';
7
```

📊 Result 1 (1)

☐	schema_name
☐	bmw

Query Data Flow



Redshift query editor v2

Create

Load data

Filter resources

Serverless: bmwwg

native databases (2)

dev

public

Tables 0

Views 0

Funci... 0

Stored... 0

sample_data_dev

external databases (1)

Serverless: default-workgroup

Untitled 1

Run

Limit 100

Explain

Isolated session

Schedule

```
1 CREATE TABLE IF NOT EXISTS bmw.connected_cars (  
2     vehicle_id VARCHAR(30),  
3     model VARCHAR(50),  
4     fuel_type VARCHAR(30),  
5     region VARCHAR(50),  
6     battery_level INT,  
7     speed INT,  
8     last_update VARCHAR(50)  
9 );  
10
```

Result 1

Summary

Returned rows: 0

Elapsed time: 281ms

Result set query:



Query Data Flow

- COPY `bmw.connected_cars`
- FROM
`'s3://bmwsensordata/sensor_data/bmw_connected_cars.csv'`
- FORMAT AS CSV
- IGNOREHEADER 1
- REGION 'us-east-1'
- IAM_ROLE 'arn:aws:iam::622997067231:role/service-role/AmazonRedshift-CommandsAccessRole-20260829T183959';

Query Data Flow



Redshift query editor v2

Filter resources

Serverless: default-workgr...
native databases (2)
dev
bmw
Tables 1
c...
Views 0
Funci... 0
Stored...0
public
sample_data_dev
external databases (1)

Run Limit 100 Explain Isolated session Serverless: de... dev

Schedule

```
1 COPY bmw.connected_cars
2 FROM 's3://bmwsensordata/sensor_data/bmw_connected_cars.csv'
3 FORMAT AS CSV
4 IGNOREHEADER 1
5 REGION 'us-east-1'
6 IAM_ROLE 'arn:aws:iam::622997067231:role/service-role/AmazonRedshift-CommandsAccessRole-20260830T074933';
```

Row 6, Col 106, Chr 2

Result 1

Export Chart

Summary

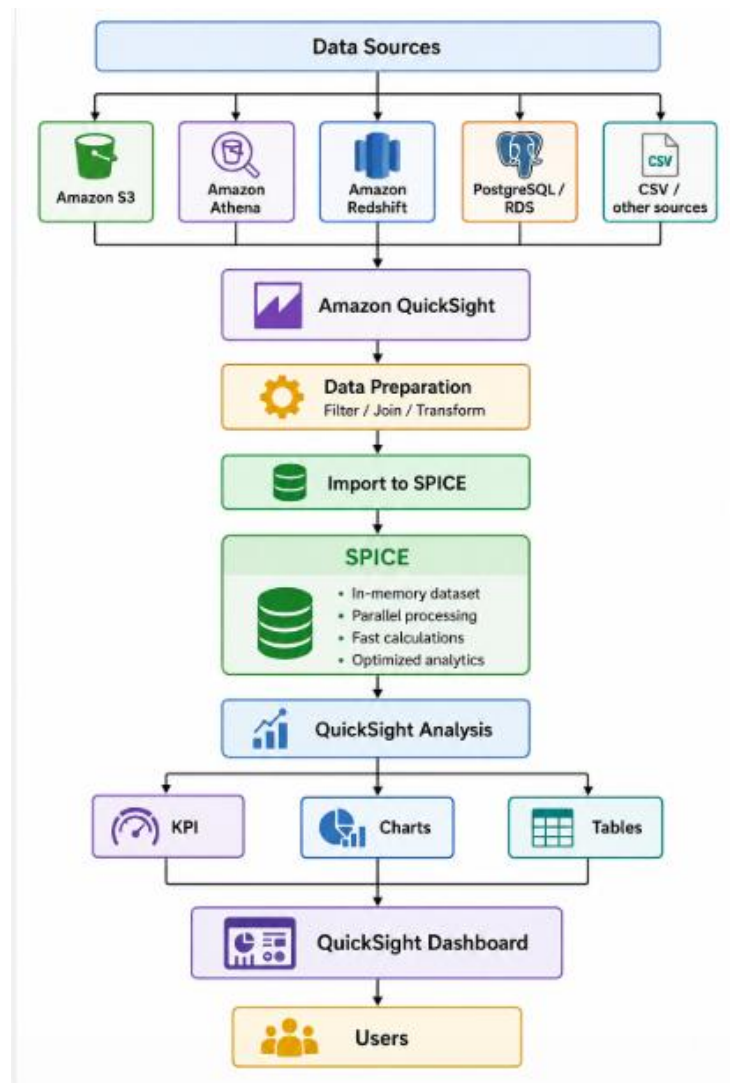
Info:

- Load into table 'connected_cars' completed, 15 record(s) loaded successfully.

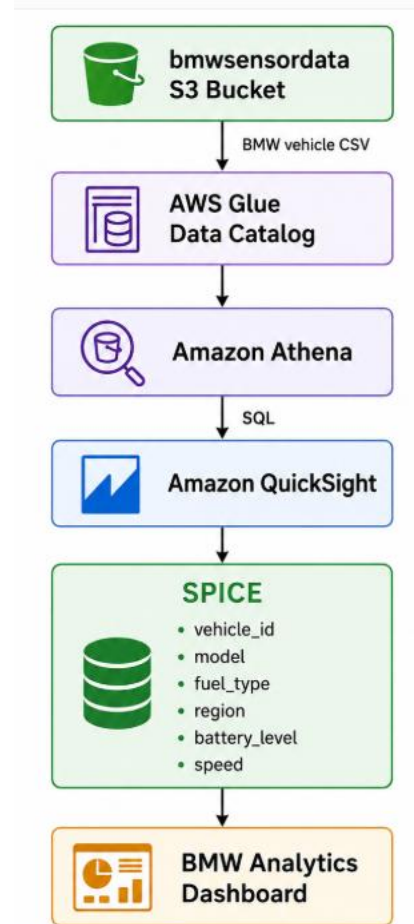


- In **Amazon QuickSight**, **SPICE** stands for **Super-fast, Parallel, In-memory Calculation Engine**.
- It imports a copy of your dataset into QuickSight's optimized in-memory storage.
- Dashboard queries don't have to go back to Athena, Redshift, PostgreSQL, or another source for every visual interaction.

SPICE



SPICE



Spice vs Direct Query

Feature	SPICE	Direct Query
Data location for analytics	QuickSight SPICE	Original source
Dashboard performance	Usually faster	Depends on source
Query source every interaction	No	Yes
Athena repeated query charges	Reduced	Can be higher
Data freshness	Depends on refresh	Current source data
Source DB workload	Lower	Higher
Best for	Dashboards/BI	Near-real-time data

Amazon QuickSight



← → ↺ 🏠

us-east-1.quicksight.aws.amazon.com/sn/account/eswaribala/start/data?tab=dataset

☆ 📁

Amazon Quick | Datasets

eswaribala

New chat

Search

My stuff

Spaces

Research

Chat agents

Apps Preview

Flows

Analyses

Dashboards

Data

My folders

Shared folders

More >

Recents 🔍

No conversations yet

Chat with datasets

Ask questions about your structured data and get AI-powered answers. Datasets are collections of organized business data like sales figures, customer metrics, or operational statistics.

Dismiss

What are the top customers by sales

WSR agent
Response events

Top Sales by Customer

Integrated as: Total sales

Line chart showing sales over time

Bar chart showing sales by customer

Table of sales data

Ask a question about this dashboard...

Datasets

Topics

Data sources

📄 +

Create your first dataset and find it here.

Create dataset

377

Amazon Quicksight



Create data source

Data sources connect Quick to your data from external applications or databases.

Search datasets



Upload a file

.csv, .tsv, .xlsx, .json



Amazon Athena



Amazon Aurora



Amazon OpenSearch Service



Amazon RDS



Amazon Redshift
Auto-discovered



Amazon Redshift



Amazon S3



Amazon S3 Analytics



Amazon S3 Tables
Apache Iceberg tables



Amazon Timestream



Databricks



Exasol



GitHub



Glue Data Catalog

Preview



Google BigQuery



Google Sheets



Impala



Jira



MariaDB



Cancel

Next



New Amazon Athena data source ×

Data source name

AWSDataCatalog

Athena workgroup

[primary]



✓ Validated

SSL is enabled

Create data source

Amazon Quicksight



Choose your table

×

AWSDataCatalog

Catalog: contain sets of databases.

AwsDataCatalog

Database: contain sets of tables.

bmw_database

Tables: contain the data you can visualize.

Search for a table...

Q

☒ bmw_connected_cars

Edit/Preview data

Use custom SQL

Select

Amazon Quick

eswaribala

Search

Q

Manage account

Account

Control top-level settings and preferences

Account settings

Manage assets

Amazon Q

Subscriptions

Plans your account has enabled

Manage subscriptions

SPICE capacity

Index capacity

Identity

Control who can log into this account

Manage users

Manage groups

SSO

Security

Manage Account

Account name: eswaribala



Understand Quick user subscriptions and roles

The new Quick Professional subscription maps to the Reader Pro (Professional) role. The new Quick Enterprise subscription maps to the Author Pro (Enterprise) and Admin Pro (Enterprise) roles. To learn more about the features included in these subscription, [visit the Quick pricing page](#). Manage users [here](#).

Shortcuts



Manage users
Invite your team to build



Manage assets
Ensure users have access to the right resources



Manage subscriptions
Check current plans and additional offers



AWS resources
Allow connections to first-party services



Brand customization
Show off your brand to all users



Custom permissions
Choose the right capabilities for your users

Documentation

See all

User upgrades

Manage

381

Amazon Quicksight



Allow access and autodiscovery for these resources

- ☒  Amazon Redshift
- ☒  Amazon RDS
- ☒  IAM
- ☒  Amazon S3 (3 buckets selected)
[Select S3 buckets](#)
- ☐  Amazon S3 Tables
[Select S3 table buckets](#)
- ☒  Amazon Athena
Make sure you've chosen the right Amazon S3 buckets for Quick access
- ☐  Amazon S3 Storage Analytics
- ☐  Amazon OpenSearch Service
- ☐  Amazon SageMaker
- ☐  Amazon Timestream
- ☐  AWS SecretsManager
[Select secrets](#)

Amazon Quicksight



Amazon Quick Datasets > bmw_connected_cars eswaribala

File Data Help SPICE Save & publish Cancel

TRANSFORM OUTPUT DATASET: Switch back to legacy experience

Steps
Select a step to add a transformation
Search

INPUT
Add data

COLUMN TRANSFORMATIONS
Add calculated columns
Change data type
Rename columns
Select columns

COMBINE TRANSFORMATIONS
Append
Join

OTHER TRANSFORMATIONS
Aggregate
Filter

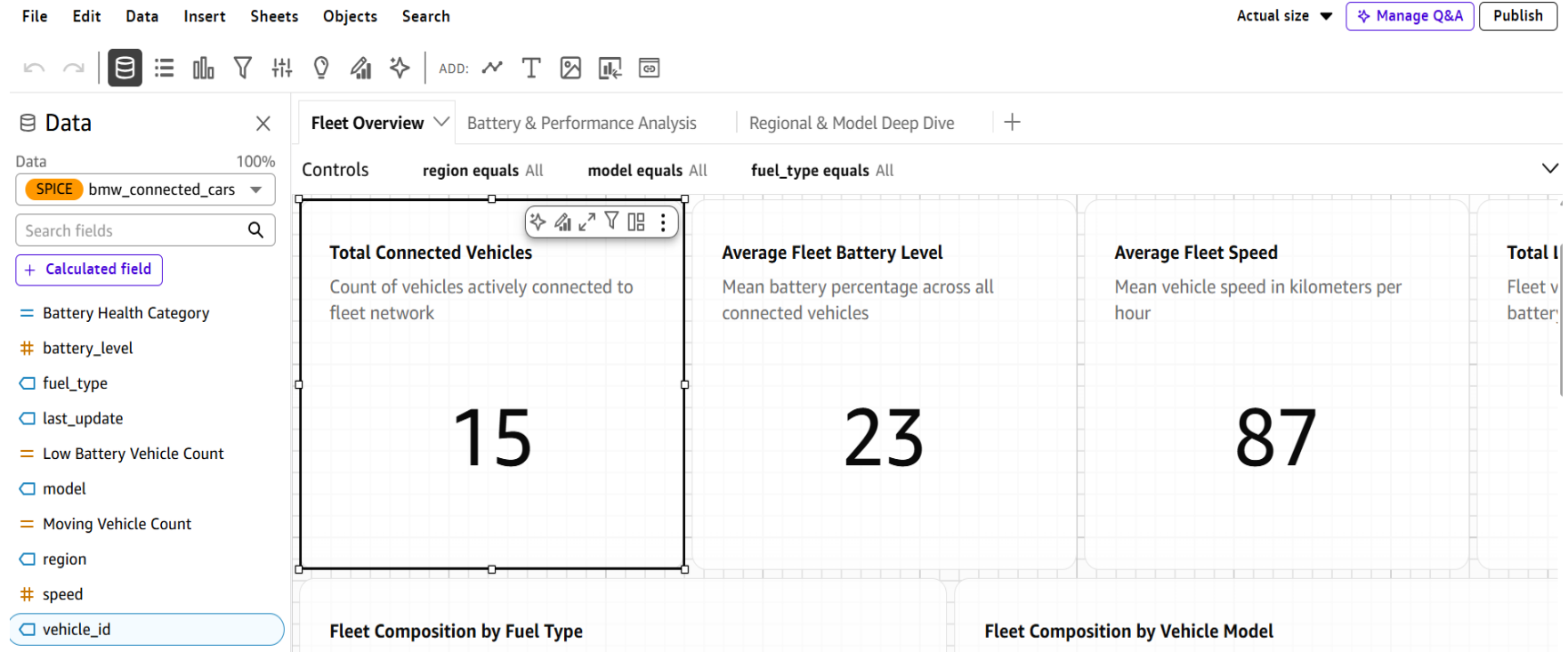
bmw_connected_cars

Configure Preview

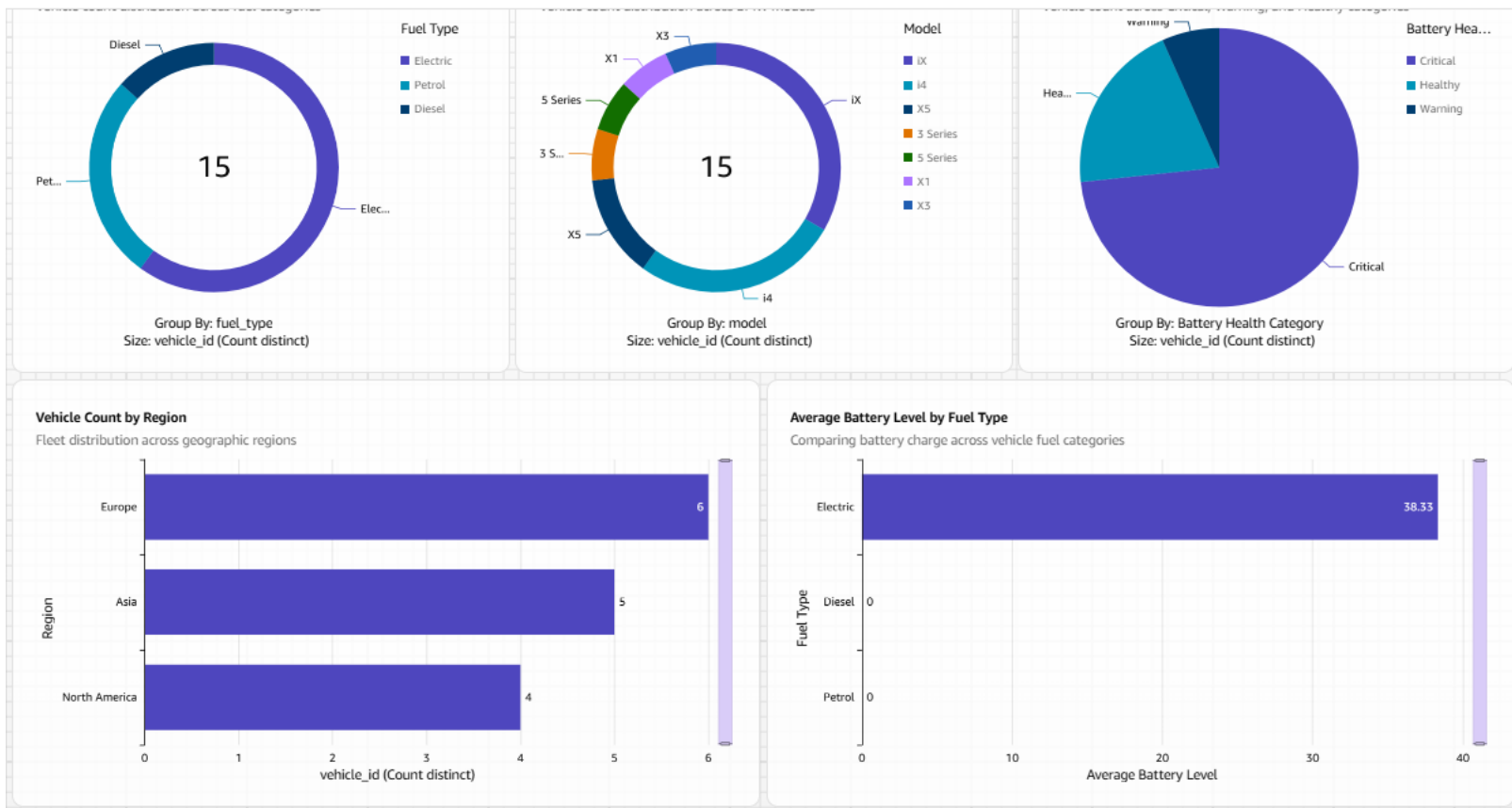
vehicle_id	model	fuel_type	region	battery_level
V001	iX	Electric	Europe	82
V002	X5	Petrol	Europe	0
V003	i4	Electric	Asia	45
V004	3 Series	Diesel	North America	0
V005	iX	Electric	Europe	15

1-15 of 15

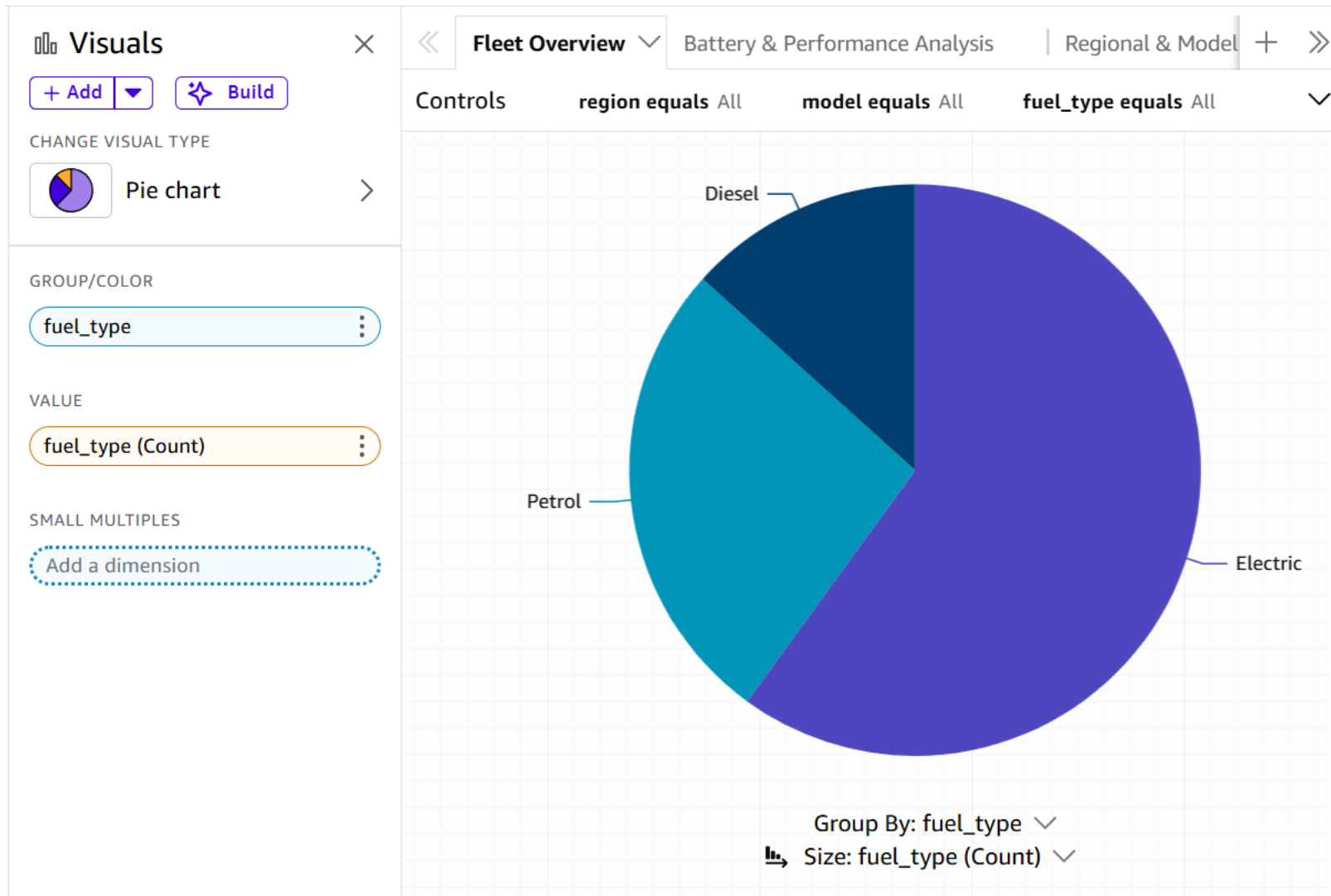
Amazon Quicksight



Amazon Quicksight



Amazon Quicksight



1. SPICE capacity, full refresh, and incremental refresh



 Manage account

Account
Control top-level settings and preferences

- Account settings
- Manage assets
- Amazon Q

Subscriptions
Plans your account has enabled

- Manage subscriptions
- SPICE capacity**
- Index capacity



Purchase SPICE capacity×

Additional SPICE capacity

GB

0.38/GB per month

Cancel

Purchase

SPICE usage (0 bytes)

0 bytes

SPICE join

- CREATE EXTERNAL TABLE bmw_master (
 - model string,
 - segment string,
 - base_price int,
 - plant string,
 - country string
-)
- ROW FORMAT DELIMITED
- FIELDS TERMINATED BY ','
- LOCATION 's3://bmwsensordata/sensor_data/'
- TBLPROPERTIES (
 - 'skip.header.line.count'='1'
-);

SPICE join – create table in athena

- CREATE EXTERNAL TABLE bmw_master (
 - model string,
 - segment string,
 - base_price int,
 - plant string,
 - country string
-)
- ROW FORMAT DELIMITED
- FIELDS TERMINATED BY ','
- LOCATION 's3://bmwsensordata/master/'
- TBLPROPERTIES (
 - 'skip.header.line.count'='1'
-);

SPICE join – create table in athena



us-east-1.quicksight.aws.amazon.com/sn/account/eswaribala/data-sets/d10a4436-91ac-4978-90e0-81013a760c00/prepare

Amazon Quick Datasets > bmw_connected_cars eswaribala

File Data Help SPICE Save & publish

TRANSFORM OUTPUT DATASET:

Switch back to legacy experi

Steps

Select a step to add a transformation

Search

INPUT

- Add data

COLUMN TRANSFORMATIONS

- Add calculated columns
- Change data type
- Rename columns
- Select columns

COMBINE TRANSFORMATIONS

- Append
- Join

OTHER TRANSFORMATIONS

- Aggregate
- Filter

Diagram: A visual representation of a join operation. A box labeled 'bmw_connected_cars' is connected to a box labeled 'Join 1' with an arrow labeled 'LEFT'. Another box labeled 'bmw_master' is connected to the same 'Join 1' box with an arrow labeled 'RIGHT'. The 'Join 1' box has a plus sign on its right side.

Configure Preview Join 1

Columns from both tables with identical names have been auto renamed to resolve conflicts in the join result. [Show auto-resolved columns](#)

Left table: **bmw_connected_cars**

Search columns

- vehicle_id
- model
- fuel type

Join type: Left join

Join keys:

Add a column from left table = Add a column from right table

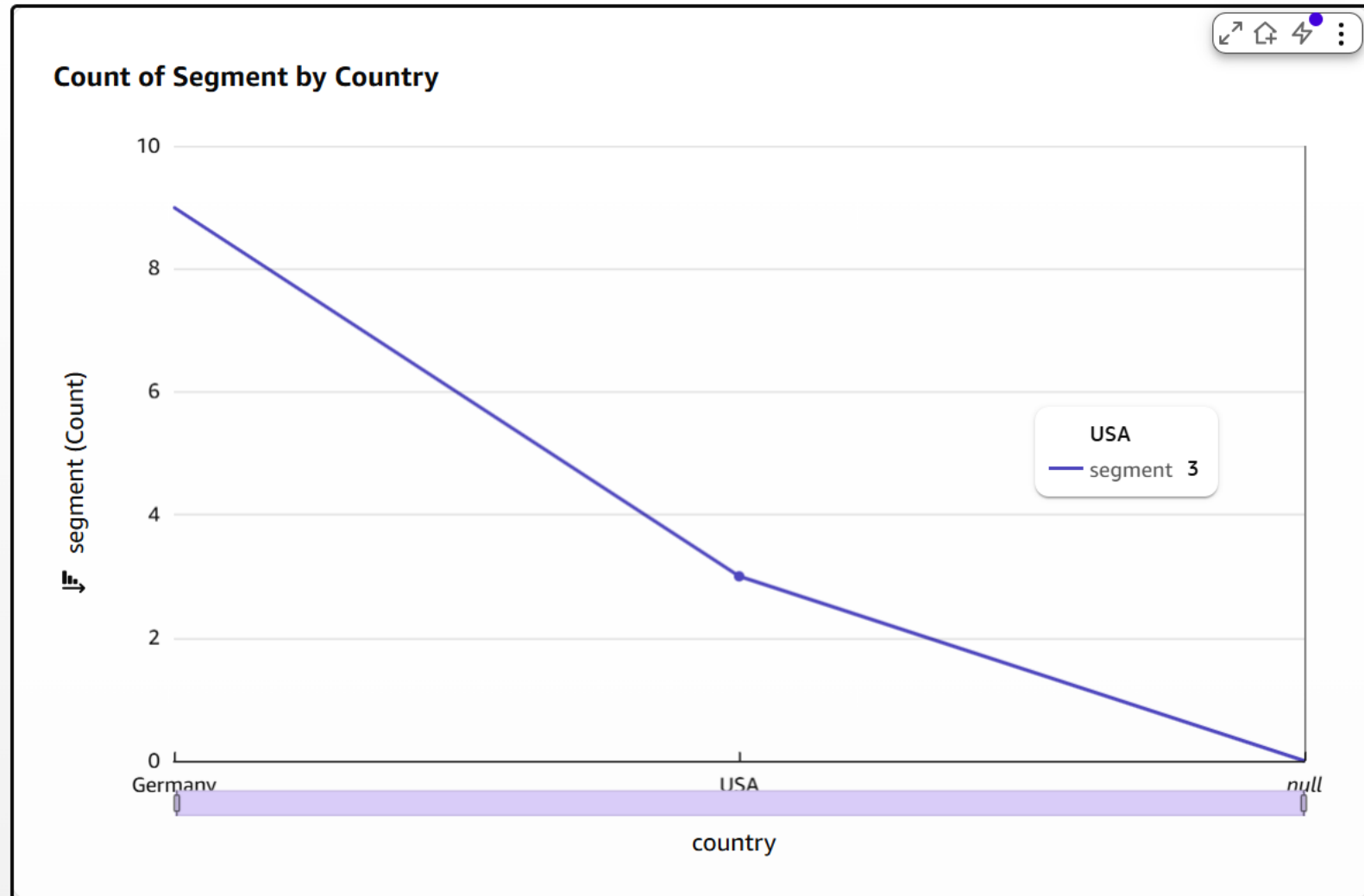
Join keys are missing

Right table: **bmw_master**

Search columns

- model
- segment
- base price

SPICE join – create table in athena



SPICE join – Join, Change Data Type and Rename



Amazon Quick Datasets > **bmw_connected_cars** eswaribala

File Data Help SPICE Save & publish Cancel

TRANSFORM OUTPUT DATASET: Switch back to legacy experience

Steps
Select a step to add a transformation

Search

INPUT

- Add data

COLUMN TRANSFORMATIONS

- Add calculated columns
- Change data type
- Rename columns
- Select columns

COMBINE TRANSFORMATIONS

- Append
- Join

OTHER TRANSFORMATIONS

- Aggregate
- Filter

Workflow:

- bmw_connected_cars** (INPUT) → **Left Join** (COMBINE TRANSFORMATIONS)
- bmw_master** (INPUT) → **Left Join** (COMBINE TRANSFORMATIONS)
- Left Join** → **Base_Price to Decimal Dat...** (COLUMN TRANSFORMATIONS)
- Base_Price to Decimal Dat...** → **Base_Price to Base_Cost** (COLUMN TRANSFORMATIONS)

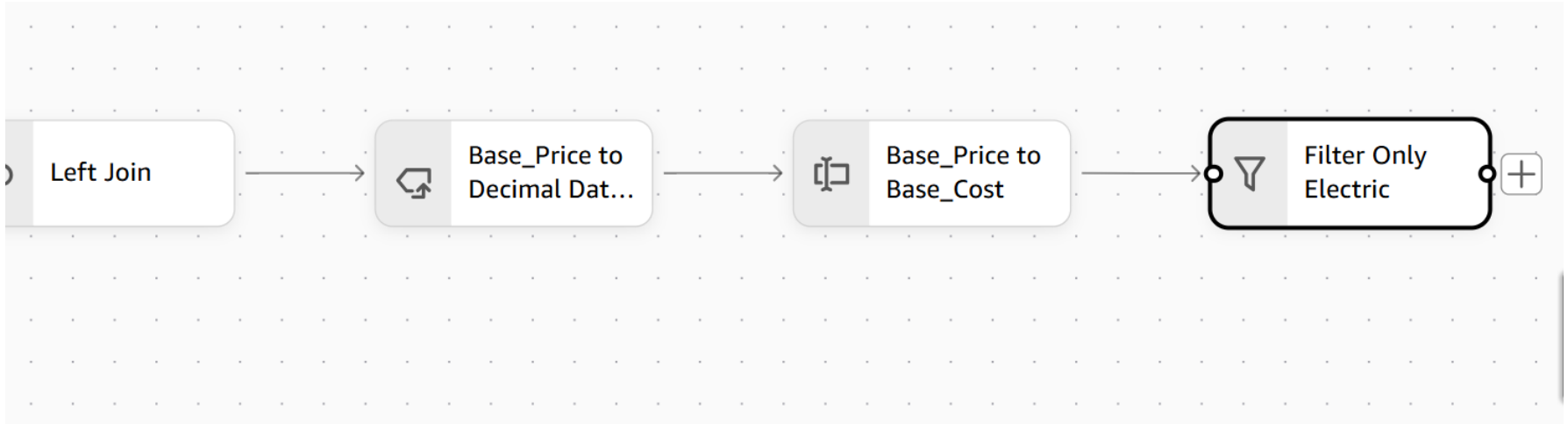
Configure **Preview** **bmw_connected_cars**

Table name
bmw_connected_cars

Table comes from
ATHENA

DATA SOURCE NAME

SPICE join – Join, Change Data Type and Rename



```
graph LR; A[Left Join] --> B[Base_Price to Decimal Dat...]; B --> C[Base_Price to Base_Cost]; C --> D[Filter Only Electric];
```

Configure **Preview**

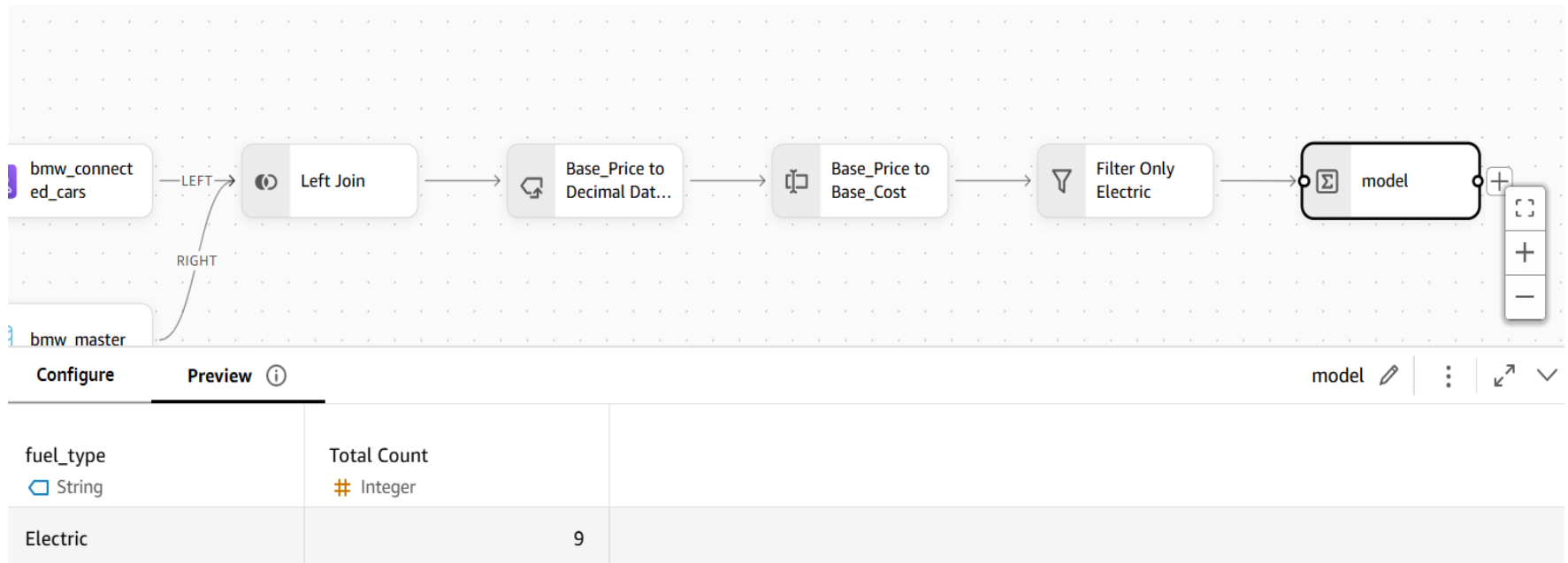
Filter Only Electric   

1 filter condition. All filter conditions use AND logic.

[+ Filter column](#)

Filter condition	Actions
 fuel_type Contains Electric	

SPICE - Aggregation



SPICE – Calculated Field



Add calculated column

Increased_Price 

Cancel

Save

```
1 {base_price} * 1.10
```

Columns



Parameters



Functions



 Only scalar functions are available

Search functions



SPICE – Custom SQL



Data

Choose your table ×

AwsDataCatalog

Catalog: contain sets of databases.

AwsDataCatalog

Database: contain sets of tables.

bmw_database

Tables: contain the data you can visualize.

Search for a table...

☐ bmw_connected_cars

☒ bmw_master

Edit/Preview data Use custom SQL Select

SPICE – Custom SQL



Amazon Quick

Datasets > New custom SQL

eswaribala

File Data Help

SPICE

Save & publish

Close

TRANSFORM OUTPUT

DATASET:

Steps

Select a step to add a transformation

Search

INPUT

Add data

COLUMN TRANSFORMATIONS

Add calculated columns

Change data type

Rename columns

Select columns

COMBINE TRANSFORMATIONS

Append

Join

OTHER TRANSFORMATIONS

Aggregate

Filter

Pivot

Unpivot

New custom SQL

Configure Preview

vehicle_id	model	fuel_type	region	battery_level	speed	last_update
V001	iX	Electric	Europe	82	110	2024-07-01
V002	X5	Petrol	Europe	0	95	2024-07-01
V003	i4	Electric	Asia	45	130	2024-07-01
V004	3 Series	Diesel	North America	0	80	2024-07-01
V005	iX	Electric	Europe	15	60	2024-07-01
V006	X3	Petrol	Asia	0	105	2024-07-01
V007	i4	Electric	North America	90	140	2024-07-01
V008	5 Series	Diesel	Europe	0	70	2024-07-01



- AWS Glue is a fully managed, serverless data integration service from AWS.
- It helps you discover, prepare, move, and integrate data from many sources for analytics, machine learning, and application development — without managing any servers or infrastructure.
- It supports batch ETL, streaming ETL, and ELT workloads, and integrates tightly with Amazon S3 data lakes, Amazon Athena, Amazon Redshift, Amazon EMR, and other AWS analytics services.



AWS GLUE

Fully managed serverless data integration service

AWS Glue makes it easy to discover, prepare, move and integrate data for analytics, machine learning and application development.

1. WHAT IS AWS GLUE?

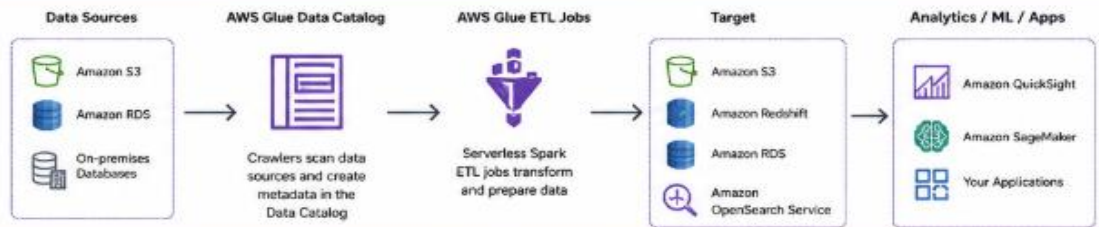
AWS Glue is a serverless data integration service that helps you:

- ✓ Discover data with a Data Catalog
- ✓ Prepare and transform data with ETL
- ✓ Move data between data stores
- ✓ Orchestrate and schedule ETL jobs
- ✓ Monitor and manage data pipelines



AWS Glue

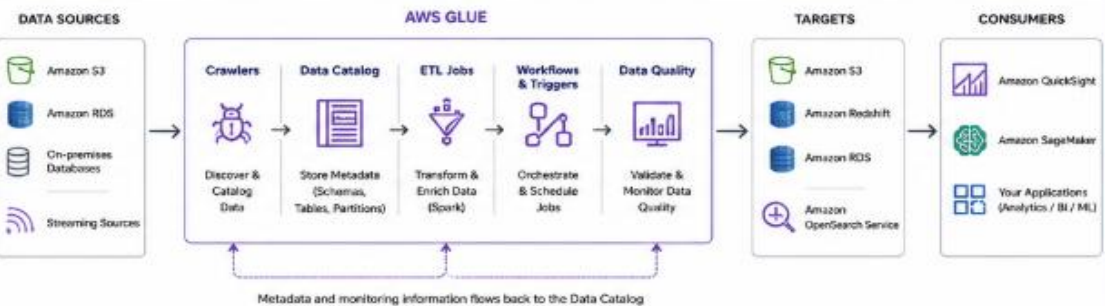
2. HOW AWS GLUE WORKS



3. KEY COMPONENTS

-  **Data Catalog**
Central metadata repository that stores information about your data assets.
-  **Crawlers**
Automatically scan data sources, discover schemas, and update the Data Catalog.
-  **ETL (Extract, Transform, Load) Jobs**
Serverless Spark jobs to clean, transform and load data at scale.
-  **Workflows & Triggers**
Orchestrate jobs and define dependencies. Schedule or run jobs based on events.
-  **Data Quality**
Define rules and monitor data quality to ensure trusted data.

4. TYPICAL ARCHITECTURE



5. BENEFITS

-  **Serverless**
No infrastructure to provision or manage.
-  **Scalable**
Automatically scales to process petabytes of data.
-  **Cost-effective**
Pay only for the resources you use.
-  **Faster Time to Value**
Built-in connectors, catalogs and ETL simplify data prep.
-  **Secure & Compliant**
Leverage AWS security, IAM, encryption and auditing.



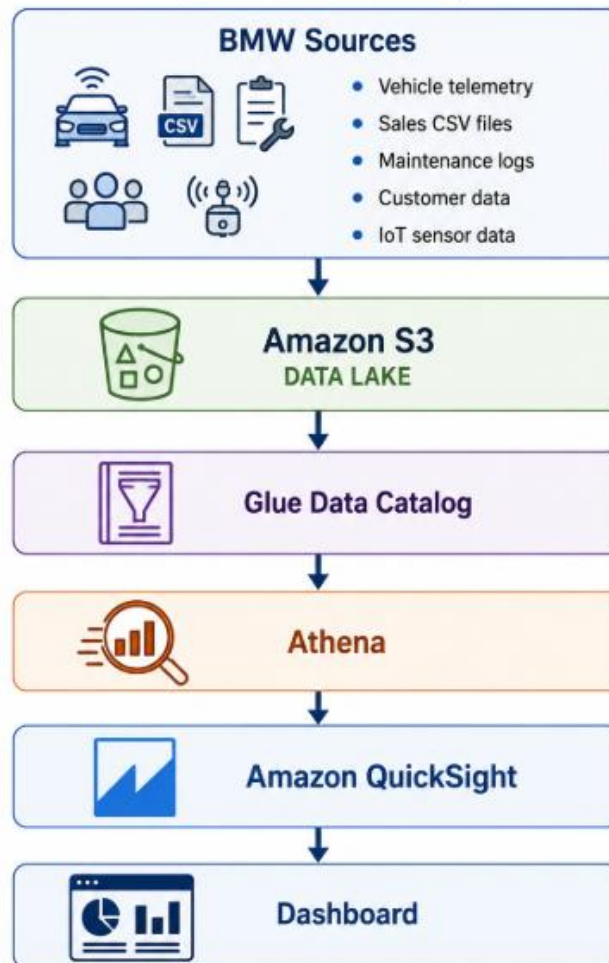
Data Lake

- A **data lake** is a centralized repository that stores large volumes of **raw, semi-structured, and structured data** in its original form until you need to analyze it.

Data Lake



BMW Data Lake to Dashboard Architecture





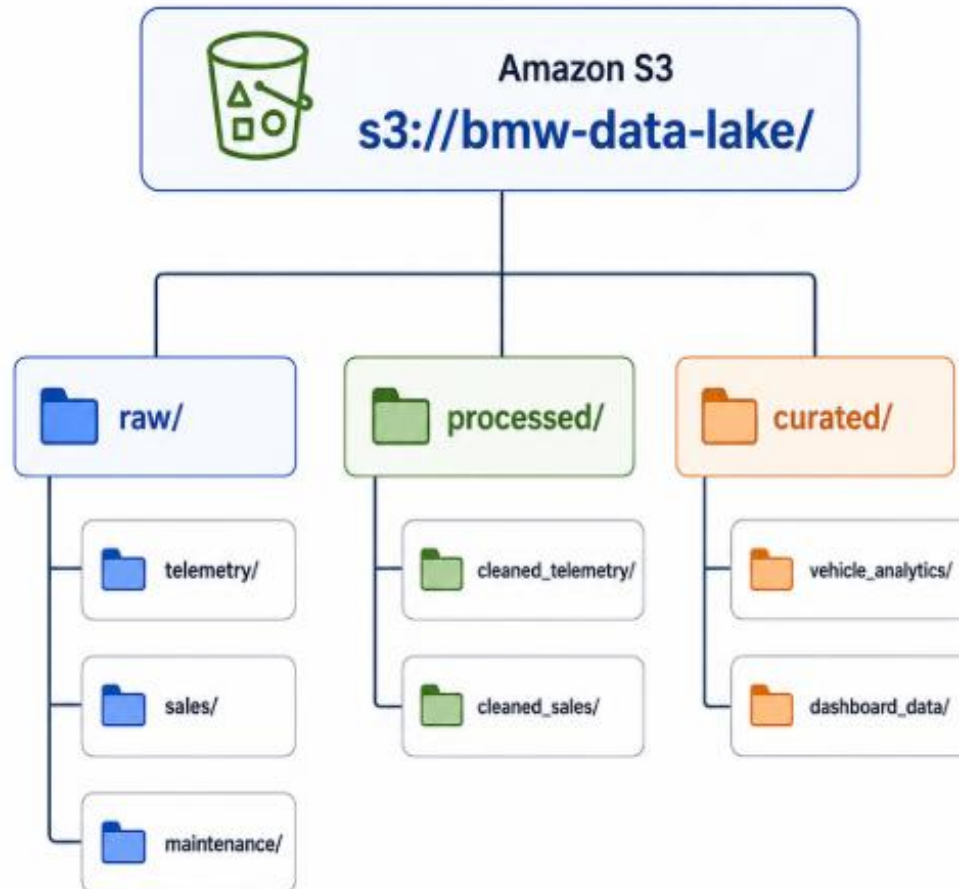
Data Lake

- In AWS, **Amazon S3 is commonly used as the storage layer for a data lake** because it can hold CSV, JSON, Parquet, images, logs, and other file types at very large scale.

Data Lake



BMW Data Lake S3 Folder Structure



Data Lake



Zone	Purpose
Raw	Original source data, unchanged
Processed	Cleaned, standardized, transformed data
Curated	Business-ready analytical datasets

Traditional Data Lake vs Transactional Lake Format

Capability	Traditional Data Lake	Transactional Lakehouse
Storage	S3 / ADLS / GCS / HDFS	Same object storage
File formats	CSV, JSON, Parquet, ORC	Usually Parquet + table metadata
Transactions	Limited / none	ACID transactions
UPDATE	Difficult; rewrite files	Supported
DELETE	Difficult	Supported
MERGE / UPSERT	Custom ETL required	Native or table-format supported
Concurrent writes	Risk of conflicts	Transaction management
Schema enforcement	Usually weak	Stronger enforcement

Traditional Data Lake vs Transactional Lake Format

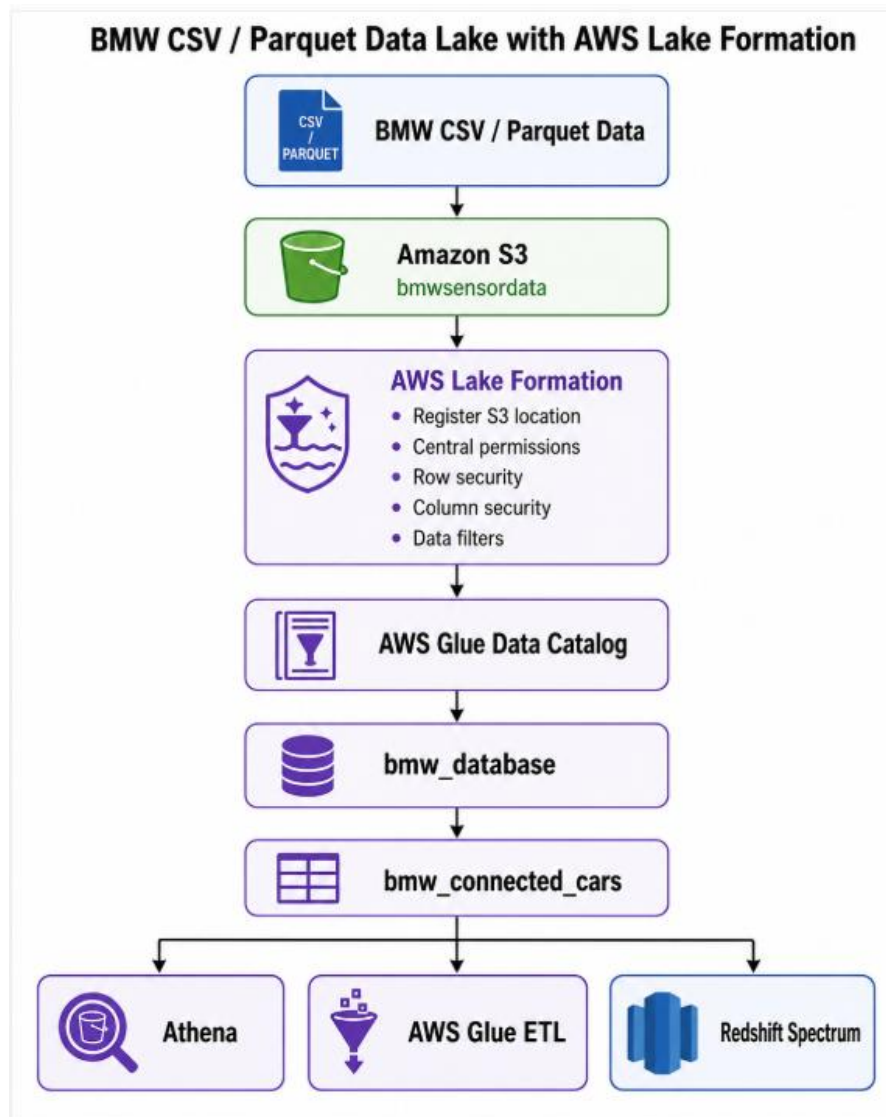
Schema evolution	Manual	Managed
Metadata	Hive/Glue-style catalog	Catalog + transactional table metadata
Time travel	Normally unavailable	Common capability
Data versioning	Manual	Snapshot/version based
Streaming + batch	Separate pipelines often needed	Unified table model
Data quality	Pipeline responsibility	Better table-level guarantees
Examples	S3 + Parquet + Glue + Athena	Delta Lake, Apache Iceberg, Apache Hudi



Delta Lake vs Apache Iceberg vs Apache Hudi

Feature	Delta Lake	Apache Iceberg	Apache Hudi
ACID	Yes	Yes	Yes
Schema evolution	Yes	Yes	Yes
Time travel	Yes	Yes	Yes
UPDATE	Yes	Yes, engine dependent	Yes
DELETE	Yes	Yes	Yes
MERGE	Yes	Yes, engine dependent	Yes
Streaming support	Strong	Strong ecosystem support	Very strong
CDC / upserts	Strong	Supported through engines	Major strength
Hidden/advanced partition evolution	Some platform-dependent features	Major strength	Supported approaches
Metadata management	Transaction log	Snapshot/manifest metadata	Timeline + metadata
Common association	Databricks/Spark ecosystem	Multi-engine/open ecosystem	Incremental/CDC workloads

Target architecture



Snowflake



- Snowflake is a **cloud-native data platform** mainly used for data warehousing, analytics, data sharing, and increasingly lakehouse-style workloads.



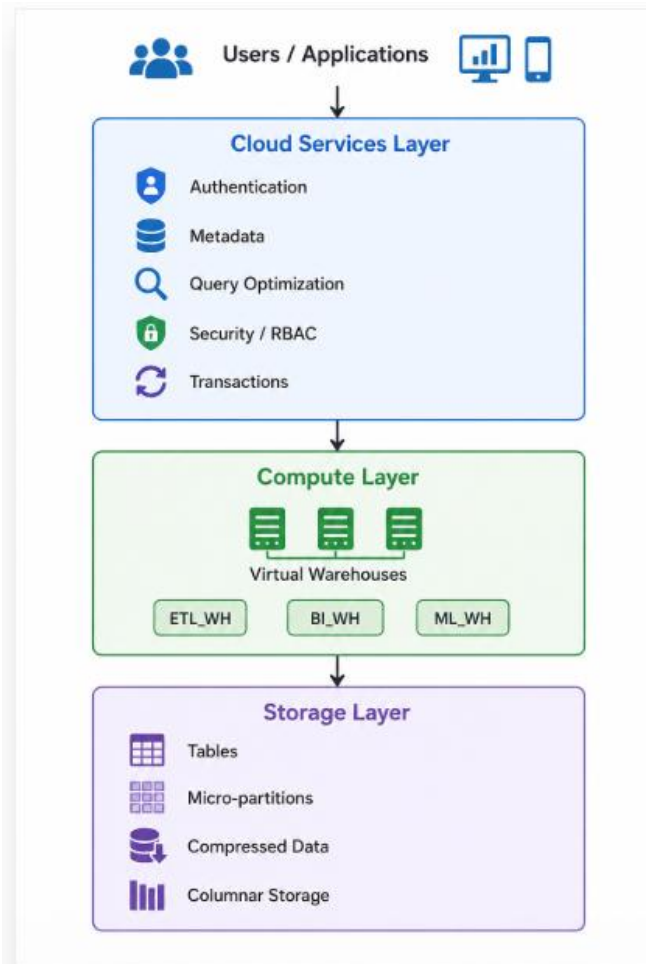
Data lake vs Snow Flake

AWS Data Lake	Snowflake
S3 stores data	Snowflake manages storage
Glue discovers schema	Snowflake has native catalog/schema
Lake Formation handles governance	Snowflake has native RBAC/governance
Athena executes SQL	Virtual Warehouses execute SQL
Several AWS services work together	Much is integrated in one platform
Excellent for raw object storage	Excellent for structured analytics
Strong AWS ecosystem integration	Multi-cloud data platform

Understand Snowflake's cloud-native architecture



- 1. High-level Snowflake architecture



Understand Snowflake's cloud-native architecture



- 2. Storage Layer
- When you load data into Snowflake, Snowflake manages the physical storage for you.
- For example:
- BMW_DATABASE
 - |
 - |— SALES
 - |— VEHICLES
 - |— TELEMETRY
 - |— MAINTENANCE
- Internally Snowflake stores this data using optimized **micro-partitions**.
- SALES TABLE
 - |
 - |— Micro-partition 1
 - |— Micro-partition 2
 - |— Micro-partition 3
 - |— Micro-partition 4

Understand Snowflake's cloud-native architecture

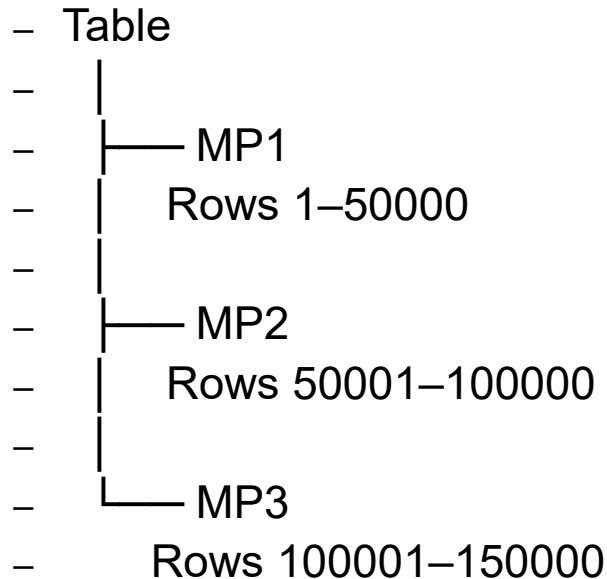


- You don't manually create or manage these partitions.
- Snowflake automatically handles:
 - Compression
 - Partitioning
 - Metadata
 - File organization
 - Encryption
 - Storage optimization
- **Micro-partitions**
- A table might logically contain:
- `sale_id | model | city | price`
- -----
- `1001 | X5 | Mumbai | 8500000`
- `1002 | iX | Delhi | 9500000`
- `1003 | X3 | Chennai | 6800000`
- ...

Understand Snowflake's cloud-native architecture



- Snowflake internally divides the data into smaller storage units:

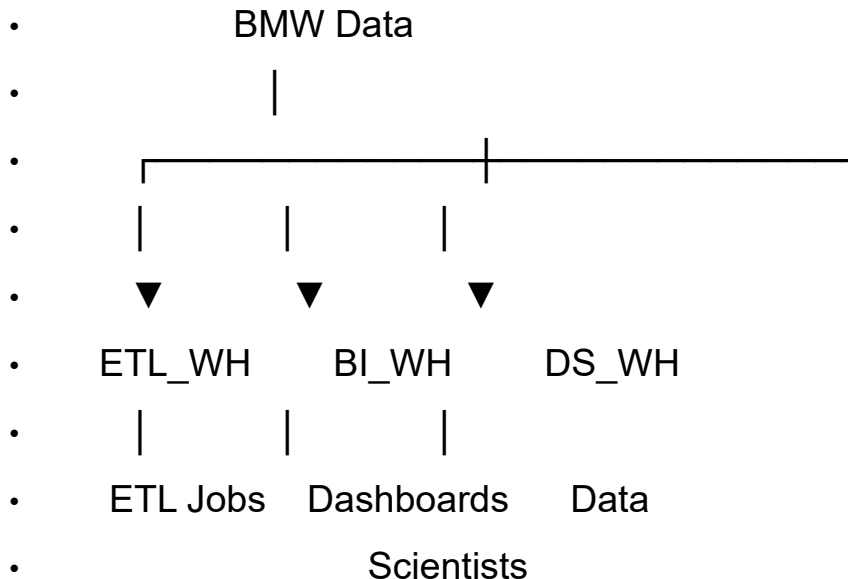


- Metadata might contain information such as:
- MP1:
 - city min = Bangalore
 - city max = Mumbai
 - price min = 4,000,000
 - price max = 9,000,000

Understand Snowflake's cloud-native architecture



- 3. Compute Layer
- Snowflake compute is provided by:
- **Virtual Warehouses**
- A Virtual Warehouse is essentially a Snowflake-managed compute cluster.
- For example:



- All warehouses can access the **same underlying data**.
- This is one of Snowflake's major architectural advantages.



- 3. Compute Layer

- For example:

- SALES TABLE



- You do not need three copies of the table.

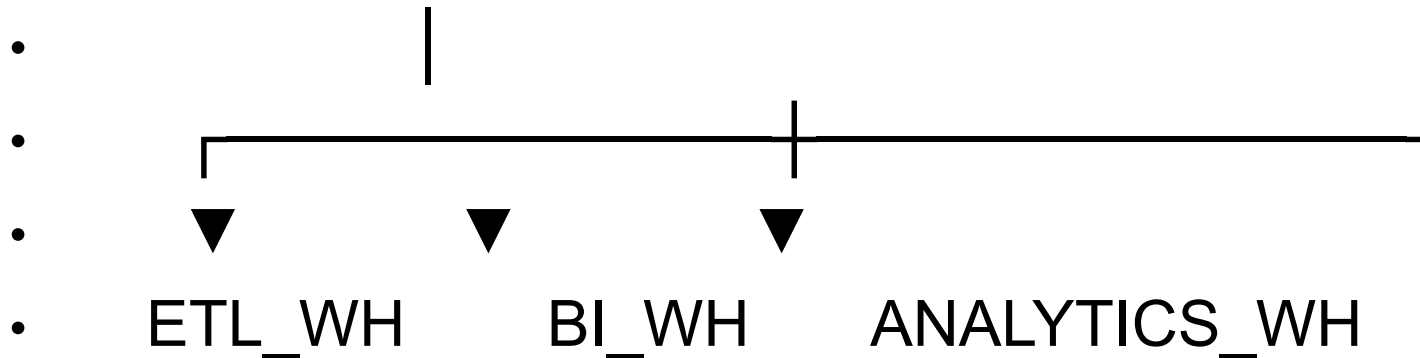
Understand Snowflake's cloud-native architecture



- A heavy ETL process may slow down dashboard queries.

- In Snowflake:

- Shared Data



- Each workload receives independent compute.
- So:
- Heavy ETL
- doesn't necessarily slow:
- BI dashboard queries



- **5. Virtual Warehouse sizes**
- Warehouses can be resized according to workload.
- Conceptually:
 - X-Small
 - Small
 - Medium
 - Large
 - X-Large
 - 2X-Large
- ...



- **5. Virtual Warehouse sizes**
- For example:
 - Development → X-Small
 - BI queries → Small
 - Large ETL → Medium/Large
 - Heavy analytics → Large+
- Example:
- `CREATE WAREHOUSE bmw_etl_wh`
- `WAREHOUSE_SIZE = 'SMALL'`
- `AUTO_SUSPEND = 300`
- `AUTO_RESUME = TRUE;`
- Use it:
- `USE WAREHOUSE bmw_etl_wh;`

Understand Snowflake's cloud-native architecture



- **6. Auto Suspend and Auto Resume**

- One cloud-native benefit is that compute doesn't need to run continuously.
- Example:
- 9:00 AM
- User executes query
- ↓
- Warehouse starts
- ↓
- Query executes
- ↓
- No activity
- ↓
- 5 minutes
- ↓
- Warehouse suspends



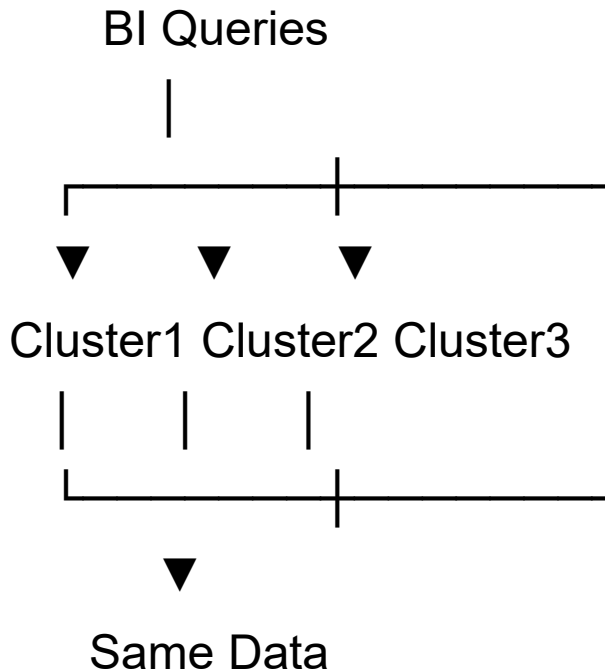
- **6. Auto Suspend and Auto Resume**
- With:
- `AUTO_SUSPEND = 300`
- `AUTO_RESUME = TRUE;`
- Snowflake can resume the warehouse automatically when another query arrives.
- This helps control compute cost.



- **7. Multi-cluster Warehouses**

- Imagine 1,000 dashboard users suddenly execute queries.
- Instead of only making one machine larger, Snowflake can support **multi-cluster warehouses** for concurrency.

- Conceptually:



- This is primarily useful for handling many simultaneous queries.



- **8. Cloud Services Layer**
- The third major layer is the **Cloud Services Layer**.
- Think of this as Snowflake's control plane.
- It handles things such as:
 - Authentication
 - Authorization
 - Metadata management
 - SQL parsing
 - Query optimization
 - Transaction management
 - Infrastructure coordination
 - Security
 - Access control

Understand Snowflake's cloud-native architecture



- For example, when you run:
 - `SELECT model, AVG(price)`
 - `FROM vehicle_sales`
 - `GROUP BY model;`

Understand Snowflake's cloud-native architecture



- the flow is approximately:

SQL



Cloud Services



Parse SQL



Check user permissions



Read metadata



Optimize query



Select relevant micro-partitions



Send work to Virtual Warehouse



Warehouse executes query



Result returned



- **9. Complete query flow**
- Consider:
- `SELECT model, AVG(price)`
- `FROM vehicle_sales`
- `WHERE city = 'Mumbai'`
- `GROUP BY model;`

Understand Snowflake's cloud-native architecture



- Snowflake processes it approximately like this:

1. User submits SQL



2. Cloud Services Layer

- authentication
- authorization
- parsing
- optimization



3. Metadata checked



4. Unnecessary micro-partitions pruned



5. Virtual Warehouse processes data



6. Storage layer supplies required blocks



7. Results calculated



8. Result returned



- **10. Caching**

- Snowflake uses caching to improve performance.
- Three concepts are especially useful to understand.

- **Result Cache**

- Suppose you execute:
- `SELECT COUNT(*)`
- `FROM vehicle_sales;`
- and then run the same query again under suitable conditions.

Understand Snowflake's cloud-native architecture



- Snowflake may reuse the previously computed result:

First Query



Warehouse



Compute result



Result cache

Second identical query



Result Cache



Result

- The warehouse might not need to recompute everything.



- **Warehouse Cache**

- When a warehouse reads data, some data can remain cached locally on the compute nodes.

Storage



Warehouse



Local cache

- Repeated access may therefore be faster while the warehouse remains active.

- **Metadata Cache**

- Snowflake's services maintain metadata describing storage, including micro-partition characteristics.
 - This helps Snowflake decide:
 - Which partitions should be read?
 - Which partitions can be ignored?



- **11. Shared Data, Independent Compute**

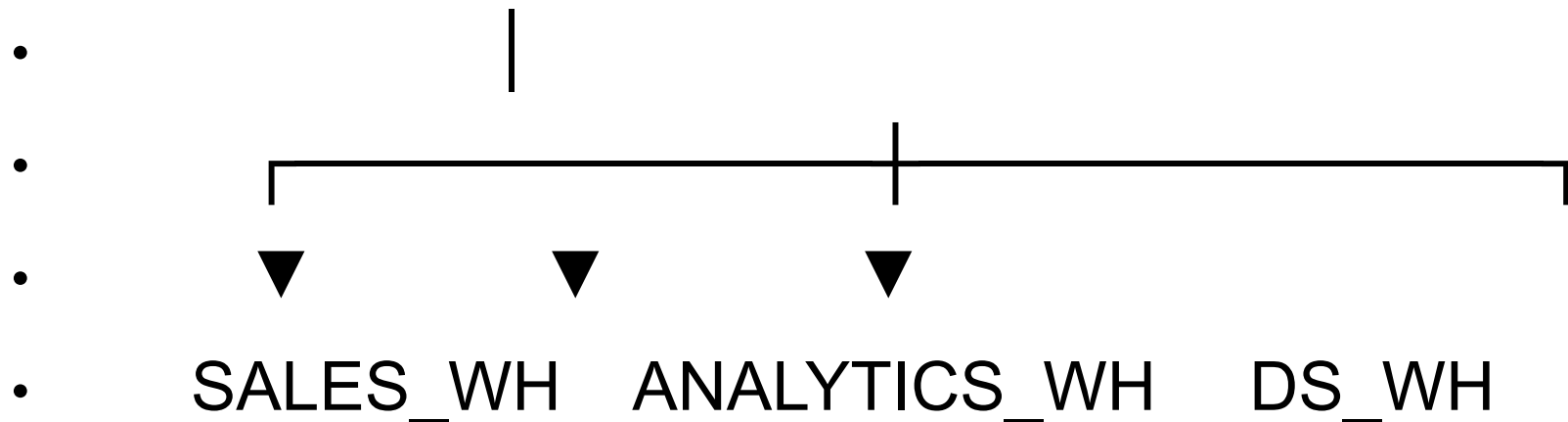
- This is probably the single most important concept.
- Suppose BMW has:
- BMW_DATABASE
 - |
 - |—— SALES
 - |—— VEHICLES
 - |—— TELEMETRY
- Three departments need the same data.
- Instead of:
- Sales copy
- Analytics copy
- Data Science copy

Understand Snowflake's cloud-native architecture



- Snowflake can use:

- BMW_DATABASE



- One storage layer.
- Multiple independent compute layers.



- **12. Snowflake is cloud-native**
- Snowflake wasn't designed as a traditional on-premises database that was simply moved into a VM.
- Its architecture takes advantage of cloud capabilities such as:
 - Elastic compute
 - Object-style storage
 - Distributed processing
 - Automatic scaling
 - Managed infrastructure
 - Pay-for-use compute
 - High availability
- Snowflake is available across major clouds including:
 - AWS
 - Microsoft Azure
 - Google Cloud
- But users work mainly through Snowflake rather than managing the underlying cloud infrastructure directly.



- **13. What Snowflake manages for you**
- Traditional systems may require database administrators to manage:
 - Servers
 - Storage allocation
 - Indexes
 - Partition maintenance
 - Database installation
 - Patching
 - Cluster infrastructure
- Snowflake abstracts much of this.
- You primarily work with:
 - Databases
 - Schemas
 - Tables
 - Views
 - Roles
 - Warehouses
 - SQL



- **14. Logical object hierarchy**
- Snowflake's logical organization is:

Organization



Account



Database



Schema



Table / View

Understand Snowflake's cloud-native architecture

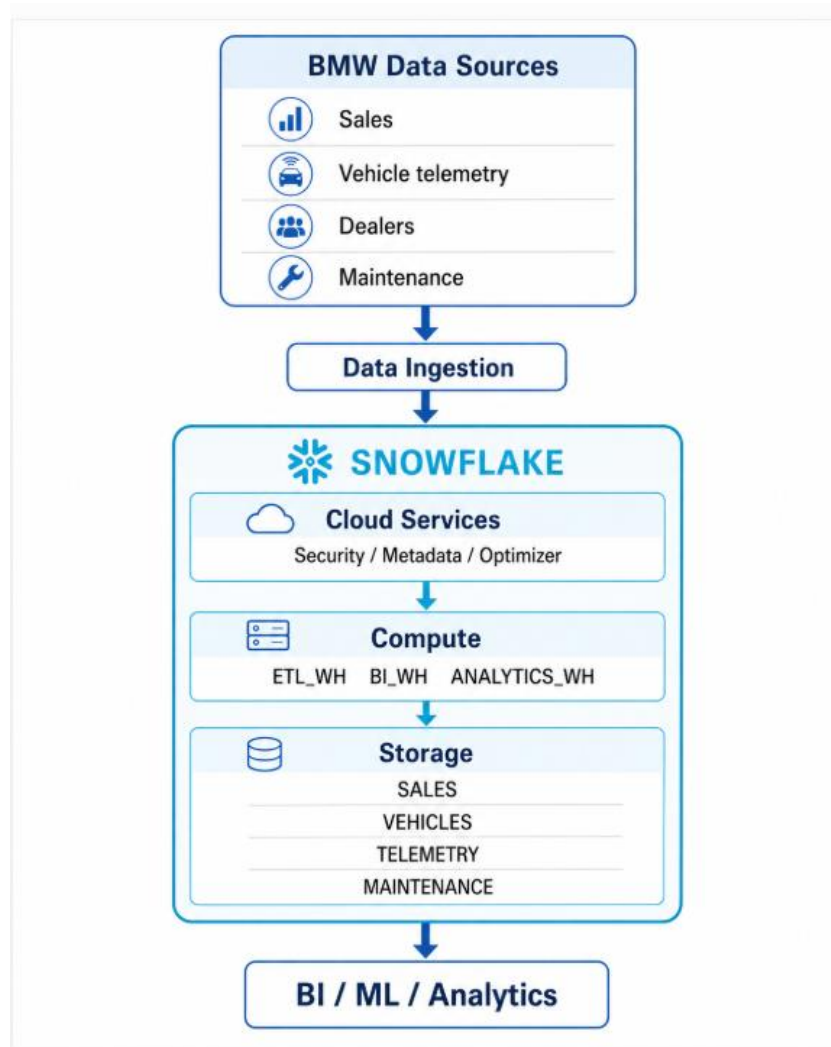


- Example:
- BMW_DATABASE
 - |
 - |— RAW
 - | |— SALES
 - | |— TELEMETRY
 - |
 - |— CLEAN
 - | |— SALES
 - | |— TELEMETRY
 - |
 - |— ANALYTICS
 - | |— SALES_SUMMARY
 - | |— VEHICLE_PERFORMANCE
- A full table name might therefore be:
- BMW_DATABASE.RAW.SALES



- **15. Storage vs database vs warehouse**
 - This terminology often causes confusion.
- **Database**
 - A logical container:
 - BMW_DATABASE
 - Contains schemas and tables.
- **Warehouse**
- Compute resources:
 - BMW_WH
 - Executes SQL.
- **Storage**
 - Where Snowflake physically stores table data.
- Therefore:
 - Database $\neq$ Warehouse

Simple BMW architecture

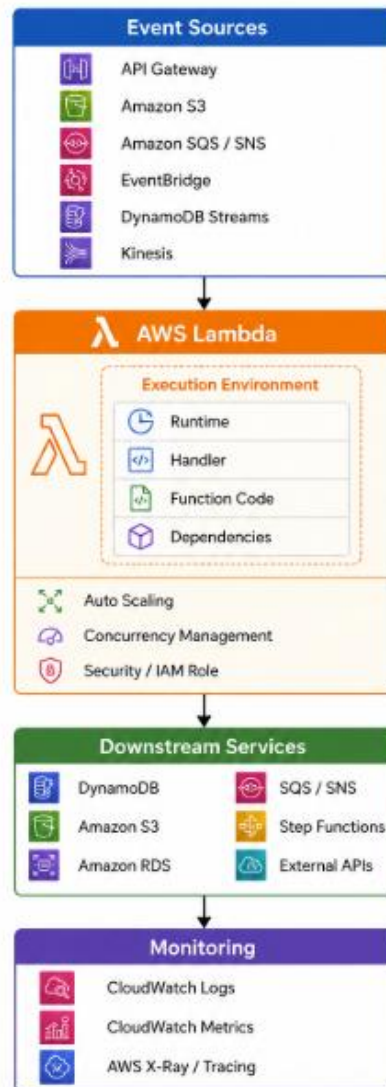




Lambda

- Serverless compute service from AWS
- Run code without managing servers
- Automatically scales based on requests/events
- Pay only for execution time and resources used
- Supports event-driven applications

Lambda Architecture





Why AWS Lambda?

- No server provisioning
- Automatic scaling
- High availability
- Reduced infrastructure management
- Pay-per-use pricing
- Fast development and deployment



How Lambda Works

- **Event Source → Lambda Function → Processing → Target Service**
- Examples:
 - S3 upload → Lambda → Process file
 - API Gateway → Lambda → Database
 - SQS → Lambda → Process messages
 - EventBridge → Lambda → Scheduled task



Supported Languages

- Python
- Java
- Node.js
- .NET
- Ruby
- Custom runtimes
- Container-image-based Lambda functions



Lambda Function Components

- Function code
- Runtime
- Handler
- IAM execution role
- Environment variables
- Memory configuration
- Timeout
- Trigger
- Destination / downstream service



Lambda Handler Example

```
def lambda_handler(event, context):  
    print(event)  
  
    return {  
        "statusCode": 200,  
        "body": "Hello from AWS Lambda"  
    }
```



Lambda Triggers

- Common triggers:
 - Amazon API Gateway
 - Amazon S3
 - Amazon SQS
 - Amazon SNS
 - Amazon EventBridge
 - DynamoDB Streams
 - Kinesis
 - Application Load Balancer



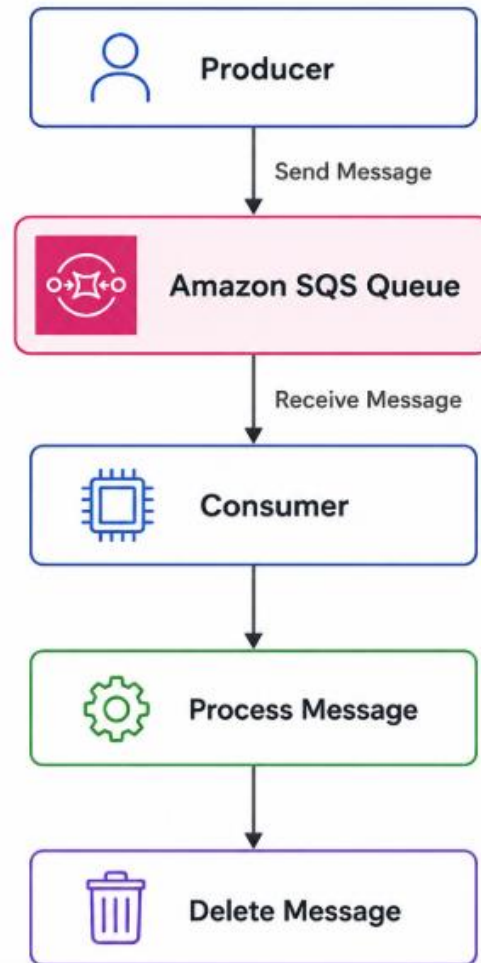
Lambda Execution Role

- Lambda uses an **IAM Role** to access AWS services.
- Example permissions:
 - Read S3 objects
 - Send messages to SQS
 - Access DynamoDB
 - Write CloudWatch logs



- **What is Amazon SQS?**
 - Fully managed **message queue service**
 - Used to **decouple applications and microservices**
 - Supports asynchronous communication
 - Highly scalable and durable
 - Helps applications handle workload spikes

AWS SQS





Why use SQS?

- Producer and consumer do not need to run at the same time
- Prevents direct dependency between services
- Handles sudden traffic spikes
- Supports retry mechanisms
- Improves fault tolerance
- Easy integration with AWS Lambda

Types of SQS Queues

- **1. Standard Queue**
 - Very high throughput
 - At-least-once delivery
 - Best-effort ordering
 - Suitable for most applications
 - $A \rightarrow B \rightarrow C \rightarrow D$
 - Messages may occasionally be delivered more than once or out of order.

Types of SQS Queues

- **2. FIFO Queue**
- **FIFO = First In, First Out**
- Maintains message order
- Supports exactly-once processing semantics
- Useful when message sequence matters

Message 1



Message 2



Message 3

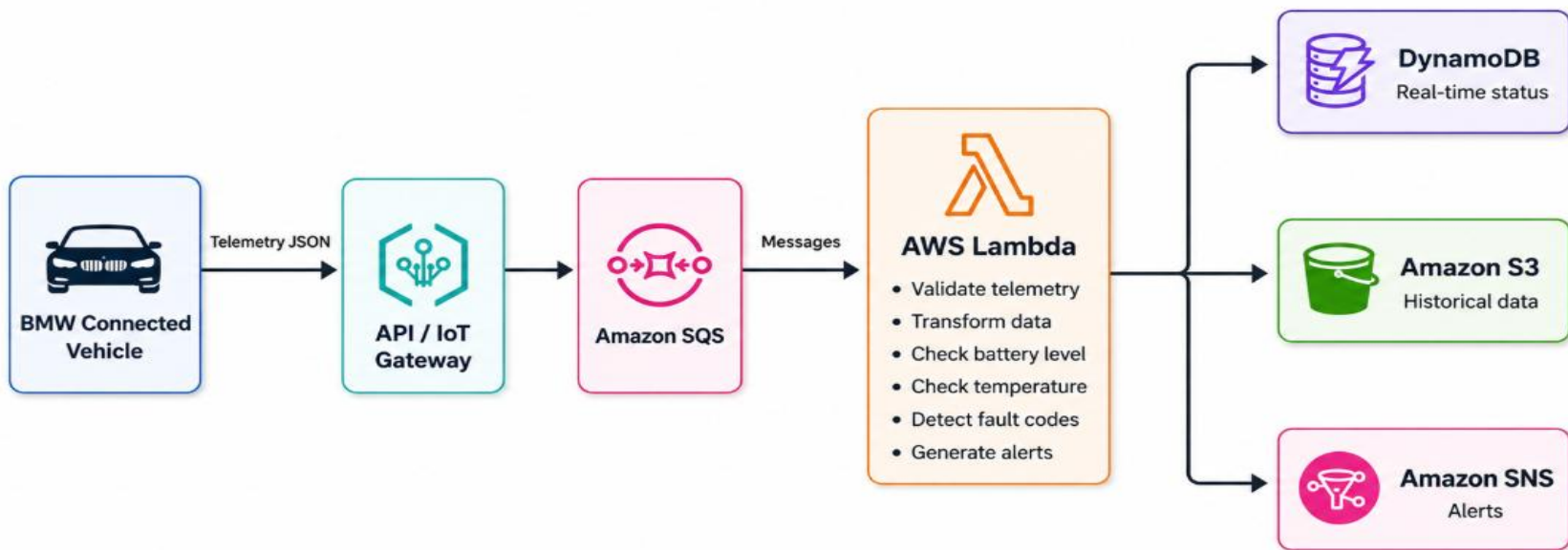
- Common use cases:
- Banking transactions
- Order processing
- Inventory updates



BMW Use Case – AWS Lambda + SQS

- A good BMW example is **connected vehicle telemetry processing**.
- **Business Scenario**
- BMW vehicles continuously send telemetry such as:
 - Vehicle ID
 - Speed
 - Battery level
 - Engine temperature
 - Fuel level
 - GPS location
 - Fault codes
 - Timestamp

BMW Use Case – AWS Lambda + SQS



Lambda Function



CodeTestMonitorConfigurationAliasesVersions

Code sourceInfo

Open in Visual Studio Code ↗▼Update ▼

← → telemetrypublisher

lambda_function.py ×

lambda_function.py

```
1 import json
2
3 def lambda_handler(event, context):
4     # TODO implement
5     return {
6         'statusCode': 200,
7         'body': json.dumps('Hello from Lambda!')
8     }
9
```

Amazon Q Tip 1/3: Start typing to get suggestions ([ESC] to exit)

EXPLORER

TELEMETRYPUBLISHER

lambda_function.py

DEPLOY ✓ Current

Deploy (Ctrl+Shift+U)

Test (Ctrl+Shift+I)

Lambda Function



Configuration

General configuration

Triggers

Permissions

Destinations

Function URL

Environment variables

Tags

VPC

Execution role

Role name

[telemetrypublisher-role-hmy9wsg7](#)



[View role document](#)

[Edit](#)

Resource summary

To view the resources and actions that your function has permission to access, choose a service.



Amazon CloudWatch Logs

3 actions 2 resources



By action

By resource

Resource

Actions

arn:aws:logs:us-east-1:622997067231:*

Allow: logs:CreateLogGroup

Lambda Function



Attach policy to telemetrypublisher-role-hmy9wsg7

► Current permissions policies (1)

Other permissions policies (1/1226)



Q AWSLambdaSQSQueueExecutionRole X

Filter by Type

All types ▼

1 match

< 1 >



Policy name



Type



Description

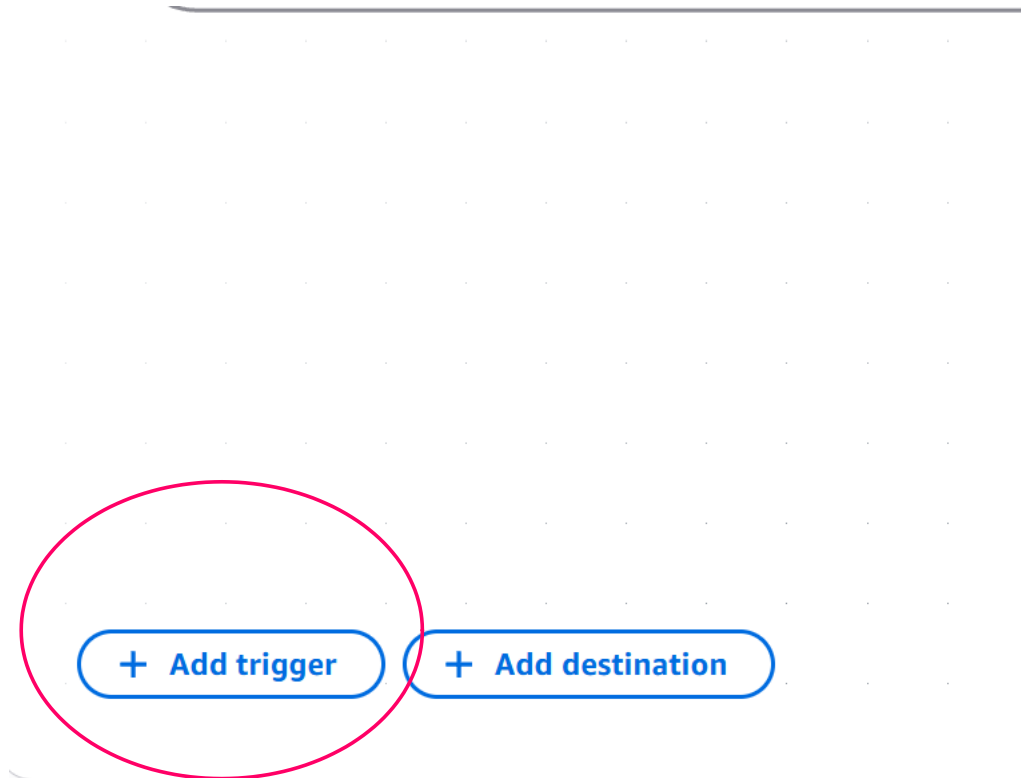


[AWSLambdaSQSQueueExecutionRole](#)

AWS managed

Provides receive message, delete mess...

Lambda Function – Add Trigger



Lambda Function – Add Trigger

Add trigger

Trigger configuration [Info](#)



SQS

aws event-source-mapping polling queue



SQS queue

Choose or enter the ARN of an SQS queue.



arn:aws:sqs:us-east-1:622997067231:telemetryqueue



Event poller configuration



Activate trigger

Select to activate the trigger now. Keep unchecked to create the trigger in a deactivated state for testing (recommended).

Provisioned mode [Info](#)

Select to configure provisioned mode for your event source mapping, which allows you to optimize the throughput of your event-driven applications. [View pricing](#).

Python Code to trigger sqs



```
Problems  Terminal 1
>
> Terminal
(awsenv) E:\BMWTWAug2026\awsaws>aws configure
AWS Access Key ID [*****ps1]): AKIAZCDME2XPTNZYOX62
AWS Secret Access Key [*****CP4s]: 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
Default region name [us-east-1]:
Default output format [None]:

(awsenv) E:\BMWTWAug2026\awsaws>aws sts get-caller-identity --region us-east-1
{
  "UserId": "AIDAZCDME2XPXV5B6IWCR",
  "Account": "622997067231",
  "Arn": "arn:aws:iam::622997067231:user/eswaribala"
}

(awsenv) E:\BMWTWAug2026\awsaws>python src/awsmodule/utils/aws_sqs.py
64c7ec5e-6806-4495-9112-59a55871218b

(awsenv) E:\BMWTWAug2026\awsaws>
```


Lambda Monitor



Code

Test

Monitor

Configuration

Aliases

Versions

Monitor

Info

Filter metrics by

Function

View CloudWatch logs

Alarm recommendations

Investigate with AI - new

3h

1d

1w

UTC timezone

Explore related

CloudWatch metrics

Lambda sends runtime metrics for your functions to Amazon CloudWatch. The metrics shown are an aggregate view of all function runtime activity. To view metrics for the unqualified or \$LATEST resource, choose **Filter metrics by**. To view metrics for a specific function version or alias, choose **Aliases** or **Versions**, select the alias or version, and then choose **Monitor**.

Invocations

Count

2

Duration

Milliseconds

2.14

Error count and success rate

Count

1

%

100

461

Lambda Monitor



CloudWatch Logs

Lambda logs all requests handled by your function and automatically stores logs generated by your code through Amazon CloudWatch Logs. To validate your code, instrument it with custom logging statements. The following tables list the most recent and most expensive function invocations across all function activity. To view logs for a specific function version or alias, visit the **Monitor** section at that level.

Recent invocations

#	: Timestamp	: RequestId	: LogStream	: DurationInMS
▼ 1	2026-09-02T23:14:52.434Z	e5a572dd-1522-55b3-b96f-11cfa53b6b26	2026/09/02/[LATEST]5173f0e9e06d40859cc9e37c0e8266f7	32.57
	@aws.account	622997067231		
	@aws.region	us-east-1		
	@billedDuration	33.0		
	@data_format	Default		
	@data_source_name	Unknown		
	@data_source_type	Unknown		
	@duration	32.57		
	@entity.Attributes.Lambda.Function	telemetrypublisher		

Lambda Monitor



CloudWatch

Favorites and recents

Ingestion

Dashboards

Alarms 0 0 0

AI Operations

GenAI Observability

Application Signals
(APM)

Infrastructure
Monitoring

Logs

Log events



Actions ▼

Start tailing

Create metric filter

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#)

Filter events - press enter to search

1m

1h



UTC timezone ▼

Display ▼



Message

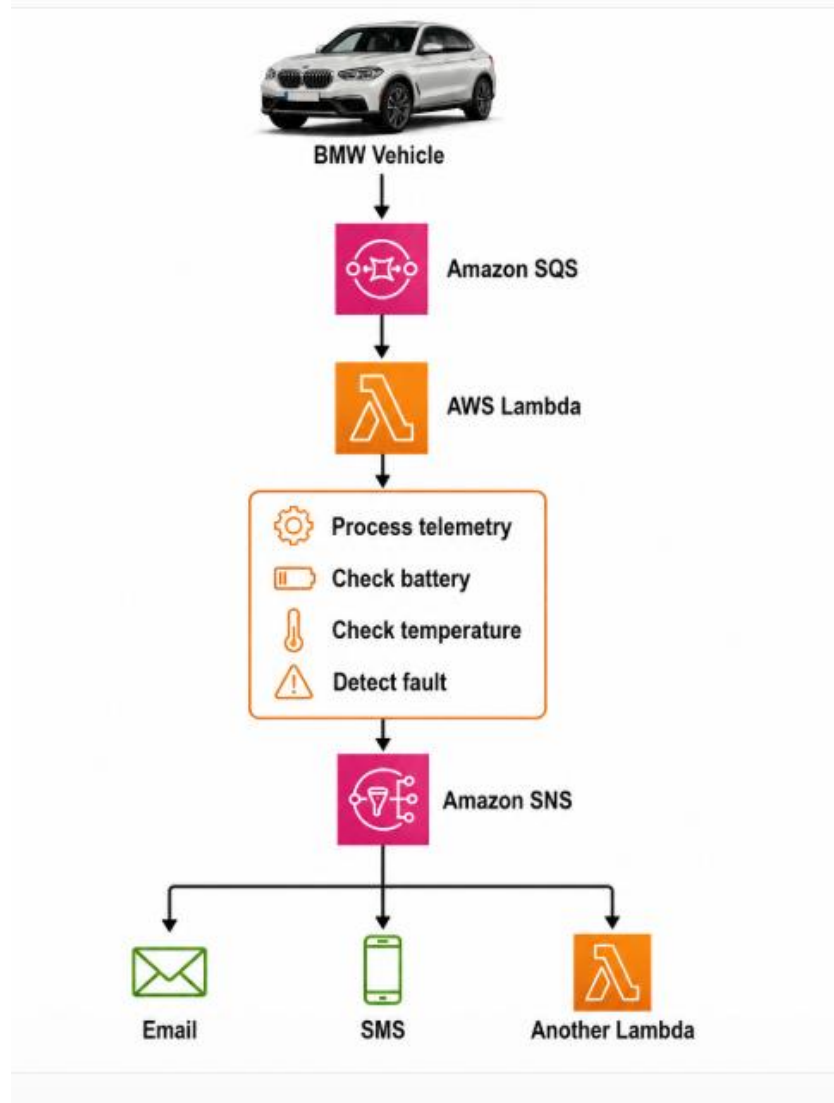
```
{
  "messageId": "64c7ec5e-6806-4495-9112-59a55871218b",
  "receiptHandle":
  "AQEBGR67YseVsrD5H/xxLzMFZd/6yH+m7pr05iVSZ+GkaWrdxQAAjbyi62sly704T14Fk6lKi0XxjYCCF+2wcFYIRfrDXx6Yk4ZCtufewxw9dfGeAvIQWp
  3b0Nt6cSbR9dPYPLkbolbp31VfDiAidClD8bHA3vxs4o02w8Lfm/3y9bJ2ioObf/+TtBcEccwVlu/+iivK3y3V5tkXyuugLj3pyEdEQgpiB+mKx6io4JZ3R
  NdB8Weh1gPGTLgda0WrdRkMyDOPng244itt9+VIB3kcHr/kw1kYRN7a2CCjEutvBjZKi7eFbjbPZCMBTQdF3aPxNBESwD2k5r8HYXcBNChjGa2K4u63Ak1s
  0YVfLINE8UGsylvGePscUY5ytxcCnBBEY5pAE/gDGingpdsUh8D6dtw==",
  "body": "{\"vehicle_id\": \"BMW1002\", \"model\": \"BMW i4\", \"battery_level\": 15, \"temperature\": 88,
  \"speed\": 70}\",
  "attributes": {
```



Amazon SNS

- **Amazon SNS (Simple Notification Service)** is a **publish/subscribe messaging service**.
- In our BMW Lambda example, it is useful for sending alerts when Lambda detects conditions such as low battery, high temperature, or fault codes.

Amazon SNS



Application Integration

Amazon Simple Notification Service

Pub/sub messaging for microservices and serverless applications.

Amazon SNS is a highly available, durable, secure, fully managed pub/sub messaging service that enables you to decouple microservices, distributed systems, and event-driven

Create topic

Topic name

A topic is a message channel. When you publish a message to a topic, it fans out the message to all subscribed endpoints.

Next step

[Start with an overview](#)

Amazon SNS



bmw-vehicle-alerts

[Edit](#)[Delete](#)[Publish message](#)

Details

Name

 bmw-vehicle-alerts

ARN

 arn:aws:sns:us-east-1:622997067231:bmw-vehicle-alerts

Display name

-

Type

Standard

Topic owner

622997067231

AWS SNS Subscription



Create subscription

Details

Topic ARN

arn:aws:sns:us-east-1:622997067231:bmw-vehicle-alerts



Protocol

The type of endpoint to subscribe

Email



Endpoint

An email address that can receive notifications from Amazon SNS.

eswaribala@zohomail.in

AWS SNS Subscription



The screenshot shows an email client interface with a sidebar on the left containing icons for Mail, Calendar, ToDo, Notes, Contacts, Bookmarks, and Web Tabs. The main pane displays a list of emails under the 'Notification' folder. One email is highlighted with a red circle:

- From: no-reply@sns.amazonaws.com
- Subject: AWS Notification - Subscription Confirmation
- Time: 5:12 AM

The email content is displayed on the right side of the interface. It includes a 'no-reply@sns.amazonaws.com' header, a '5:12 AM' timestamp, and a confirmation link. The confirmation link is circled in red:

You have chosen to subscribe to the topic:
arn:aws:sns:us-east-1:622997067231:bmw-vehicle-alerts

To confirm this subscription, click or visit the link below (If this was in error no action is necessary):
[Confirm subscription](#)

Please do not reply directly to this email. If you wish to remove yourself from receiving all future SNS subscription confirmation requests please send an email to [sns-opt-out](#)

AWS SNS Subscription



Simple Notification Service

Subscription confirmed!

You have successfully subscribed.

Your subscription's id is:

arn:aws:sns:us-east-1:622997067231:bmw-vehicle-alerts:08de901e-d294-47a0-b9a9-b6c2f6d025ad

If it was not your intention to subscribe, [click here to unsubscribe](#).



Subscriptions

Access policy

Delivery policy (HTTP/S)

Delivery status logging



Subscriptions (1)

Edit

Delete

Request confirmation

Confirm subscription

Create subscription

Search

< 1 >



ID



Endpoint



Status



Protocol



[08de901e-d294-47...](#)

eswaribala@zohom...

Confirmed

EMAIL

AWS Lambda connect to SNS



Configuration

General configuration

Triggers

Permissions

Destinations

Function URL

Environment variables

Tags

Execution role

Role name

[telemetryconsumer-role-0a8lyp5f](#)



Resource summary

To view the resources and actions that your function has permission to access



Amazon CloudWatch Logs

3 actions 3 resources

By action

By resource

Resource

AWS Lambda connect to SNS – Inline Policy

Modify permissions in sns:publish [Info](#)

Add permissions by selecting services, actions, resources, and conditions. Build permission

Policy editor

Visual

JSON

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Effect": "Allow",  
6       "Action": "sns:Publish",  
7       "Resource": "arn:aws:sns:us-east-1:123456789012:bmw-v",  
8     }  
9   ]  
10 }
```

AWS Lambda connect to SNS – Inline Policy



The screenshot displays an email client interface with a dark sidebar on the left containing icons for Mail, Calendar, ToDo, Notes, Contacts, Bookmarks, and Web Tabs. The main pane shows a 'Notification' folder with 1 unread email. The email list includes:

- no-reply@sns.amazonaws.com: BMW Low Battery Alert (5:30 AM)
- no-reply@sns.amazonaws.com: BMW Low Battery Alert (5:30 AM)
- no-reply@sns.amazonaws.com: AWS Notification - Subscription Confirmation (5:12 AM)

The selected email, 'BMW Low Battery Alert', is shown in detail on the right. It includes a 'no' status indicator, the sender 'no-reply@sns.amazonaws.com', and the recipient 'eswaribala'. The email body contains the following text:

BMW Low Battery Alert

Vehicle ID: BMW1002
Battery Level: 15%

Low battery detected.

--

If you wish to stop receiving notifications from this topic, please click or visit the link below to unsubscribe:
<https://sns.us-east-1.amazonaws.com/unsubscribe.html?SubscriptionArn=arn:aws:sns:us-east-1:622997067231:bmw-vehicle-alerts:08de901e-d294-47a0-b9a9-b6c2f6d025ad&Endpoint=eswaribala@zohoma>



- **What is Amazon EMR?**

- Amazon EMR stands for **Elastic MapReduce**.
- It is AWS's managed platform for running large-scale **big-data processing and analytics workloads**.
- EMR can process terabytes or petabytes of data using distributed computing.
- It reduces the effort required to manually install, configure, scale, and manage big-data clusters.
- EMR commonly works with data stored in **Amazon S3**.



Why EMR?

- **Why use AWS EMR?**

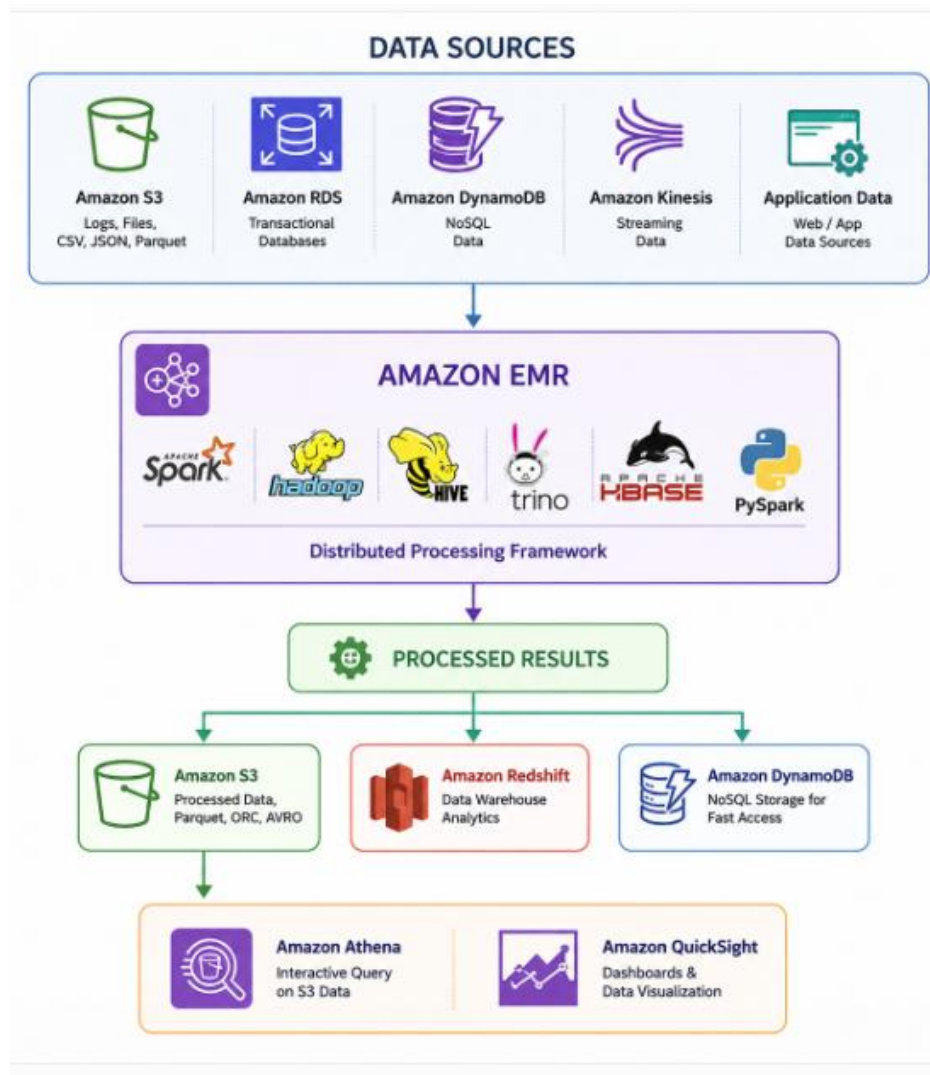
- Process very large datasets faster using parallel processing.
- Run Apache Spark and Hadoop workloads without building the complete infrastructure manually.
- Scale compute resources according to workload.
- Perform ETL, batch processing, analytics, machine learning preprocessing, and log analysis.
- Integrate with other AWS services such as S3, Glue, Athena, Redshift, DynamoDB, Kinesis, CloudWatch, and IAM.



Major frameworks supported by EMR

- **Apache Spark** – distributed processing, ETL, analytics, machine learning.
- **Apache Hadoop** – distributed storage and MapReduce processing.
- **Apache Hive** – SQL-style querying over large datasets.
- **Trino / Presto** – interactive distributed SQL queries.
- **Apache HBase** – NoSQL database for large datasets.
- **Apache Flink** – stream-processing workloads in supported EMR environments.
- **Jupyter / notebooks** – interactive Spark development and analysis.

EMR Architecture





EMR deployment options

- **EMR on EC2**
 - Traditional EMR cluster.
 - You choose EC2 instance types and number of nodes.
 - Provides maximum configuration control.
 - Suitable for long-running and complex Spark/Hadoop environments.



EMR deployment options

- **EMR Serverless**

- No EC2 cluster administration.
- AWS automatically provisions resources for the job.
- Good for Spark and Hive workloads.
- Suitable for intermittent ETL and data-processing jobs.



EMR deployment options

- **EMR on EKS**
 - Runs EMR workloads on Amazon EKS.
 - Suitable when an organization already uses Kubernetes.
 - Allows Spark workloads and other Kubernetes applications to share infrastructure.



EMR versus AWS Glue

AWS EMR	AWS Glue
Big-data processing platform	Serverless data-integration/ETL service
More infrastructure and framework control	More managed experience
Spark, Hadoop, Hive, Trino, HBase, etc.	Primarily ETL/data integration
Suitable for complex big-data workloads	Suitable for managed ETL pipelines
Supports long-running processing environments	Commonly job-oriented
Cluster/serverless/EKS choices	Serverless service



EMR versus AWS Athena

EMR	Athena
Processing platform	SQL query service
Spark/Hadoop/Hive etc.	SQL
Good for transformation	Good for querying S3
Can run complex pipelines	No cluster management
Handles large distributed processing	Interactive/ad-hoc analytics



EMR versus AWS Redshift

EMR	Redshift
Big-data processing engine/platform	Data warehouse
Spark/Hadoop workloads	SQL analytics
ETL and transformation	BI/reporting
Often reads/writes S3	Stores structured warehouse data
Flexible processing	Optimized analytical warehouse



When to use EMR

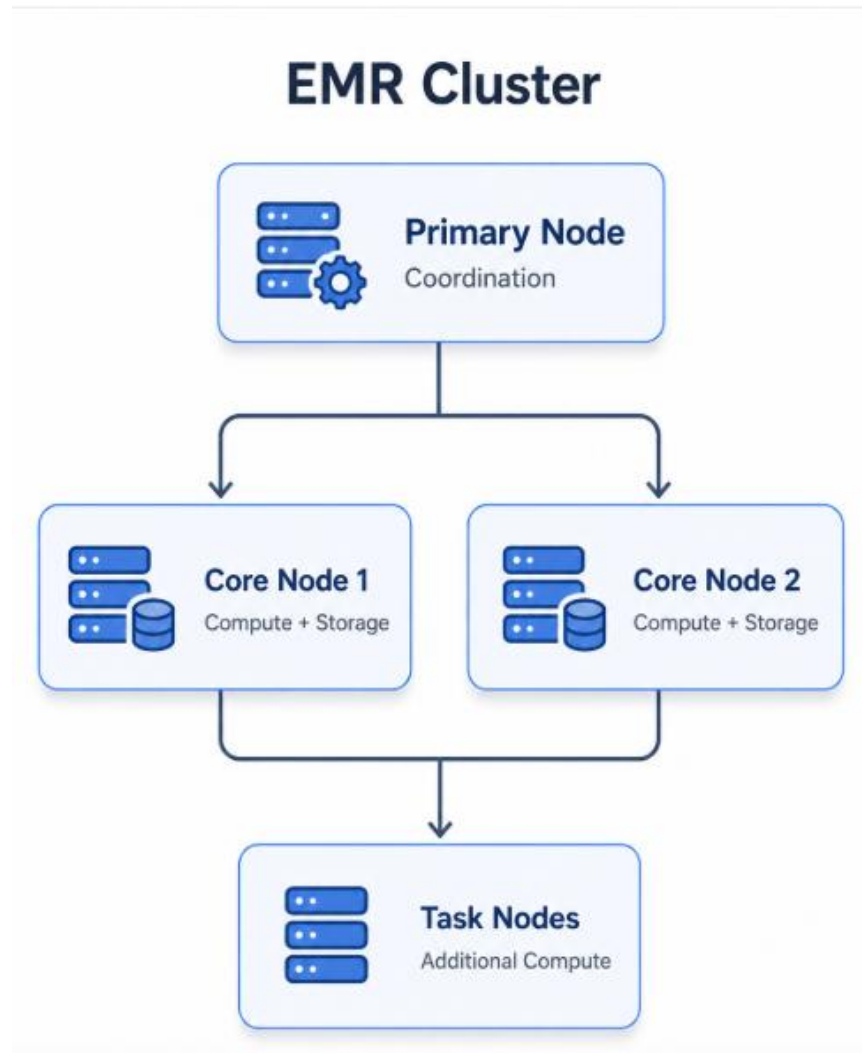
- Very large datasets.
- Complex Spark applications.
- Existing Hadoop/Spark applications.
- Need control over Spark configuration.
- Need multiple open-source big-data frameworks.
- Large-scale machine-learning data preparation.
- Complex data engineering pipelines.
- Migration of existing Hadoop workloads to AWS.



When Not to use EMR

- Small datasets that can be processed by Lambda.
- Simple SQL queries over S3 where Athena is sufficient.
- Straightforward managed ETL where AWS Glue is easier.
- Traditional BI warehouse workloads better suited to Redshift.

Amazon EMR Cluster Design, Sizing & Autoscaling





Amazon EMR Cluster Design, Sizing & Autoscaling

- **Primary Node**

- Controls and coordinates the EMR cluster.
- Runs cluster-management services.
- Tracks jobs running on the cluster.
- Monitors the health of Core and Task nodes.
- Runs services such as:
 - YARN ResourceManager
 - HDFS NameNode
 - Spark coordination components
- Normally does **not perform the bulk of distributed processing**.
- Every EMR cluster requires a Primary node.
- The Primary node is **not scaled by EMR managed scaling**.



Amazon EMR Cluster Design, Sizing & Autoscaling

Primary

- |
- | — Receive Spark job
- | — Divide work
- | — Allocate resources
- | — Monitor executors
- | — Track job completion

Amazon EMR Cluster Design, Sizing & Autoscaling

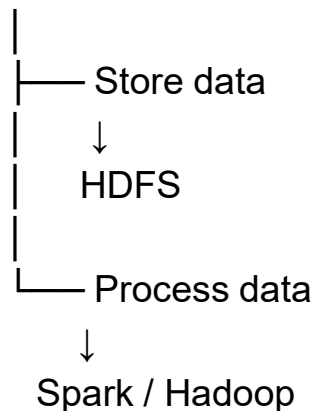
- **High-availability Primary nodes**
- For critical production environments, EMR supports configurations with **three Primary nodes**, allowing important components to fail over if a Primary node becomes unavailable.
- Conceptually:
 - Primary 1
 - Primary 2
 - Primary 3
 - ↓
 - Core / Task Nodes
- This is useful for long-running production clusters where availability matters.

Amazon EMR Cluster Design, Sizing & Autoscaling

- **2. Core Nodes**

- Core nodes have **two main responsibilities**:

- Core Node

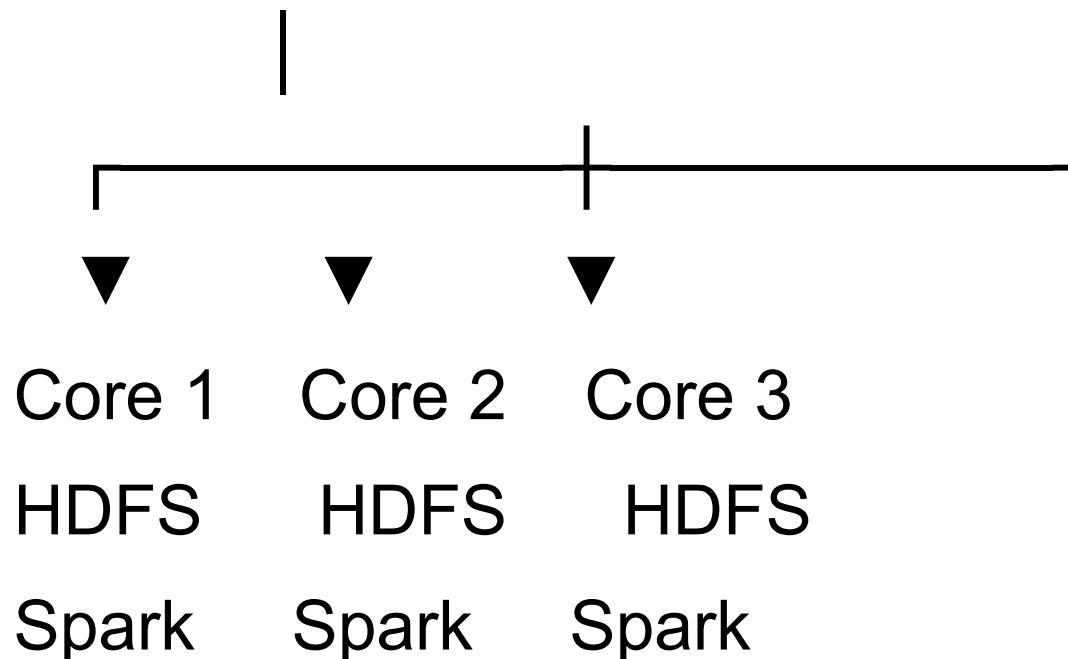


- Core nodes typically run:
 - HDFS DataNode
 - YARN NodeManager
 - Spark executors
 - Hadoop MapReduce tasks
- Therefore:
 - **Core Node = Compute + HDFS Storage**

Amazon EMR Cluster Design, Sizing & Autoscaling

- **2. Core Nodes**
- Suppose BMW telemetry is temporarily stored in HDFS:

Primary



Amazon EMR Cluster Design, Sizing & Autoscaling

- **3. Task Nodes**

- Task nodes provide **compute capacity only**.

- Task Node

- |
- | — Spark Executor
- | — MapReduce Tasks
- | — YARN Containers

- No HDFS DataNode

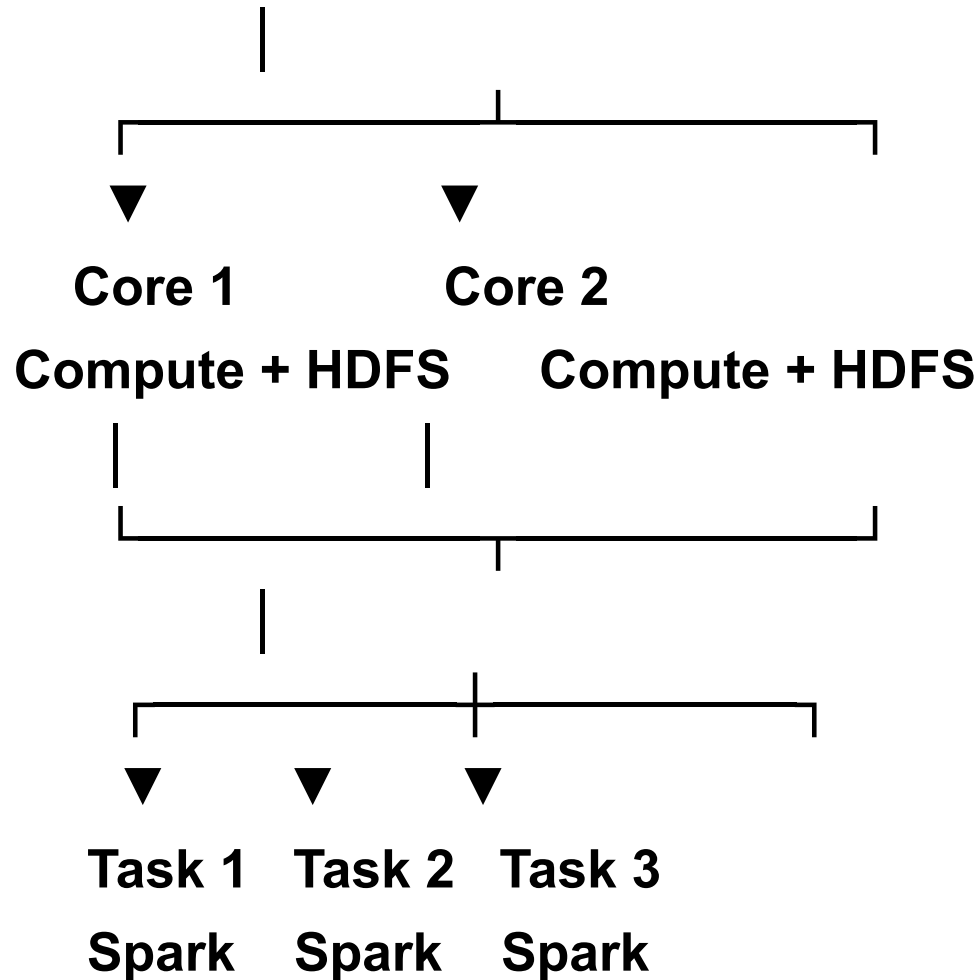
- Therefore:

- **Task Node = Compute only**

- Task nodes don't store HDFS data, making them particularly useful for elastic scaling.

Amazon EMR Cluster Design, Sizing & Autoscaling

Primary



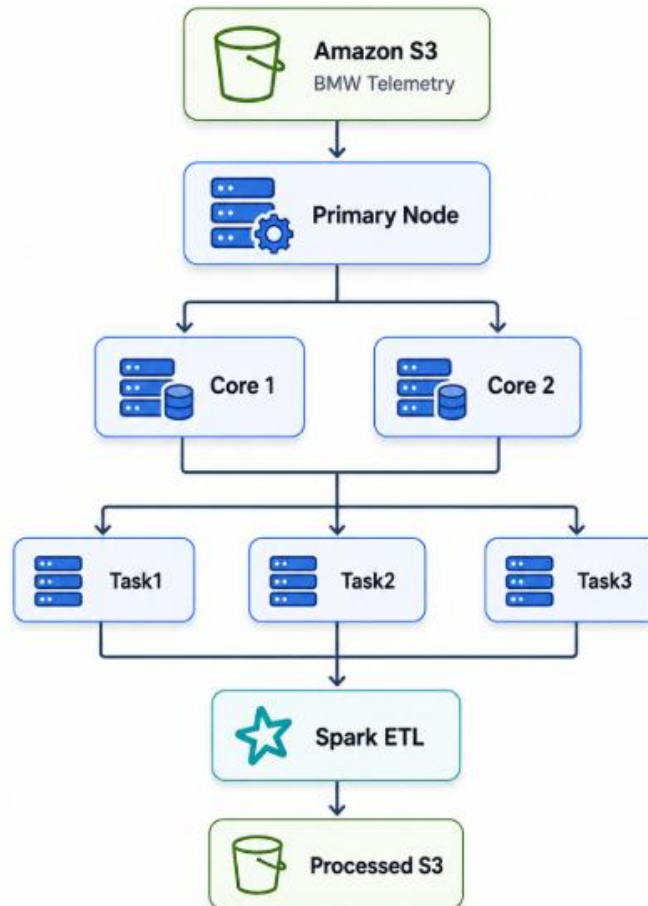
4. Easy way to remember the node roles

Node	Management	Processing	HDFS Storage	Scalable
Primary	✓	Limited	Metadata/coor dination	✗ Managed scaling
Core	✗	✓	✓	✓
Task	✗	✓	✗	✓

Example EMR Cluster



BMW Telemetry Processing with EMR



EMR Cluster



- Refer doc for Hands on